

基于因果滚动优化的微网购电与储能调控

摘要

针对含光伏和电化学储能的并网微网购电调度问题，本文建立一套可执行、可审计的因果滚动优化模型。我们先把 10 min 功率数据换算为分段电量，用线性规划描述外网购电、储能充放电、弃光和紧急购电，并以显式能量平衡保证每段供需一致。随后针对负载、光伏和电价的不确定性，限定每次决策只能读取当时已经发布的预报和此前实际观测，以过去四个同星期日的曲线预测负载和波动电价。在 0、6、12、18 时采用 24 h 跨日滚动窗口，只执行至下一次决策；下一日计划在午夜前仅作预览。为减轻短视窗末端放空和光伏预报系统偏差，进一步引入经一月滚动验证选择的终端 SOC 规则、按发布时间与提前期分组的历史残差均值校正，以及保留时间相关性的历史残差轨迹场景。

统一以 2025 年 2 月 1 日 6000 kWh 为正式起点，并要求 12 月 31 日回到 6000 kWh。问题一的不确定性模型得到全天购电量 59,482.6990 kWh、购电费 35,126.95 元。问题二的因果日前策略总费用为 17,646,794.82 元，完全信息对照为 12,230,383.65 元。问题三最终采用 all_three，其 S0-S3 费用变化见正文；问题四在波动电价下得到同口径结果。六组消融、逐月配对差和 7 天移动块区间用于分离三项增强的作用。对 S2 与 S3 还采用三种 HiGHS 方法和三种并列破除系数重复求解；稳定性结论为 数值不可区分 和 数值不可区分。全量审计覆盖字段、日期、每天 144 个唯一时段、信息边界、费用和 SOC 连续性，新版正式结果的失败项为 0。

本文的贡献在于把完全信息最优值与可执行策略分开报告，并用决策版本链证明滚动策略没有读取未来实际量。模型保持线性且可由统一流水线从原始附件重建，适合在题目给定的高频数据和严格提交格式下复核。

关键词：微网调度；储能优化；模型预测控制；随机规划；因果回测；光伏预测

问题重述

微网包含小区负载、光伏发电和储能设备，并可从外部电网购电。储能容量上限为 12000 kWh，运行范围为 1200-10800 kWh，最大充放电功率为 5000 kW，充放电效率均为 90%。题目要求在满足负载的前提下减少购电费用，并依次考虑固定日型、全年实际波动、日内多次光伏预报和实时波动电价。

问题一给定固定电价、固定负载和一天光伏预测，需要在每天 0 点形成确定性计划，并令 0 点与 24 点储电量相同。问题二引入全年变化的负载和光伏，计划不足时以交易时刻电价的 5 倍紧

急购电。问题三允许在 6、12、18 点依据更新的 24 h 光伏预报调整尚未发生的购电计划，并对上调和下调收取不同费用。问题四把固定电价替换为十分钟实时电价，重新研究问题二和问题三。

题目同时提出一个实际建模难点：附件 2 给出的是事后实际负载和光伏，附件 4 给出的是事后实际电价，但并未提供每天 0 点可用的负载与价格预报。若规划器直接读取当天实际值，结果只能作为完全信息对照，不能代表现实中可执行的日前策略。本文因此把因果策略作为正式主结果，完全信息模型只用于衡量信息价值。

问题分析

数据时间轴

附件 1、2、4 的日内时间戳为 00:10,00:20,...,24:00。为了覆盖完整物理日，本文把第一个值解释为区间 $[00:00,00:10)$ ，即附件时间戳是区间右端点。官方结果模板的标签保持原样，不对数组作整体平移；程序另存源时间戳、物理区间和官方标签三者的映射。

功率数据单位为 kW，而决策量和储电量单位为 kWh。每段长度为 $\Delta t = 1/6 \text{ h}$ ，因此负载和光伏分段电量分别为 $l_t = L_t \Delta t$ 与 $g_t = G_t \Delta t$ 。所有能量平衡在 kWh 口径下建立。

决策层次

问题一是单日确定性 LP。问题二和问题四的日前部分需要先根据历史信息形成预测计划，再用实际量执行和结算。问题三和问题四的日内部分属于滚动时域控制：每次更新从当前真实 SOC 出发，冻结已经执行的时段，只优化未来 24 h，并执行最近的 6 h。

跨日预测的作用主要体现在储能状态。18 点之后若只看到当天剩余 6 h，优化器容易把储能末端价值处理得过低；加入下一日预览后，模型能够权衡当晚放电与次日早间需求。下一日的购电量在午夜前并非正式承诺，不能直接沿用到次日结果表。

不确定性与检验

点预测可能存在随发布时间和提前期变化的系统偏差。本文用最近 28 天同组残差均值校正光伏预报，并以最近 60 天相同时钟小时的实际光伏 99.5% 分位数限制异常值。为表示误差轨迹的时间相关性，本文从最近最多 20 条完整历史残差轨迹中建立等概率场景。第一个 6 h 块跨场景一致，之后允许场景递补。

所有增强必须通过同口径消融验证。本文固定六组实验，并用逐日配对差、逐月差和 7 天移动块区间报告不确定性。对于只有分币级差异的 S2 与 S3，必须由不同求解方法和并列破除系数共同验证；否则只报告数值不可区分。

模型假设

1. 微网按单节点能量系统处理，忽略线路潮流、无功功率和微网内部输电损耗。
2. 储能充电效率和放电效率均为 0.9，容量与功率限制在全年不变，不计温度、老化和循环寿命成本。
3. 光伏可弃用。规划模型中的弃光不得超过当段预测光伏；执行后的“未用能量”另行记录冻结购电承诺、储能动作与实际净负荷之间形成的富余，不能反向解释为可售电量。
4. 外网购电非负，题目不允许把微网余电按负购电量出售。
5. 负载、光伏和价格的实际值只在相应物理时段执行后可见。规划阶段只能读取此前观测和当时已发布预报。
6. 问题三主结算按最终调整量相对 0 点计划计算；相邻版本逐次结算另作敏感性分析。
7. 正式评估从 2025 年 2 月 1 日开始，初始 SOC 为 6000 kWh；1 月只用于训练和参数选择。2025 年 12 月 31 日 24 点 SOC 回到 6000 kWh。
8. LP 中不设置充放电互斥二进制变量。采用极小正并列破除费用选择低吞吐最优解，并在全量审计中要求同时充放次数为 0。

符号说明

符号	含义	单位
t	十分钟物理时段索引	无
d	日期索引	无
τ	决策时刻，取 0、6、12、18 时	h
Δt	单段时长，取 1/6	h
p_t	交易时刻实际电价	元/kWh
$p_{\text{hat}}(t \text{ given } \tau)$	决策时刻可用的预测电价	元/kWh
l_t, g_t	实际负载、实际光伏分段电量	kWh
$l_{\text{hat}}(t \text{ given } \tau), g_{\text{hat}}(t \text{ given } \tau)$	可用信息下的负载、光伏预测电量	kWh
B_t	0 点形成的计划购电量	kWh
A_t	当前版本或最终版本的承诺购电量	kWh
U_t, V_t	相对 0 点计划的下调量和上调量	kWh
Q_t	实际执行中的紧急购电量	kWh
C_t, D_t	储能入口充电量、出口放电量	kWh
E_t	时段结束时的储电量	kWh
W_t	实际未用能量	kWh
η	充电和放电效率，取 0.9	无
$E_{\min}, E_{\max}$	SOC 下限和上限，1200 与 10800	kWh
$P_{\max}$	最大充放电功率，5000	kW

符号	含义	单位
omega	历史残差场景索引	无
pi_omega	场景概率	无

基础能量模型

能量平衡

对于任一执行时段，微网的实际能量平衡为

$$A_t + Q_t + g_t + D_t = I_t + C_t + W_t.$$

储能状态转移为

$$E_t = E_{(t-1)} + \eta C_t - D_t/\eta.$$

约束为

$$\begin{aligned} E_{\min} &\leq E_t \leq E_{\max}, \\ 0 &\leq C_t, D_t \leq P_{\max} \Delta t, \\ A_t, Q_t, W_t &\geq 0. \end{aligned}$$

C_t 是进入电池前的交流侧电量，充电后只有 ηC_t 写入 SOC； D_t 是电池向交流侧输出的电量，SOC 因此减少 D_t/η 。

预测计划与实际执行

规划器把 I_t, g_t, p_t 替换为决策时刻可见的预测值，求出承诺购电和储能动作。时段结束后按实际 I_t, g_t, p_t 结算。若固定的承诺和储能动作不足以满足实际负载，则

$$\begin{aligned} Q_t &= \max(I_t + C_t - A_t - g_t - D_t, 0), \\ W_t &= \max(A_t + g_t + D_t - I_t - C_t, 0). \end{aligned}$$

这两个变量不会同时为正。紧急购电费用为 $5 p_t Q_t$ 。规划过程不能通过读取未来实际值提前消除 Q_t 。

问题一确定性计划

问题一的电价、负载和光伏日型已知。以 B_t, C_t, D_t, E_t, W_t 为变量，求解

$$\min \sum_t p_t B_t + \epsilon \sum_t (C_t + D_t)$$

满足

$$\begin{aligned} B_t + g_t + D_t &= I_t + C_t + W_t, \\ E_t &= E_{(t-1)} + \eta C_t - D_t/\eta, \\ E_0 &= E_{144} = 6000, \\ 0 &\leq W_t \leq g_t, \end{aligned}$$

以及容量、功率和非负约束。 ϵ 只用于破除退化并列解，不计入报告购电费。

问题一的主要结果如下。指定时段按官方模板位置抽取，但方法部分已经披露右端点物理解释。

指标	结果
全天购电量	59,482.6990 kWh

指标	结果
全天购电费	35,126.95 元
无储能购电费	48,052.05 元
费用降低比例	26.90%
0 点和 24 点储电量	6000 kWh 和 6000 kWh

题面指定时段和四小时储能汇总列入附录指定日期结果表。

问题二因果日前策略

历史同星期日预测

对正式日期 d ，设 H_d 是日期 d 以前最近四个、星期几相同的日期集合。负载和光伏基线为

$$l_hat_d(d,t \text{ given } 0) = \text{mean}\{l_h(h,t): h \in H_d\},$$

$$g_hat_d(d,t \text{ given } 0) = \text{mean}\{g_h(h,t): h \in H_d\}.$$

第一次正式决策只使用 1 月历史。随着日期推进，模型可以使用已经发生的 2-12 月观测，但窗口始终只向后看。该方法是移动窗口季节均值，不等同于直接使用同一日型或未来实际量。

日前优化和结算

每天 0 点按预测量求基础 LP，形成 B_t, C_t, D_t 。执行时保持这些动作，预测误差由紧急购电和未用能量平衡。正式评估的日末 SOC 直接传给下一日，不人为重置；除最后一天强制回到 6000 kWh 外，每个日窗口采用一月选中并写入配置的终端规则。这样既保持跨日状态连续，又避免短时域无代价放空。

因果日前策略的正式总费用为

$$Z_2 = \sum_t p_t B_t + 5 * \sum_t p_t Q_t.$$

完全信息对照使用相同的日期、SOC 端点、物理约束和费用，只把规划输入替换为实际负载与光伏。它用于估计信息差距，不作为正式 result2.xlsx 的策略。

情景	总费用 元	紧急购电量 kWh	定位
因果日前策略	17,646,794.82	1,376,680.8095	正式结果
完全信息对照	12,230,383.65	0.0000	事后参照

指定日期 2025-03-20、06-21、09-23、12-21 的计划、四小时充放电和紧急购电连续区间由全精度明细自动聚合。只有当紧急购电量四位小数显示后仍大于 0 时才写入结果表，避免把正的小量静默删除。

问题三日内滚动调整

调整费用

令 B_t 是 0 点正式计划， A_t 是某次更新后的承诺，定义

$$\begin{aligned} A_t - B_t &= V_t - U_t, \\ U_t, V_t &\geq 0. \end{aligned}$$

主口径的时段费用为

$$z_t = p_t \min(B_t, A_t) + 0.5 p_t U_t + 1.5 p_t V_t + 5 p_t Q_t.$$

等价写法为

$$z_t = p_t B_t - 0.5 p_t U_t + 1.5 p_t V_t + 5 p_t Q_t.$$

下调意味着取消相应原计划购电，但另付被取消部分 50% 的违约金；上调部分按 1.5 倍计价。该解释与“原计划费用不退、再付违约金”不同，本文在模型约定中明确披露。

若相邻承诺版本为 $A_t^0=B_t, A_t^1, \dots, A_t^K$ ，逐次交易敏感性费用为

$$\begin{aligned} z_{t_seq} &= p_t B_t \\ &\quad - 0.5 p_t \sum_k \max(A_t^{k-1} - A_t^k, 0) \\ &\quad + 1.5 p_t \sum_k \max(A_t^k - A_t^{k-1}, 0) \\ &\quad + 5 p_t Q_t. \end{aligned}$$

主结果和逐次结果使用完全相同的决策链，仅改变结算规则。

跨日滚动窗口

每个决策时刻 τ 构造从当前时刻开始的未来 24 h 预测。规划器以当前实际 SOC 为初始状态，冻结所有 $t < \tau$ 的购电、充放电和结算，只实施 $[\tau, \tau+6h)$ 的决策。到下一时刻后，以新观测和新预报重算。

0 点对当天 144 段形成正式计划。6、12、18 点只能调整当天尚未发生段。窗口进入次日后的变量只用于估计储能后续价值，不写入次日正式承诺；次日 0 点重新形成该日计划。

光伏偏差校正

对发布时间 r 和提前小时 h ，定义历史残差

$$e_{-}(d, r, h) = g_{\text{actual}}(d, r, h) - g_{\text{raw_forecast}}(d, r, h).$$

使用决策日期以前最多 28 天同组残差均值 $b_{\text{hat}}(r, h)$ ，得到

$$g_{\text{bias_corrected}} = \text{clip}(g_{\text{raw_forecast}} + b_{\text{hat}}(r, h), 0, q_{0.995}).$$

其中 $q_{0.995}$ 是此前最多 60 天相同时钟小时实际光伏的 99.5% 分位数。所有历史日期严格早于当前决策日。

历史残差场景

对当前发布时间，从历史中取最近最多 20 条完整残差轨迹 e^{ω} ，构造

$$g_{\text{hat}}^{\omega} = \text{clip}(g_{\text{corrected}} + e^{\omega}, 0, \text{cap}_t).$$

场景等概率。第一执行块的 A_t, C_t, D_t 对所有 ω 相同；后续变量允许按场景变化。目标为各场景预测费用的算术平均，不加入方差、CVaR 或最坏情形项。

官方模板要求 0 点保存当天完整计划，而随机模型在第一个 6 h 之后给出的是场景递补，不能把任一分支提前当成正式承诺。因此，0 点同时用相同的校正点预测求一个确定性全日计划；该计划填充首个 6 h 之后的 B_t，随机模型的跨场景共同解替换首个 6 h。未来场景分支只用于评估当前动作的后续成本，并保存在证据文件中。后续每次更新也只把下一执行块的共同决策写入承诺链。

终端规则

一月滚动验证比较每日闭环、24 h 首尾 SOC 相同和软目标三类方案。软目标在目标函数中加入

$\lambda \cdot \text{abs}(E_{\text{window_end}} - 6000)$,

其中 λ 依次取预测价格中位数的 0.5、1、1.5 倍。正式规则 soft_target 在进入 2 月前冻结。

消融和更新频率结果

六组消融结果如下。

模式	实验组	总费用 元	紧急购电 kWh	未用能量 kWh	
Q3	原始预测基线	18,574,252.52	1,413,346.2686	2,558,486.5155	
Q3	仅跨日终端	18,576,083.56	1,413,415.1060	2,557,270.7766	
Q3	仅偏差校正	17,444,035.34	1,195,902.3765	2,253,007.8124	
Q3	偏差校正+跨日	17,453,613.68	1,195,920.5004	2,252,522.5930	
Q3	场景+跨日	16,799,379.99	890,470.4724	2,549,720.1403	
Q3	三项组合	16,662,138.66	914,394.1682	2,513,557.3866	
Q4-3	原始预测基线	19,544,333.76	1,417,682.8332	2,552,749.6324	
Q4-3	仅跨日终端	19,535,919.79	1,419,008.6493	2,548,860.6348	
Q4-3	仅偏差校正	18,341,838.33	1,196,044.7137	2,250,598.2059	
Q4-3	偏差校正+跨日	18,331,075.19	1,195,864.6816	2,249,422.3261	
Q4-3	场景+跨日	17,684,597.00	889,632.4055	2,553,466.8915	
Q4-3	三项组合	17,539,062.52	913,745.4818	2,514,319.4789	正文只保留可由控制实验支持的结论：

Q3 中，三项组合相对原始因果基线费用减少 1,912,113.85 元，紧急购电量减少 498,952.1004kWh。

最终组合模型的 S0-S3 结果如下。

策略	更新时间	主结算总费用 元	逐次结算总费用 元	紧急购电量 kWh	相对 S0 差额 元
S0	0	16,478,098.00	16,478,098.00	834,315.1721	0
S1	0 6	16,738,159.37	16,244,607.08	900,953.5459	260,061.37
S2	0 6 12	16,663,100.44	16,453,262.40	914,743.7679	185,002.43
S3	0 6 12 18	16,662,138.66	16,457,572.00	914,394.1682	184,040.66

S2 与 S3 的主费用差为 -961.77 元，7 天移动块区间为 -1,439.13 至 -573.36 元。多求解器判定为 数值不可区分。因此本文的限定结论是：

S2 与 S3 的差异在多求解器复算中数值不可区分，不据此主张 18 点更新有稳定收益。

问题四波动电价

因果价格预测

附件 4 是实际实时价格，不能在当天 0 点全部读入规划器。对正式日期 d，价格预测取此前最近四个同星期日的同位置平均：

$$p_hat_d(t \text{ given } \tau) = \text{mean}\{p_h(t): h \text{ in } H_d\}.$$

若 $\tau > 0$ ，已发生价格可用于结算，但尚未发生价格仍由历史预测。Q4-2 每天 0 点形成一次计划；Q4-3 按问题三四个时刻滚动更新。实际 p_t 只用于相应时段发生后的最终费用。

问题四之二

Q4-2 的物理模型与问题二相同，只把规划价格改为 p_hat ，执行费用用实际 p_t 。正式因果结果和完全信息对照为：

情景	总费用 元	紧急购电量 kWh	定位
因果波动电价日前策略	18,660,362.33	1,376,708.3702	正式结果
完全信息价格对照	12,783,021.74	0.0000	事后参照

问题四之三

Q4-3 使用与问题三相同的跨日、偏差校正和场景结构，区别是规划价格为因果预测价格，结算价格为实际实时价格。最终选中组别为 all_three。

策略	更新时间	主结算总费用 元	逐次结算总费用 元	紧急购电量 kWh	相对 S0 差额 元
S0	0	17,360,605.13	17,360,605.13	835,546.2767	0
S1	0 6	17,622,935.18	17,109,516.07	899,866.3753	262,330.05
S2	0 6 12	17,539,727.92	17,332,483.85	914,049.6888	179,122.79

策略	更新时间	主结算总费用 元	逐次结算总费用 元	紧急购电量 kWh	相对 S0 差额 元
S3	0 6 12 18	17,539,062.52	17,337,192.33	913,745.4818	178,457.39

Q4-3 的 S2 与 S3 主费用差为 -665.40 元，7 天移动块区间为 -1,115.13 至 -263.82 元；

多求解器判定为 数值不可区分。限定性解释为：

S2 与 S3 的差异在多求解器复算中数值不可区分，不据此主张 18 点更新有稳定收益。

求解流程

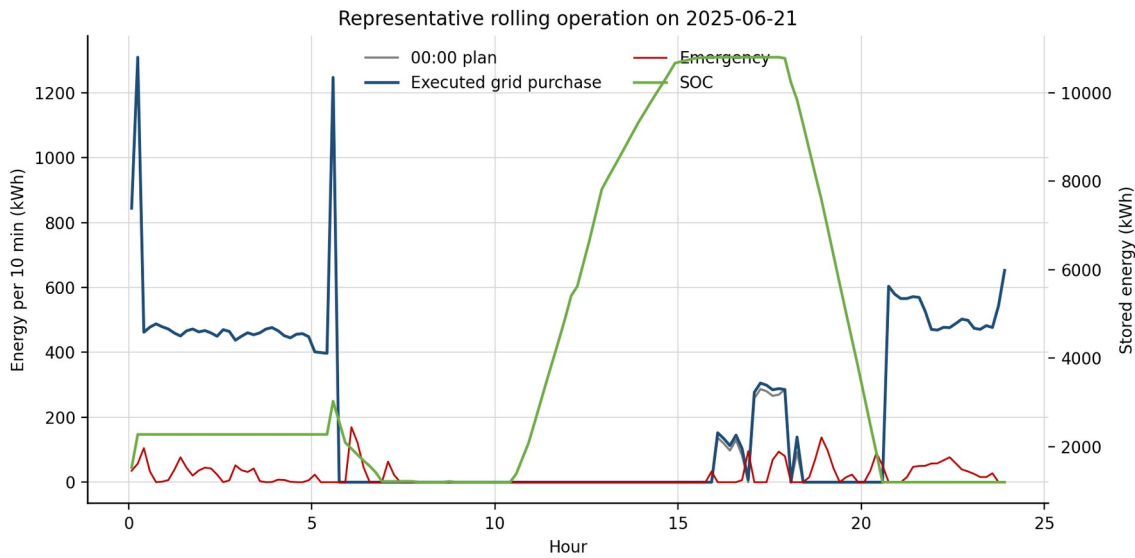


图 代表日的计划购电、最终执行、紧急购电与 SOC 轨迹

统一流水线按以下顺序运行：数据模式检查、时间映射、滚动预测、优化求解、实际执行、费用结算、全量审计、结果冻结、工作簿和图表生成、论文注入与最终打包。

每次滚动求解执行以下步骤。

- 9. 根据决策时刻建立 24 h 有效时段索引。
- 10. 读取不晚于当前时刻的光伏预报，并从严格早于决策日的历史生成负载和价格预测。
- 11. 若实验组启用偏差校正，则只用历史残差校正光伏。
- 12. 若实验组启用场景，则构造最近最多 20 条完整残差轨迹，并添加第一块非预见性约束。
- 13. 从执行模拟器读取当前 SOC，冻结过去动作和当天已有承诺。
- 14. 求解 LP，保存求解状态、残差、版本链和配置哈希。
- 15. 仅执行到下一决策时点，使用实际量计算紧急购电、未用能量和实际费用。

16. 更新 SOC 并进入下一决策时点。

实现的单一入口为：

```
bash run_pipeline.sh --config config/final.yaml
```

结果分析

六组消融

固定电价和波动电价下的 A0-A5 均使用相同信息边界和端点。最终结果应按“候选方案减基线”的符号报告，并回答每项增强是否同时改变总费用、紧急购电和未用能量。

Q3 中，三项组合相对原始因果基线费用减少 1,912,113.85 元，紧急购电量减少 498,952.1004kWh。Q4-3 中，三项组合相对原始因果基线费用减少 2,005,271.24 元，紧急购电量减少 503,937.3514kWh。

[[FIGURE:fig1_ablation_cost.png|六组增强消融的总费用比较]]

[[FIGURE:fig2_update_schedule.png|最终组合模型的 S0-S3 更新频率比较]]

如果组合效果不等于各单项效果之和，应从储能状态和紧急购电触发时段解释交互，而不能把差异简单归因于某个模块。

月度差异与区间

逐月差异表和 7 天移动块区间用于检查全年平均是否由少数月份驱动。正式分析为：

逐月差异未采用跨月平均替代，各月均从逐日配对费用计算。完整点估计和 7 天移动块区间见冻结比较表；区间跨 0 时不作显著性表述。

[[FIGURE:fig4_monthly_effects.png|三项组合相对原始因果基线的逐月费用差和移动块区间]]

当区间跨 0 时，只报告点估计方向，不使用“显著降低”一词。

结算规则敏感性

最终相对 0 点计划和逐次交易的费用差为：

Q3 S3 的逐次结算比主口径高 -204,566.66 元；Q4-3 S3 相差 -201,870.19 元。

[[FIGURE:fig3_cost_components.png|最终组合策略的费用分项]]

该差异衡量制度解释，不是优化器误差。主结果采用前者是为了匹配官方模板只保存 0 点计划和最终调整量的结构；决策版本日志使后者能够独立复算。

完全信息差距

问题二和 Q4-2 的完全信息对照与因果策略使用同一日期、物理约束、费用函数和 6000 到 6000 kWh 端点。因此两者差额可解释为未来负载、光伏和价格信息的价值上界之一。它不是可实现收益承诺，也不能与信息集合不同的旧结果直接比较。

Q2 因果策略比完全信息对照高 5,416,411.17 元；Q4-2 的对应差额为 5,877,340.59 元。

模型检验

物理可行性

新版结果共覆盖 334 个正式日期，每天 144 个唯一时段。全量重算的最大能量平衡残差为 $9.095\text{e-}13$ kWh，最大 SOC 动态残差为 $4.547\text{e-}12$ kWh，SOC 范围为 1,200.0000 至 10,800.0000 kWh，同时充放次数为 0。

因果性

decision_version 共记录 384336 条承诺版本。历史日期不早于决策日的记录数为 0，预报发布时间越界数为 0，过去时段改写数为 0，承诺链不一致数为 0。

数值稳定性

HIGHS 自动、对偶单纯形和内点法，以及三种并列破除系数的 S2/S3 复算表由 frozen/stability_summary.csv 注入。各次求解均保存全精度目标和可行性残差。

[[FIGURE:fig6_stability.png|S2 与 S3 的多求解器和并列破除系数复算]]任何方法间方向或分币结果不一致都被归为数值不可区分。

工作簿一致性

五份工作簿由全精度冻结结果生成，四位小数只用于显示。紧急购电区间按每日连续正值段重建，只省略四位小数显示后为 0 的记录。指定日期、四小时充放电、论文数字和工作簿汇总均由同一键值源生成。新版审计结果为 PASS 81 项、WARN 0 项、FAIL 0 项；旧成果仍保留 FAIL 1 项和不可验证 4 项。

模型评价

优点

第一，模型把决策信息与事后实际量分离。每次决策均保留历史截止日和预报发布时间，可以直接审计是否读取未来信息。

第二，跨日 24 h 窗口保持 SOC 连续，并允许下一日预览影响当前储能动作，同时不提前形成下一日承诺。该设计兼顾储能的跨日物理作用和题目规定的每日 0 点正式计划。

第三，三项增强通过固定的六组消融分别检验。历史残差整轨迹保留样本内时间相关，第一块非预见性约束避免场景模型根据尚未发生的情形选择当前动作。

第四，结果采用全精度单一来源。论文、图表、五份工作簿、审计报告和 SHA-256 清单由同一流水线生成，减少手工复制和舍入不一致。

局限

第一，负载和价格预测采用最近四个同星期日均值，透明且因果，但不能利用天气、节假日等外生变量。题目没有提供这些数据，本文不额外引入来源难以审计的信息。

第二，历史场景数最多为 20，早期决策可用样本更少。经验轨迹重采样不保证覆盖所有极端误差，也没有估计连续概率分布。

第三，模型忽略线路、无功功率、电池老化和寿命成本。结论只适用于题面给定的单节点能量模型。

第四，主结算口径来自对题目和模板结构的明确解释。逐次结算可能产生不同费用，因此本文并报两种结果，不把制度差异解释为策略收益。

第五，7 天移动块区间依赖历史序列近似可重采样的假设。334 天样本包含季节变化，区间只能作为历史依赖下的经验不确定性描述。

结论

本文从确定性日调度出发，建立了因果日前计划和跨日滚动调整模型，并在波动电价下保持相同的信息边界与物理约束。正式结果表明：

1. 确定性 LP 能在题设边界内完成日内削峰填谷。
2. 因果日前结果与完全信息结果存在明确的信息差距，正式结果不使用未来实际量。
3. 24 小时跨日窗口、偏差校正和历史残差场景的作用由同口径消融确定，不能由单次费用差单独归因。
4. S2-S3 结论严格服从多求解器和分币稳定性门。

完全信息模型只用于同口径事后参照。最终结论以六组消融、逐月配对差、移动块区间和多求解器稳定性共同支持；对不能稳定到分币的差异，本文不作收益判断。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于文献检索辅助、代码调试、审计检查和语言润色，详细使用情况见支撑材料。

参考文献

- [1] Parisio A, Rikos E, Glielmo L. A model predictive control approach to microgrid operation optimization[J]. IEEE Transactions on Control Systems Technology, 2014, 22(5): 1813-1827. DOI: 10.1109/TCST.2013.2295737.
- [2] Mayne D Q, Rawlings J B, Rao C V, et al. Constrained model predictive control: Stability and optimality[J]. Automatica, 2000, 36(6): 789-814. DOI: 10.1016/S0005-1098(99)00214-9.

[3] Olivares D E, Lara J D, Cañizares C A, et al. Stochastic-predictive energy management system for isolated microgrids[J]. IEEE Transactions on Smart Grid, 2015, 6(6): 2681-2693. DOI: 10.1109/TSG.2015.2469631.

[4] Birge J R, Louveaux F. Introduction to Stochastic Programming[M]. New York: Springer, 1997. DOI: 10.1007/b97617.

[5] Pinson P, Madsen H, Nielsen H A, et al. From probabilistic forecasts to statistical scenarios of short-term wind power production[J]. Wind Energy, 2009, 12(1): 51-62. DOI: 10.1002/we.284.

[6] Hyndman R J, Athanasopoulos G. Forecasting: Principles and Practice[M/OL]. 3rd ed. Melbourne: OTexts, 2021. <https://otexts.com/fpp3/>.

[7] Künsch H R. The jackknife and the bootstrap for general stationary observations[J]. The Annals of Statistics, 1989, 17(3): 1217-1241. DOI: 10.1214/aos/1176347265.

[8] Sioshansi R, Denholm P, Jenkin T, et al. Estimating the value of electricity storage in PJM: Arbitrage and some welfare effects[J]. Energy Economics, 2009, 31(2): 269-277. DOI: 10.1016/j.eneco.2008.10.005.

[9] SciPy Developers. scipy.optimize.linprog and HiGHS methods[EB/OL]. SciPy 1.16.2 documentation. <https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.optimize.linprog.html>.

[10] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范 (2026 年修订稿) [EB/OL]. https://www.mcm.edu.cn/html_cn/node/4cd596519c9eb9fbd866398f6df0caa3.html.

[11] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛人工智能工具使用规定 (2026 年试行) [EB/OL]. https://www.mcm.edu.cn/html_cn/node/fef94648f2836ab6cc81586f4c38512b.html.

附录

支撑材料文件清单

最终清单由打包脚本注入，至少包括：

- 五份正式结果工作簿；
- 统一配置文件和单一运行入口；
- 完整可运行源程序与依赖说明；
- 冻结结果清单、压缩全精度明细和结果溯源表；
- 人读审计报告、机器审计 JSON、实验表和必要日志；
- 图表及其数据；
- 文献证据；
- AI 工具使用详情.pdf；
- SHA-256 清单。

程序运行说明

```
bash run_pipeline.sh --config config/final.yaml
```

运行环境、软件版本、随机种子、配置哈希和最终文件哈希由 model_run_summary.json、resolved_config.json、frozen_manifest.json 和 SHA256SUMS.json 注入。

结果字段溯源

论文中每一个数字应映射到冻结结果键和原始明细字段。完整溯源表由 evidence/result_provenance.csv 注入。

指定日期结果表

以下各表由全精度执行明细自动生成；紧急区间按配置的四位显示阈值重建。

Q1 指定时段购电量

时间段	购电量 kWh
10:00-10:10	0.0000
12:00-12:10	486.4029
14:00-14:10	0.0000
16:00-16:10	394.9315
18:00-18:10	636.9826
20:00-20:10	0.0000

全天购电量 kWh	全天购电费 元
59,482.6990	35,126.95

全天购电量 kWh	全天购电费 元

Q1 四小时充放电量

时间段	充电量 kWh	放电量 kWh
0:00-4:00	4,500.0000	0.0000
4:00-8:00	833.3333	6,365.8412
8:00-12:00	4,787.9643	1,702.9970
12:00-16:00	5,286.0352	91.1014
16:00-20:00	0.0000	5,780.1319
20:00-24:00	5,333.3333	2,859.8681

0:00 与 24:00 储电量均为 6000.0000 kWh。

Q2 指定日期汇总

日期	0 点计划量 kWh	最终购电量 kWh	总费用 元	紧急购电量 kWh
2025-03-20	72,137.3456	72,137.3456	46,404.50	576.8743
2025-06-21	29,775.1603	29,775.1603	29,817.04	3,441.6008
2025-09-23	61,970.2454	61,970.2454	66,642.90	6,457.7870
2025-12-21	87,690.3824	87,690.3824	79,621.65	6,622.3226

Q2 指定日期紧急购电区间

日期	紧急购电时间段	购电量 kWh
2025-03-20	0:20-0:30	41.9917
2025-03-20	1:00-2:00	73.8862
2025-03-20	2:30-2:40	3.1818
2025-03-20	3:00-3:30	66.8749
2025-03-20	5:50-6:00	1.6846
2025-03-20	11:10-11:20	5.7111
2025-03-20	12:40-12:50	6.7261
2025-03-20	15:50-16:10	16.8758
2025-03-20	18:00-18:20	16.4935
2025-03-20	18:40-19:10	120.1784
2025-03-20	19:40-20:20	71.8469
2025-03-20	21:10-22:10	73.8271
2025-03-20	23:30-23:50	77.5963

日期	紧急购电时间段	购电量 kWh
2025-06-21	0:00-0:40	229.9266
2025-06-21	0:50-2:30	333.6683
2025-06-21	2:40-3:40	170.7966
2025-06-21	3:50-4:30	14.3260
2025-06-21	4:50-5:00	5.2338
2025-06-21	5:20-6:30	201.7087
2025-06-21	6:40-7:10	37.2327
2025-06-21	13:30-14:40	646.2062
2025-06-21	15:20-16:30	389.3716
2025-06-21	16:50-18:00	315.3897
2025-06-21	18:40-19:20	245.6383
2025-06-21	19:30-19:50	38.2431
2025-06-21	20:10-20:40	176.7765
2025-06-21	21:10-23:40	637.0829
2025-09-23	2:00-2:10	2.7805
2025-09-23	4:50-5:10	25.5337
2025-09-23	5:40-15:30	5,301.5435
2025-09-23	15:50-17:50	1,020.5773
2025-09-23	20:10-20:30	42.1224
2025-09-23	20:50-21:10	43.6122
2025-09-23	22:20-22:30	21.6175
2025-12-21	0:00-0:20	32.8213
2025-12-21	1:00-1:50	151.5752
2025-12-21	2:00-2:20	82.9892
2025-12-21	2:30-3:20	167.7223
2025-12-21	3:30-16:00	5,759.2440
2025-12-21	16:10-16:40	56.4025
2025-12-21	16:50-17:00	9.2871
2025-12-21	17:10-17:30	31.8028
2025-12-21	18:00-19:00	92.9199
2025-12-21	19:20-19:30	9.0021
2025-12-21	19:40-20:10	103.5840
2025-12-21	21:40-22:00	34.2659
2025-12-21	22:50-23:10	50.2326
2025-12-21	23:30-23:50	40.4737

Q3 指定日期汇总

日期	0点计划量 kWh	最终购电量 kWh	总费用 元	紧急购电量 kWh
2025-03-20	68,779.6314	68,414.0917	51,750.29	2,284.1843
2025-06-21	30,307.8170	30,537.3909	27,843.06	2,796.8018
2025-09-23	71,478.0066	72,168.8632	51,033.79	1,246.6983
2025-12-21	90,245.5696	91,060.8028	75,492.66	4,733.2853

Q3 指定日期紧急购电区间

日期	紧急购电时间段	购电量 kWh
2025-03-20	0:20-0:30	41.9917
2025-03-20	1:00-2:00	73.8862
2025-03-20	2:30-2:40	3.1818
2025-03-20	3:00-3:30	66.8749
2025-03-20	5:50-6:30	195.0354
2025-03-20	7:00-7:20	63.4070
2025-03-20	8:00-8:30	228.4439
2025-03-20	9:00-9:20	183.8412
2025-03-20	10:00-10:10	22.9599
2025-03-20	11:00-11:50	346.9058
2025-03-20	12:30-12:50	116.2522
2025-03-20	13:10-14:00	94.2693
2025-03-20	14:50-15:00	71.8996
2025-03-20	15:40-16:00	183.8916
2025-03-20	16:30-17:00	223.9490
2025-03-20	17:50-18:20	23.8600
2025-03-20	18:40-19:10	120.2642
2025-03-20	19:40-20:20	71.8473
2025-03-20	21:10-22:10	73.8271
2025-03-20	23:30-23:50	77.5963
2025-06-21	0:00-0:40	229.9266
2025-06-21	0:50-2:30	333.6683
2025-06-21	2:40-3:40	170.7953
2025-06-21	3:50-4:30	16.4086
2025-06-21	4:50-5:10	28.8882
2025-06-21	6:00-6:30	336.7521
2025-06-21	7:00-7:20	85.2100
2025-06-21	15:50-16:00	32.9809
2025-06-21	16:40-17:00	101.4849

日期	紧急购电时间段	购电量 kWh
2025-06-21	17:30-18:00	243.2148
2025-06-21	18:30-19:20	365.2200
2025-06-21	19:30-19:50	38.3942
2025-06-21	20:10-20:40	176.7751
2025-06-21	21:10-23:40	637.0829
2025-09-23	2:00-2:10	2.7805
2025-09-23	4:50-5:10	25.3986
2025-09-23	5:40-5:50	8.9430
2025-09-23	6:00-6:30	146.4188
2025-09-23	7:00-7:30	289.6226
2025-09-23	8:00-8:30	183.6379
2025-09-23	10:00-10:10	34.7398
2025-09-23	11:00-11:10	51.2378
2025-09-23	13:40-14:00	66.5760
2025-09-23	14:50-15:00	43.9015
2025-09-23	15:50-16:00	119.4027
2025-09-23	16:40-17:00	76.7700
2025-09-23	17:30-18:00	88.4579
2025-09-23	18:30-18:40	1.4592
2025-09-23	20:10-20:30	42.1223
2025-09-23	20:50-21:10	43.6122
2025-09-23	22:20-22:30	21.6175
2025-12-21	0:00-0:20	32.8213
2025-12-21	1:00-1:50	151.5752
2025-12-21	2:00-2:20	82.9892
2025-12-21	2:30-3:20	167.7223
2025-12-21	3:30-7:30	1,296.5574
2025-12-21	8:00-8:40	360.1408
2025-12-21	9:00-9:50	410.4809
2025-12-21	10:30-12:00	1,228.9730
2025-12-21	12:40-13:10	126.8957
2025-12-21	13:40-14:00	120.6130
2025-12-21	14:40-15:00	71.2852
2025-12-21	15:20-16:00	128.2512
2025-12-21	16:20-17:00	192.8922
2025-12-21	17:10-17:30	31.6106

日期	紧急购电时间段	购电量 kWh
2025-12-21	18:00-19:00	92.9188
2025-12-21	19:20-19:30	9.0021
2025-12-21	19:40-20:10	103.5840
2025-12-21	21:40-22:00	34.2659
2025-12-21	22:50-23:10	50.2326
2025-12-21	23:30-23:50	40.4737

Q4-2 指定日期汇总

日期	0点计划量 kWh	最终购电量 kWh	总费用 元	紧急购电量 kWh
2025-03-20	73,022.3930	73,022.3930	48,074.39	576.8743
2025-06-21	29,933.4936	29,933.4936	25,707.90	3,441.6008
2025-09-23	61,970.2454	61,970.2454	69,230.39	6,457.7870
2025-12-21	89,463.3554	89,463.3554	96,647.95	6,622.3226

Q4-2 指定日期紧急购电区间

日期	紧急购电时间段	购电量 kWh
2025-03-20	0:20-0:30	41.9917
2025-03-20	1:00-2:00	73.8862
2025-03-20	2:30-2:40	3.1818
2025-03-20	3:00-3:30	66.8749
2025-03-20	5:50-6:00	1.6846
2025-03-20	11:10-11:20	5.7111
2025-03-20	12:40-12:50	6.7261
2025-03-20	15:50-16:10	16.8758
2025-03-20	18:00-18:20	16.4935
2025-03-20	18:40-19:10	120.1784
2025-03-20	19:40-20:20	71.8469
2025-03-20	21:10-22:10	73.8271
2025-03-20	23:30-23:50	77.5963
2025-06-21	0:00-0:40	229.9266
2025-06-21	0:50-2:30	333.6683
2025-06-21	2:40-3:40	170.7966
2025-06-21	3:50-4:30	14.3260
2025-06-21	4:50-5:00	5.2338
2025-06-21	5:20-6:30	201.7087
2025-06-21	6:40-7:10	37.2327

日期	紧急购电时间段	购电量 kWh
2025-06-21	13:30-14:40	646.2062
2025-06-21	15:20-16:30	389.3716
2025-06-21	16:50-18:00	315.3897
2025-06-21	18:40-19:20	245.6383
2025-06-21	19:30-19:50	38.2431
2025-06-21	20:10-20:40	176.7765
2025-06-21	21:10-23:40	637.0829
2025-09-23	2:00-2:10	2.7805
2025-09-23	4:50-5:10	25.5337
2025-09-23	5:40-15:30	5,301.5435
2025-09-23	15:50-17:50	1,020.5773
2025-09-23	20:10-20:30	42.1224
2025-09-23	20:50-21:10	43.6122
2025-09-23	22:20-22:30	21.6175
2025-12-21	0:00-0:20	32.8213
2025-12-21	1:00-1:50	151.5752
2025-12-21	2:00-2:20	82.9892
2025-12-21	2:30-3:20	167.7223
2025-12-21	3:30-16:00	5,759.2440
2025-12-21	16:10-16:40	56.4025
2025-12-21	16:50-17:00	9.2871
2025-12-21	17:10-17:30	31.8028
2025-12-21	18:00-19:00	92.9199
2025-12-21	19:20-19:30	9.0021
2025-12-21	19:40-20:10	103.5840
2025-12-21	21:40-22:00	34.2659
2025-12-21	22:50-23:10	50.2326
2025-12-21	23:30-23:50	40.4737

Q4-3 指定日期汇总

日期	0点计划量 kWh	最终购电量 kWh	总费用 元	紧急购电量 kWh
2025-03-20	69,612.9647	69,176.2091	53,685.49	2,274.7882
2025-06-21	30,439.1698	30,668.7438	24,175.02	2,800.4960
2025-09-23	71,478.0066	72,194.4292	53,312.40	1,230.1052
2025-12-21	92,112.6303	92,863.4735	90,935.04	4,738.6810

Q4-3 指定日期紧急购电区间

日期	紧急购电时间段	购电量 kWh
2025-03-20	0:20-0:30	41.9917
2025-03-20	1:00-2:00	73.8862
2025-03-20	2:30-2:40	3.1818
2025-03-20	3:00-3:30	66.8749
2025-03-20	5:50-6:30	195.0354
2025-03-20	7:00-7:20	63.4070
2025-03-20	8:00-8:30	228.4439
2025-03-20	9:00-9:20	174.4451
2025-03-20	10:00-10:10	22.9599
2025-03-20	11:00-11:50	346.9058
2025-03-20	12:30-12:50	116.2522
2025-03-20	13:10-14:00	94.2693
2025-03-20	14:50-15:00	71.8996
2025-03-20	15:40-16:00	183.8916
2025-03-20	16:30-17:00	223.9490
2025-03-20	17:50-18:20	23.8600
2025-03-20	18:40-19:10	120.2642
2025-03-20	19:40-20:20	71.8473
2025-03-20	21:10-22:10	73.8271
2025-03-20	23:30-23:50	77.5963
2025-06-21	0:00-0:40	229.9266
2025-06-21	0:50-2:30	333.6683
2025-06-21	2:40-3:40	170.7953
2025-06-21	3:50-4:30	16.4086
2025-06-21	4:50-5:10	28.8882
2025-06-21	6:00-6:30	340.4463
2025-06-21	7:00-7:20	85.2100
2025-06-21	15:50-16:00	32.9809
2025-06-21	16:40-17:00	101.4849
2025-06-21	17:30-18:00	243.2148
2025-06-21	18:30-19:20	365.2200
2025-06-21	19:30-19:50	38.3942
2025-06-21	20:10-20:40	176.7751
2025-06-21	21:10-23:40	637.0829
2025-09-23	2:00-2:10	2.7805
2025-09-23	4:50-5:10	25.3986
2025-09-23	5:40-5:50	8.9430

日期	紧急购电时间段	购电量 kWh
2025-09-23	6:00-6:30	144.7191
2025-09-23	7:00-7:30	289.6226
2025-09-23	8:00-8:30	183.6379
2025-09-23	10:00-10:10	34.7398
2025-09-23	11:00-11:10	57.3177
2025-09-23	11:30-11:40	3.1174
2025-09-23	13:40-14:00	66.5760
2025-09-23	14:50-15:00	33.3542
2025-09-23	15:50-16:00	105.8593
2025-09-23	16:40-17:00	76.7700
2025-09-23	17:30-18:00	88.4579
2025-09-23	18:30-18:40	1.4592
2025-09-23	20:10-20:30	42.1223
2025-09-23	20:50-21:10	43.6122
2025-09-23	22:20-22:30	21.6175
2025-12-21	0:00-0:20	32.8213
2025-12-21	1:00-1:50	151.5752
2025-12-21	2:00-2:20	82.9892
2025-12-21	2:30-3:20	167.7223
2025-12-21	3:30-7:30	1,296.5574
2025-12-21	8:00-8:40	360.1408
2025-12-21	9:00-9:50	414.0401
2025-12-21	10:30-12:00	1,227.0200
2025-12-21	12:40-13:10	126.8957
2025-12-21	13:40-14:00	120.6130
2025-12-21	14:40-15:00	75.0748
2025-12-21	15:20-16:00	128.2512
2025-12-21	16:20-17:00	192.8922
2025-12-21	17:10-17:30	31.6106
2025-12-21	18:00-19:00	92.9188
2025-12-21	19:20-19:30	9.0021
2025-12-21	19:40-20:10	103.5840
2025-12-21	21:40-22:00	34.2659
2025-12-21	22:50-23:10	50.2326
2025-12-21	23:30-23:50	40.4737

完整可运行源程序

以下代码与支撑材料中的同名文件一致；运行入口为 run_pipeline.sh。

config/final.yaml

```
{
  "schema_version": "1.0",
  "source_root": "../../../Desktop/CUMCM2026Problems",
  "output_root": "outputs/c_final_v1",
  "study": {
    "year": 2025,
    "training_start": "2025-01-01",
    "training_end": "2025-01-31",
    "official_start": "2025-02-01",
    "official_end": "2025-12-31",
    "initial_soc_kwh": 6000.0,
    "terminal_soc_kwh": 6000.0
  },
  "storage": {
    "capacity_kwh": 12000.0,
    "soc_min_kwh": 1200.0,
    "soc_max_kwh": 10800.0,
    "power_kw": 5000.0,
    "charge_efficiency": 0.9,
    "discharge_efficiency": 0.9,
    "interval_minutes": 10
  },
  "forecast": {
    "same_weekday_profiles": 4,
    "bias_window_days": 28,
    "cap_window_days": 60,
    "cap_quantile": 0.995,
    "scenario_count": 20,
    "scenario_history_days": 20
  },
  "rolling": {
    "decision_hours": [0, 6, 12, 18],
    "horizon_hours": 24,
    "execute_hours": 6,
    "settlement": "final_relative_to_00_plan",
    "sequential_sensitivity": true,
    "terminal_candidates": ["daily_closed", "state_cycle", "soft_target"],
    "soft_penalty_multipliers": [0.5, 1.0, 1.5],
    "selected_terminal_rule": "soft_target",
    "selected_soft_penalty_multiplier": 0.5,
    "selection_method": "minimum combined Q3/Q4-3 realized January validation cost; validation dates 2025-01-08..2025-01-31"
  },
  "cost": {
    "down_adjustment_multiplier": 0.5,
    "up_adjustment_multiplier": 1.5,
    "emergency_multiplier": 5.0,
    "throughput_tiebreak_yuan_per_kwh": 1e-9
  },
  "audit": {
    "numeric_tolerance": 1e-7,
    "feasibility_tolerance": 1e-8,
    "display_decimals": 4,
    "emergency_display_omit_threshold_kwh": 0.00005
  },
  "experiments": {
    "arms": ["raw_baseline", "crossday_only", "bias_only", "bias_crossday", "scenario_crossday", "all_three"],
    "official_arm": "all_three",
    "official_scenario": "S3",
    "schedules": {"S0": [0], "S1": [0, 6], "S2": [0, 6, 12], "S3": [0, 6, 12, 18]},
    "solver_methods": ["highs", "highs-ds", "highs-ipm"],
    "tiebreak_coefficients": [1e-10, 1e-9, 1e-8],
    "moving_block_days": 7,
    "bootstrap_replicates": 2000,
    "bootstrap_seed": 20250912,
    "stability_workers": 4
  }
}
```

requirements.txt

```
numpy==2.5.3
scipy==1.16.2
pandas>=2.2
matplotlib>=3.8
openpyxl>=3.1
python-docx>=1.1
pypdf>=5.0
```

run_pipeline.sh

```
#!/usr/bin/env bash
set -euo pipefail
```

```
PROJECT_DIR="$(cd "$(dirname "$0")" && pwd)"
cd "$PROJECT_DIR"
TASK_PY_PATH="$PROJECT_DIR"
if [[ -d "$PROJECT_DIR/work/python_deps" ]]; then
    TASK_PY_PATH="$PROJECT_DIR/work/python_deps:$TASK_PY_PATH"
fi
export PYTHONPATH="$TASK_PY_PATH${PYTHONPATH:+:$PYTHONPATH}"
```

```
CONFIG_PATH="config/final.yaml"
SKIP_MODEL=0
while [[ $# -gt 0 ]]; do
    case "$1" in
        --config) CONFIG_PATH="$2"; shift 2 ;;
        --skip-model) SKIP_MODEL=1; shift ;;
        *) echo "Unknown argument: $1" >&2; exit 2 ;;
    esac
done
```

```
RUNTIME_BASE="$HOME/.cache/codex-runtimes/codex-primary-runtime/dependencies"
PY_BIN="$CODEX_PYTHON_BIN-${RUNTIME_BASE}/python/bin/python3"
NODE_BIN="$CODEX_NODE_BIN-${RUNTIME_BASE}/node/bin/node"
NODE_PACKAGES="$CODEX_NODE_MODULES-${RUNTIME_BASE}/node/node_modules"
if [[ ! -x "$PY_BIN" ]]; then PY_BIN="python3"; fi
if [[ ! -x "$NODE_BIN" ]]; then NODE_BIN="node"; fi
```

```
if [[ ! -e node_modules ]]; then ln -s "$NODE_PACKAGES" node_modules; fi
```

```
mkdir -p outputs/c_final_v1/logs outputs/c_final_v1/qa work/paper_build
```

```
if [[ "$SKIP_MODEL" -eq 0 ]]; then
    "$PY_BIN" scripts/run_model.py --config "$CONFIG_PATH" --scope official 2>&1 | tee outputs/c_final_v1/logs/model_official.log
    "$PY_BIN" scripts/run_model.py --config "$CONFIG_PATH" --scope pre-parallel 2>&1 | tee outputs/c_final_v1/logs/model_experiments.log
    "$PY_BIN" scripts/run_model.py --config "$CONFIG_PATH" --scope stability 2>&1 | tee outputs/c_final_v1/logs/model_stability.log
fi
```

```
"$PY_BIN" scripts/validate_model_outputs.py --config "$CONFIG_PATH" 2>&1 | tee outputs/c_final_v1/logs/model_validation.log
```

```
"$NODE_BIN" scripts/build_workbooks.mjs 2>&1 | tee outputs/c_final_v1/logs/workbook_run.log
"$PY_BIN" src/audit/audit_final.py --config "$CONFIG_PATH" --results-dir outputs/c_final_v1 --output-dir outputs/c_final_v1/audit 2>&1 | tee outputs/c_final_v1/logs/audit_run.log
"$PY_BIN" scripts/freeze_results.py 2>&1 | tee outputs/c_final_v1/logs/freeze_run.log
"$PY_BIN" scripts/make_charts.py 2>&1 | tee outputs/c_final_v1/logs/chart_run.log
"$PY_BIN" scripts/make_evidence.py 2>&1 | tee outputs/c_final_v1/logs/evidence_run.log
"$PY_BIN" scripts/build_documents.py 2>&1 | tee outputs/c_final_v1/logs/document_build.log
```

```
DOC_SKILL="$HOME/.codex/plugins/cache/openai-primary-runtime/documents/26.905.11957/skills/documents"
```

```
"$PY_BIN" "$DOC_SKILL/scripts/privacy_scrub.py" outputs/c_final_v1/paper/C 题论文.docx --out work/paper_build/C 题论文_scrubbed.docx
```

```
mv work/paper_build/C 题论文_scrubbed.docx outputs/c_final_v1/paper/C 题论文.docx
```

```
"$PY_BIN" "$DOC_SKILL/scripts/privacy_scrub.py" work/paper_build/AI 工具使用详情.docx --out work/paper_build/AI 工具使用详情_scrubbed.docx
```

```
PAPER_RENDER="$(mktemp -d "${TMPDIR:-/tmp}/cumcm-paper-render.XXXXXX")"
```

```
AI_RENDER="$(mktemp -d "${TMPDIR:-/tmp}/cumcm-ai-render.XXXXXX")"
```

```
trap 'rm -rf "$PAPER_RENDER" "$AI_RENDER"' EXIT
```

```
"$PY_BIN" "$DOC_SKILL/render_docx.py" outputs/c_final_v1/paper/C 题论文.docx --output_dir "$PAPER_RENDER" --emit_pdf
```

```
"$PY_BIN" "$DOC_SKILL/render_docx.py" work/paper_build/AI 工具使用详情_scrubbed.docx --output_dir "$AI_RENDER" --emit_pdf
```

```
cp "$PAPER_RENDER/C 题论文.pdf" outputs/c_final_v1/paper/C 题论文.pdf
```

```
cp "$AI_RENDER/AI 工具使用详情_scrubbed.pdf" outputs/c_final_v1/paper/AI 工具使用详情.pdf
```

```
mkdir -p outputs/c_final_v1/qa/paper_render outputs/c_final_v1/qa/ai_render
```

```
cp "$PAPER_RENDER"/page-*.png outputs/c_final_v1/qa/paper_render/
```

```
cp "$AI_RENDER"/page-*.png outputs/c_final_v1/qa/ai_render/
```

```
"$PY_BIN" scripts/package_final.py 2>&1 | tee outputs/c_final_v1/logs/package_run.log
```

```
"$PY_BIN" scripts/qa_final.py 2>&1 | tee outputs/c_final_v1/logs/final_qa.log
```

```
echo "C 题完整竞赛包已生成：outputs/c_final_v1"
```

src/audit/__init__.py

```
"""Independent audit gate for the C-problem delivery."""
```

src/audit/audit_final.py

```
#!/usr/bin/env python3
```

```
"""Strict, non-mutating audit for C-problem inputs, templates and results.
```

The reader never coerces blanks, error cells, non-finite values or text to zero. Official workbook tables are checked against the supplied templates. Full-precision dispatch CSV files are preferred for physical, settlement and emergency-interval reconstruction; a workbook detail sheet is a documented fallback for legacy evidence only.

```
"""
```

```
from __future__ import annotations
```

```
import argparse
import csv
import hashlib
import json
import math
import os
import re
import shutil
import sys
from collections import Counter, defaultdict
from dataclasses import asdict, dataclass
from datetime import date, datetime, time, timedelta
from decimal import Decimal, ROUND_HALF_UP
from pathlib import Path
from typing import Any, Iterable, Mapping, Sequence
```

```
import numpy as np
from openpyxl import load_workbook
```

```
RESULT_NAMES = ("result1.xlsx", "result2.xlsx", "result3.xlsx", "result4-2.xlsx", "result4-3.xlsx")
MAIN_START = date(2025, 2, 1)
MAIN_END = date(2025, 12, 31)
FULL_START = date(2025, 1, 1)
FULL_END = date(2025, 12, 31)
ERROR_TOKENS = {"#REF!", "#DIV/0!", "#VALUE!", "#NAME?", "#N/A", "#NUM!", "#NULL!", "#SPILL!", "#CALC!"}
FOUR_HOUR_LABELS = tuple(f"{h}:00-{h + 4}:00" for h in range(0, 24, 4))
```

```
@dataclass(frozen=True)
class AuditConfig:
    display_decimals: int = 4
    emergency_display_threshold_kwh: float = 0.00005
    efficiency: float = 0.9
    interval_hours: float = 1 / 6
    storage_min_kwh: float = 1200.0
    storage_max_kwh: float = 10800.0
    storage_power_kw: float = 5000.0
    initial_soc_kwh: float = 6000.0
    terminal_soc_kwh: float = 6000.0
    numeric_tolerance: float = 1e-7
    displayed_tolerance: float = 2.1e-4
```

```
@property
def step_power_limit_kwh(self) -> float:
    return self.storage_power_kw * self.interval_hours
```

```
@dataclass
class Check:
    id: str
    status: str
    severity: str
    scope: str
    actual: Any
    expected: Any
    evidence: str
```

```

impact: str

@dataclass(frozen=True)
class Detail:
    source: Path
    source_kind: str
    rows: tuple[dict[str, Any], ...]

class Audit:
    def __init__(self) -> None:
        self.checks: list[Check] = []

    def add(self, check_id: str, status: str, severity: str, scope: str,
            actual: Any, expected: Any, evidence: str, impact: str) -> None:
        self.checks.append(Check(check_id, status, severity, scope, actual,
                                expected, evidence, impact))

def sha256(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open("rb") as handle:
        for block in iter(lambda: handle.read(1 < 20), b''):
            digest.update(block)
    return digest.hexdigest()

def sanitize_output(value: Any, project_root: Path, source_root: Path) -> Any:
    """Remove machine/user-specific absolute roots from deliverable evidence."""
    replacements = sorted(
        ((str(project_root.resolve()), "<project_root>"), (str(source_root.resolve()), "<source_root>")),
        key=lambda pair: len(pair[0]), reverse=True,
    )
    if isinstance(value, str):
        for original, placeholder in replacements:
            value = value.replace(original, placeholder)
        return value
    if isinstance(value, dict):
        return {key: sanitize_output(child, project_root, source_root) for key, child in value.items()}
    if isinstance(value, list):
        return [sanitize_output(child, project_root, source_root) for child in value]
    if isinstance(value, tuple):
        return tuple(sanitize_output(child, project_root, source_root) for child in value)
    return value

def _flatten_mapping(value: Any, output: dict[str, str]) -> None:
    if not isinstance(value, dict):
        return
    for key, child in value.items():
        normalized = str(key).lower().replace("-", "_")
        if isinstance(child, dict):
            _flatten_mapping(child, output)
        elif isinstance(child, (str, int, float, bool)) or child is None:
            output[normalized] = "" if child is None else str(child)

def flat_yaml_scalars(path: Path) -> tuple[dict[str, str], str]:
    """Read scalar values without requiring a YAML dependency.

    Nested keys are deliberately flattened because the audit accepts several
    documented aliases. This parser does not attempt to interpret arbitrary
    YAML and therefore cannot silently invent values.
    """
    if not path.is_file():
        return {}, "missing_defaults"
    text = path.read_text(encoding="utf-8")
    try:
        document = json.loads(text)
    except json.JSONDecodeError:
        document = None
    if isinstance(document, dict):
        values: dict[str, str] = {}

```

```

    _flatten_mapping(document, values)
    return values, "json"
values: dict[str, str] = {}
for raw in text.splitlines():
    line = raw.split("#", 1)[0].rstrip()
    match = re.match(r'^\s*["\']?([A-Za-z_][\w-]*)["\']?\s*:\s*([^\s]*|"[^"]*"|'\'[^\']*'\')\s*$', line)
    if match:
        values[match.group(1).lower().replace("-", "_")] = match.group(2).strip(" '\"")
return values, "flat_yaml_fallback"

```

```

def _pick_number(values: Mapping[str, str], aliases: Sequence[str], default: float) -> float:
    for name in aliases:
        if name in values:
            try:
                parsed = float(values[name])
            except ValueError as exc:
                raise ValueError(f"配置项 {name} 不是数值: {values[name]!r}") from exc
            if not math.isfinite(parsed):
                raise ValueError(f"配置项 {name} 不是有限数值: {values[name]!r}")
            return parsed
    return default

```

```

def load_config(path: Path) -> tuple[AuditConfig, dict[str, str]]:
    values, parser_source = flat_yaml_scalars(path)
    decimals = int(_pick_number(values, ("display_decimals", "output_decimals", "decimals"), 4))
    default_threshold = 0.5 * 10 ** (-decimals)
    interval_hours = _pick_number(values, ("interval_hours", "dt_hours", "dt"), math.nan)
    if math.isnan(interval_hours):
        interval_hours = _pick_number(values, ("interval_minutes",), 10.0) / 60.0
    cfg = AuditConfig(
        display_decimals=decimals,
        emergency_display_threshold_kwh=_pick_number(
            values,
            ("emergency_display_omit_threshold_kwh", "emergency_display_threshold_kwh", "emergency_display_threshold", "display_zero_threshold_kwh"),
            default_threshold,
        ),
        efficiency=_pick_number(values, ("efficiency", "eta", "storage_efficiency", "charge_efficiency"), 0.9),
        interval_hours=interval_hours,
        storage_min_kwh=_pick_number(values, ("storage_min_kwh", "soc_min_kwh", "e_min"), 1200.0),
        storage_max_kwh=_pick_number(values, ("storage_max_kwh", "soc_max_kwh", "e_max"), 10800.0),
        storage_power_kw=_pick_number(values, ("storage_power_kw", "power_kw", "power_limit_kw", "p_max"), 5000.0),
        initial_soc_kwh=_pick_number(values, ("initial_soc_kwh", "initial_soc"), 6000.0),
        terminal_soc_kwh=_pick_number(values, ("terminal_soc_kwh", "terminal_soc"), 6000.0),
        numeric_tolerance=_pick_number(values, ("numeric_tolerance", "audit_tolerance"), 1e-7),
        displayed_tolerance=_pick_number(values, ("displayed_tolerance", "display_tolerance"), 2.1e-4),
    )
    if cfg.display_decimals < 0 or cfg.display_decimals > 12:
        raise ValueError("display_decimals 必须在 0..12")
    if cfg.emergency_display_threshold_kwh < 0:
        raise ValueError("emergency_display_threshold_kwh 不能为负")
    return cfg, values, parser_source

```

```

def norm_date(value: Any) -> date:
    if isinstance(value, datetime):
        return value.date()
    if isinstance(value, date):
        return value
    if value is None or (isinstance(value, str) and not value.strip()):
        raise ValueError("日期为空")
    text = str(value).strip().replace(" ", "-").replace("/", "-")
    return datetime.strptime(text[:10], "%Y-%m-%d").date()

```

```

def date_range(start: date, end: date) -> list[date]:
    return [start + timedelta(days=i) for i in range((end - start).days + 1)]

```

```

def norm_clock(value: Any) -> str:
    if isinstance(value, datetime):
        value = value.time()
    if isinstance(value, time):
        return f"{value.hour:02d}:{value.minute:02d}"
    text = str(value).strip()
    if text in {"0:00+1", "00:00+1", "24:00", "24:00:00"}:
        return "24:00"

```

```

match = re.match(r"^(\\d{1,2});(\\d{2})(?:;\\d{2})?$", text)
if not match:
    return text
return f"{int(match.group(1)):02d};{int(match.group(2)):02d}"

```

```

def physical_labels() -> tuple[str, ...]:
    labels = []
    for pos in range(144):
        start = pos * 10
        end = (pos + 1) * 10
        a = f"{start // 60:02d};{start % 60:02d}"
        b = "24:00" if end == 1440 else f"{end // 60:02d};{end % 60:02d}"
        labels.append(f"{a}-{b}")
    return tuple(labels)

```

```

def required_number(value: Any, location: str) -> float:
    """Return a finite float, rejecting missing/error/text values explicitly."""
    if value is None or (isinstance(value, str) and not value.strip()):
        raise ValueError(f"{location}: 空值")
    if isinstance(value, str) and value.strip().upper() in ERROR_TOKENS:
        raise ValueError(f"{location}: 公式错误 {value.strip()}")
    if isinstance(value, bool):
        raise ValueError(f"{location}: 布尔值不是数值")
    try:
        number = float(value)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"{location}: 非数值 {value!r}") from exc
    if not math.isfinite(number):
        raise ValueError(f"{location}: 非有限数值 {value!r}")
    return number

```

```

def scan_required_numbers(values: Iterable[tuple[str, Any]], sample_limit: int = 20) -> tuple[list[float], dict[str, Any]]:
    numbers: list[float] = []
    errors: list[str] = []
    count = 0
    for location, value in values:
        count += 1
        try:
            numbers.append(required_number(value, location))
        except ValueError as exc:
            if len(errors) < sample_limit:
                errors.append(str(exc))
    return numbers, {"required_cells": count, "valid_numeric_cells": len(numbers), "error_count": count - len(numbers), "samples": errors}

```

```

def workbook_formula_errors(path: Path) -> dict[str, Any]:
    errors: list[str] = []
    for data_only in (False, True):
        wb = load_workbook(path, data_only=data_only, read_only=True)
        for ws in wb.worksheets:
            for row in ws.iter_rows():
                for cell in row:
                    value = cell.value
                    if cell.data_type == "e" or (isinstance(value, str) and value.strip().upper() in ERROR_TOKENS):
                        if len(errors) < 50:
                            errors.append(f"{ws.title}!{cell.coordinate}={value}")
        wb.close()
    return {"error_count": len(errors), "samples": errors}

```

```

def workbook_headers(path: Path) -> dict[str, list[Any]]:
    wb = load_workbook(path, data_only=False, read_only=True)
    result = {ws.title: list(next(ws.iter_rows(values_only=True))) for ws in wb.worksheets}
    wb.close()
    return result

```

```

def audit_schema(audit: Audit, path: Path, expected_path: Path, prefix: str) -> bool:
    expected = workbook_headers(expected_path)
    actual = workbook_headers(path)
    missing = [sheet for sheet in expected if sheet not in actual]
    header_diffs = {

```

```

    sheet: {"actual": actual.get(sheet), "expected": header}
    for sheet, header in expected.items() if actual.get(sheet) != header
}
ok = not missing and not header_diffs
audit.add(f"{prefix}-SCHEMA", "PASS" if ok else "FAIL", "critical", path.name,
    {"missing_official_sheets": missing, "header_differences": header_diffs,
     "extra_sheets": sorted(set(actual) - set(expected))},
    "All official sheets and their complete header rows exactly match the supplied template; evidence sheets may be additional.",
    f"{path} vs {expected_path}", "Schema drift can make the submission ungradable")
return ok

```

```

def audit_attachment_inputs(audit: Audit, input_dir: Path) -> tuple[tuple[Any, ...], tuple[str, ...]]:
    expected_headers = {
        "附件 1.xlsx": {"Sheet1": ("时间", "电价", "小区负载", "光伏发电预测功率")},
        "附件 2.xlsx": {"小区负载": None, "光伏发电实际功率": None},
        "附件 3.xlsx": {"Sheet1": tuple(["日期", "预报时刻"] + [f"预报{i}小时" for i in range(1, 25)])},
        "附件 4.xlsx": {"Sheet1": None},
    }
    paths = {name: input_dir / name for name in expected_headers}
    missing = [name for name, path in paths.items() if not path.is_file()]
    audit.add("INPUT-FILES", "PASS" if not missing else "FAIL", "critical", "附件 1-4",
        {"missing": missing, "All four attachments exist.", str(input_dir), "Missing inputs prevent reconstruction."})
    if missing:
        return (), ()

    # Attachment 1: exact schema, 144 unique ordered source timestamps, all numeric.
    wb1 = load_workbook(paths["附件 1.xlsx"], data_only=True, read_only=True)
    ws1 = wb1["Sheet1"]
    rows1 = list(ws1.iter_rows(values_only=True))
    header1 = tuple(rows1[0])
    clocks = tuple(norm_clock(row[0]) for row in rows1[1:])
    expected_clocks = tuple([f"{m // 60:02d}:{m % 60:02d}" for m in range(10, 1440, 10)] + ["24:00"])
    _, numeric1 = scan_required_numbers(
        (f"附件 1.xlsx!{r}:{c}", value)
        for r, row in enumerate(rows1[1:], 2) for c, value in enumerate(row[1:4], 2)
    )
    input1_ok = header1 == expected_headers["附件 1.xlsx"]["Sheet1"] and clocks == expected_clocks and len(set(clocks)) == 144 and numeric1["error_count"] == 0
    audit.add("INPUT-ATTACHMENT1", "PASS" if input1_ok else "FAIL", "critical", "附件 1 Sheet1",
        {"header": header1, "rows": len(rows1) - 1, "unique_times": len(set(clocks)),
         "ordered_times": clocks == expected_clocks, "numeric_scan": numeric1,
         "Exact 4 fields, ordered 00:10..24:00, 144 unique timestamps, and 432 finite numeric cells.",
         str(paths["附件 1.xlsx"]), "Defines Q1 and the fixed tariff."})
    wb1.close()

    shared_headers: tuple[Any, ...] = ()
    shared_dates: list[date] | None = None
    for filename, sheets in (("附件 2.xlsx", ("小区负载", "光伏发电实际功率")), ("附件 4.xlsx", ("Sheet1"))):
        wb = load_workbook(paths[filename], data_only=True, read_only=True)
        for sheet in sheets:
            ws = wb[sheet]
            rows = list(ws.iter_rows(values_only=True))
            header = tuple(rows[0])
            dates: list[date] = []
            date_errors: list[str] = []
            for idx, row in enumerate(rows[1:], 2):
                try:
                    dates.append(norm_date(row[0]))
                except ValueError as exc:
                    date_errors.append(f"row {idx}: {exc}")
            _, numeric = scan_required_numbers(
                (f"{filename}:{sheet}!R{r}C{c}", value)
                for r, row in enumerate(rows[1:], 2) for c, value in enumerate(row[1:145], 2)
            )
            time_headers = tuple(norm_clock(v) for v in header[1:145])
            ok = (header[0] == "日期\\时间" and len(header) == 145 and time_headers == expected_clocks
                  and len(set(time_headers)) == 144 and not date_errors
                  and dates == date_range(FULL_START, FULL_END) and len(set(dates)) == 365
                  and numeric["error_count"] == 0)
            audit.add(f"INPUT-{filename}-{sheet}", "PASS" if ok else "FAIL", "critical", f"{filename}/{sheet}",
                {"rows": len(rows) - 1, "dates": len(dates), "unique_dates": len(set(dates)),
                 "date_start": str(dates[0]) if dates else None, "date_end": str(dates[-1]) if dates else None,
                 "date_errors": date_errors[:20], "intervals": len(time_headers),
                 "unique_times": len(set(time_headers)), "ordered_times": time_headers == expected_clocks,
                 "numeric_scan": numeric,
                 "365 ordered unique dates and 144 ordered unique timestamps; every data cell finite numeric.",
                 str(paths[filename]), "A gap, duplicate or reordered interval contaminates all annual results."})
            if shared_headers and time_headers != tuple(norm_clock(v) for v in shared_headers[1:145]):
                audit.add(f"INPUT-{filename}-{sheet}-ALIGN", "FAIL", "critical", "input time-axis alignment",
                    {"same_headers": False, "Attachment 2/4 time headers exactly agree.", str(paths[filename]),
                     "Position-based joins would be invalid."})
            if shared_dates is not None and dates != shared_dates:
                audit.add(f"INPUT-{filename}-{sheet}-DATE-ALIGN", "FAIL", "critical", "input date-axis alignment",

```

```

        {"same_dates": False, "Attachment 2/4 dates exactly agree.", str(paths[filename]),
         "Date joins would be invalid.")
    if not shared_headers:
        shared_headers = header
    if shared_dates is None:
        shared_dates = dates
    wb.close()

# Attachment 3: exactly four ordered releases per date and 24 finite forecast values per release.
wb3 = load_workbook(paths["附件 3.xlsx"], data_only=True, read_only=True)
ws3 = wb3["Sheet1"]
rows3 = list(ws3.iter_rows(values_only=True))
header3 = tuple(rows3[0])
current: date | None = None
records: list[tuple[date, str]] = []
bad_dates: list[str] = []
numeric_pairs: list[tuple[str, Any]] = []
for row_num, row in enumerate(rows3[1:], 2):
    if row[0] not in (None, ""):
        try:
            current = norm_date(row[0])
        except ValueError as exc:
            bad_dates.append(f"row {row_num}: {exc}")
            current = None
    if current is None:
        bad_dates.append(f"row {row_num}: missing carried date")
        continue
    issue = norm_clock(row[1])
    records.append((current, issue))
    numeric_pairs.extend((f"附件 3.xlsx!R{row_num}C{col}", value) for col, value in enumerate(row[2:26], 3))
_, numeric3 = scan_required_numbers(numeric_pairs)
by_date: dict[date, list[str]] = defaultdict(list)
for day, issue in records:
    by_date[day].append(issue)
expected_issues = ["00:00", "06:00", "12:00", "18:00"]
issue_bad = {str(day): issues for day, issues in by_date.items() if issues != expected_issues}
expected_records = [(day, issue) for day in date_range(FULL_START, FULL_END) for issue in expected_issues]
forecast_ok = (header3 == expected_headers["附件 3.xlsx"]["Sheet1"] and not bad_dates
               and records == expected_records and not issue_bad and numeric3["error_count"] == 0)
audit.add("INPUT-ATTACHMENT3", "PASS" if forecast_ok else "FAIL", "critical", "附件 3",
          {"header_ok": header3 == expected_headers["附件 3.xlsx"]["Sheet1"], "records": len(records),
           "dates": len(by_date), "bad_dates": bad_dates[:20], "issue_sequence_errors": dict(list(issue_bad.items()))[:20]},
          {"numeric_scan": numeric3,
           "365 dates x 4 releases (00/06/12/18) x 24 finite forecasts, in exact order.",
           str(paths["附件 3.xlsx"]), "Forecast gaps or release-time shifts create information leakage or missing decisions."})
wb3.close()
return shared_headers, expected_clocks

```

```

def audit_official_templates(audit: Audit, template_dir: Path) -> None:
    for name in RESULT_NAMES:
        path = template_dir / name
        if not path.is_file():
            audit.add(f"TEMPLATE-{name}", "FAIL", "critical", name, {"exists": False, "Official template exists.", str(path), "Cannot verify submission layout."})
            continue
        errors = workbook_formula_errors(path)
        wb = load_workbook(path, data_only=True, read_only=True)
        plan_sheet = wb["计划购电量"]
        header = tuple(next(plan_sheet.iter_rows(values_only=True)))
        interval_headers = header[1:145] if name != "result1.xlsx" else ()
        if name == "result1.xlsx":
            labels = tuple(row[0] for row in plan_sheet.iter_rows(min_row=2, values_only=True))
            ok = len(labels) == 144 and len(set(labels)) == 144
            actual = {"sheets": wb.sheetnames, "plan_rows": len(labels), "unique_interval_labels": len(set(labels)), "formula_errors": errors}
        else:
            dates = [norm_date(row[0]) for row in plan_sheet.iter_rows(min_row=2, values_only=True)]
            ok = (dates == date_range(MAIN_START, MAIN_END) and len(set(dates)) == 334
                  and len(interval_headers) == 144 and len(set(interval_headers)) == 144)
            actual = {"sheets": wb.sheetnames, "plan_days": len(dates), "unique_dates": len(set(dates)),
                     "date_start": str(dates[0]), "date_end": str(dates[-1]),
                     "interval_headers": len(interval_headers), "unique_interval_headers": len(set(interval_headers)),
                     "formula_errors": errors}
        wb.close()
        ok = ok and errors["error_count"] == 0
        audit.add(f"TEMPLATE-{name}", "PASS" if ok else "FAIL", "critical", name, actual,
                  "Official template has exact date population and 144 unique ordered interval labels with no formula-error literals.",
                  str(path), "The template is the schema authority for the result workbook.")

```

```

CSV_ALIASES: dict[str, tuple[str, ...]] = {
    "date": ("date", "日期"),
    "position": ("position", "时段序号"),
}

```

```

"label": ("physical_interval", "official_label", "时段", "时间段"),
"plan": ("plan_00_grid_kwh", "plan_grid_kwh", "planned_grid_kwh", "计划购电量(kwh)"),
"adjusted": ("final_adjusted_grid_kwh", "adjusted_grid_kwh", "调整购电量(kwh)"),
"emergency": ("emergency_kwh", "紧急购电量(kwh)"),
"charge": ("charge_kwh", "充电量(kwh)"),
"discharge": ("discharge_kwh", "放电量(kwh)"),
"soc_start": ("soc_start_kwh", "期初储电量(kwh)"),
"soc_end": ("soc_end_kwh", "期末储电量(kwh)"),
"unused": ("unused_surplus_kwh", "curtail_pv_kwh", "curtailment_kwh", "弃光弃电量(kwh)"),
"price": ("settlement_price", "price", "电价(元/kwh)"),
"load": ("actual_load_kwh", "load_kwh", "小区负载(kwh)"),
"pv": ("actual_pv_kwh", "pv_kwh", "光伏发电(kwh)"),
"final_cost": ("final_relative_cost_yuan", "分时购电成本(元)"),
"sequential_cost": ("sequential_cost_yuan",),
"plan_cost": ("plan_cost_yuan",),
"adjustment_cost": ("adjustment_cost_yuan",),
"emergency_cost": ("emergency_cost_yuan",),
}

```

```

def normalized_header_map(header: Sequence[Any]) -> dict[str, str]:
    original = {str(value).strip().lower(): str(value) for value in header if value is not None}
    result: dict[str, str] = {}
    for canonical, aliases in CSV_ALIASES.items():
        for alias in aliases:
            if alias.lower() in original:
                result[canonical] = original[alias.lower()]
                break
    return result

```

```

def _candidate_csvs(results_dir: Path, stem: str) -> list[Path]:
    aliases = {stem.replace("-", "_"), stem.replace("result", "q").replace("-", "_")}
    candidates = []
    for path in results_dir.rglob("*.csv") if results_dir.exists() else []:
        lower = path.name.lower().replace("-", "_")
        if any(alias in lower for alias in aliases) and ("detail" in lower or "dispatch" in lower or "full_precision" in lower):
            candidates.append(path)
    return sorted(candidates, key=lambda p: ("full_precision" not in p.name.lower(), len(str(p))))

```

```

def load_csv_detail(path: Path, adjusted_required: bool) -> Detail | None:
    with path.open(newline="", encoding="utf-8-sig") as handle:
        reader = csv.DictReader(handle)
        if reader.fieldnames is None:
            return None
        mapping = normalized_header_map(reader.fieldnames)
        required = {"date", "position", "plan", "emergency", "charge", "discharge", "soc_start", "soc_end", "unused", "price", "load", "pv"}
        if adjusted_required:
            required.add("adjusted")
        if not required.issubset(mapping):
            return None
        rows: list[dict[str, Any]] = []
        for row_num, row in enumerate(reader, 2):
            item: dict[str, Any] = {"_row": row_num}
            for canonical, source_name in mapping.items():
                item[canonical] = row[source_name]
            for field in ("history_cutoff_date", "model_arm", "scenario", "decision_hour"):
                if field in row:
                    item[field] = row[field]
            rows.append(item)
    return Detail(path, "full_precision_csv", tuple(rows))

```

```

def load_xlsx_detail(path: Path, adjusted_required: bool) -> Detail | None:
    wb = load_workbook(path, data_only=True, read_only=True)
    if "核实验明" not in wb.sheetnames:
        wb.close()
        return None
    ws = wb["核实验明"]
    iterator = ws.iter_rows(values_only=True)
    header = list(next(iterator))
    mapping = normalized_header_map(header)
    required = {"date", "label", "plan", "emergency", "charge", "discharge", "soc_start", "soc_end", "unused", "price", "load", "pv"}
    if adjusted_required:
        required.add("adjusted")

```

```

if not required.issubset(mapping):
    wb.close()
    return None
index = {str(value): i for i, value in enumerate(header)}
rows: list[dict[str, Any]] = []
for row_num, row in enumerate(iterator, 2):
    item = {canonical: row[index[source_name]] for canonical, source_name in mapping.items()}
    item["position"] = ((row_num - 2) % 144) + 1
    item["_row"] = row_num
    rows.append(item)
wb.close()
return Detail(path, "rounded_workbook_detail", tuple(rows))

```

```

def locate_detail(results_dir: Path, workbook_path: Path, adjusted_required: bool) -> Detail | None:
    stem = workbook_path.stem
    for path in _candidate_csvs(results_dir, stem):
        detail = load_csv_detail(path, adjusted_required)
        if detail is not None:
            return detail
    return load_xlsx_detail(workbook_path, adjusted_required)

```

```

def parse_detail(detail: Detail, adjusted_required: bool) -> tuple[list[dict[str, Any]], dict[str, Any]]:
    required_numeric = ["position", "plan", "emergency", "charge", "discharge", "soc_start", "soc_end", "unused", "price", "load", "pv"]
    if adjusted_required:
        required_numeric.append("adjusted")
    parsed: list[dict[str, Any]] = []
    errors: list[str] = []
    for source in detail.rows:
        row_num = source["_row"]
        item: dict[str, Any] = {"_row": row_num}
        for field in ("history_cutoff_date", "model_arm", "scenario", "decision_hour"):
            if field in source:
                item[field] = source[field]
        try:
            item["date"] = norm_date(source["date"])
        except (ValueError, TypeError) as exc:
            errors.append(f"row {row_num} date: {exc}")
            continue
        if "label" in source:
            item["label"] = str(source["label"]).strip()
        try:
            for field in required_numeric:
                item[field] = required_number(source.get(field), f"row {row_num} {field}")
            for field in ("final_cost", "sequential_cost", "plan_cost", "adjustment_cost", "emergency_cost"):
                if field in source:
                    item[field] = required_number(source[field], f"row {row_num} {field}")
        except ValueError as exc:
            errors.append(str(exc))
            continue
        parsed.append(item)
    return parsed, {"rows": len(detail.rows), "valid_rows": len(parsed), "error_count": len(errors), "samples": errors[:30]}

```

```

def audit_dispatch_causality(audit: Audit, name: str, rows: Sequence[dict[str, Any]]) -> None:
    missing = 0
    future = 0
    examples: list[str] = []
    for row in rows:
        raw = row.get("history_cutoff_date")
        if raw in (None, ""):
            missing += 1
            continue
        try:
            cutoff = norm_date(raw)
        except ValueError:
            future += 1
            if len(examples) < 20:
                examples.append(f"row {row['_row']}: invalid cutoff {raw!r}")
            continue
        if cutoff >= row["date"]:
            future += 1
            if len(examples) < 20:
                examples.append(f"row {row['_row']}: cutoff {cutoff} >= decision day {row['date']}")
    ok = missing == 0 and future == 0
    audit.add(f"FINAL-{name}-DISPATCH-CAUSALITY", "PASS" if ok else "FAIL", "critical", name,
             {"rows": len(rows), "missing_history_cutoff": missing, "noncausal_or_invalid": future, "samples": examples},
             "Every dispatch row records a history cutoff strictly earlier than its decision date.",
             "full-precision dispatch detail", "A physical backtest does not prove an executable information set.")

```

```

def audit_detail_coverage(audit: Audit, name: str, rows: Sequence[dict[str, Any]], scan: dict[str, Any], detail: Detail,

```

```

        official_labels: Sequence[str]) -> bool:
    expected_dates = date_range(MAIN_START, MAIN_END)
    by_date: dict[date, list[dict[str, Any]]] = defaultdict(list)
    for row in rows:
        by_date[row["date"]].append(row)
    expected_physical = physical_labels()
    bad_days: dict[str, Any] = {}
    for day in sorted(set(by_date) | set(expected_dates)):
        day_rows = by_date.get(day, [])
        positions = [int(r["position"]) for r in day_rows]
        pos_ok = positions == list(range(1, 145))
        label_ok = True
        if day_rows and "label" in day_rows[0]:
            labels = [r["label"] for r in day_rows]
            # Evidence must use one complete declared convention, never a merely unique permutation.
            label_ok = labels == list(expected_physical) or labels == list(official_labels)
        if len(day_rows) != 144 or len(set(positions)) != 144 or not pos_ok or not label_ok:
            if len(bad_days) < 30:
                bad_days[str(day)] = {"rows": len(day_rows), "unique_positions": len(set(positions)),
                                      "ordered_positions": pos_ok, "unique_labels": len(set(r.get("label", "") for r in day_rows)),
                                      "label_ok": label_ok}
    dates = sorted(by_date)
    ok = (scan["error_count"] == 0 and len(rows) == 334 * 144 and dates == expected_dates
          and not bad_days)
    audit.add(f"FINAL-{name}-DETAIL-COVERAGE", "PASS" if ok else "FAIL", "critical", name,
             {"source": str(detail.source), "source_kind": detail.source_kind, "numeric_scan": scan,
              "rows": len(rows), "days": len(dates), "date_start": str(dates[0]) if dates else None,
              "date_end": str(dates[-1]) if dates else None, "bad_days": bad_days,
              "48096 valid rows; exact 2025-02-01..12-31 sequence; each date has ordered unique positions 1..144 and labels exactly follow the physical or official convention.",
              str(detail.source), "Missing or duplicated intervals invalidate costs and SOC continuity.")
    return ok

```

```

def audit_physics(audit: Audit, name: str, rows: Sequence[dict[str, Any]], cfg: AuditConfig, adjusted: bool) -> None:
    if not rows:
        return
    maximum = defaultdict(float)
    minimum_soc = math.inf
    maximum_soc = -math.inf
    simultaneous = negative = 0
    prior_end: float | None = None
    for idx, row in enumerate(rows):
        purchase = row["adjusted"] if adjusted else row["plan"]
        balance = purchase + row["emergency"] + row["pv"] + row["discharge"] - row["load"] - row["charge"] - row["unused"]
        dynamic = row["soc_end"] - row["soc_start"] - cfg.efficiency * row["charge"] + row["discharge"] / cfg.efficiency
        maximum["balance"] = max(maximum["balance"], abs(balance))
        maximum["soc_dynamic"] = max(maximum["soc_dynamic"], abs(dynamic))
        maximum["step_power"] = max(maximum["step_power"], row["charge"], row["discharge"])
        if prior_end is not None:
            maximum["continuity"] = max(maximum["continuity"], abs(row["soc_start"] - prior_end))
        prior_end = row["soc_end"]
        minimum_soc = min(minimum_soc, row["soc_start"], row["soc_end"])
        maximum_soc = max(maximum_soc, row["soc_start"], row["soc_end"])
        simultaneous += row["charge"] > cfg.numeric_tolerance and row["discharge"] > cfg.numeric_tolerance
        negative += sum(row[field] < -cfg.numeric_tolerance for field in ("plan", "emergency", "charge", "discharge", "unused") + (("adjusted",) if adjusted else ()))
    endpoints = {"initial": rows[0]["soc_start"], "terminal": rows[-1]["soc_end"]}
    actual = {"balance_max_kwh": maximum["balance"], "soc_dynamic_max_kwh": maximum["soc_dynamic"],
             "continuity_max_kwh": maximum["continuity"], "soc_min_kwh": minimum_soc, "soc_max_kwh": maximum_soc,
             "step_power_max_kwh": maximum["step_power"], "simultaneous_count": int(simultaneous),
             "negative_count": int(negative), "endpoints": endpoints}
    tol = cfg.numeric_tolerance if all(isinstance(r.get("plan"), float) for r in rows) else cfg.displayed_tolerance
    if name == "result1.xlsx" or len(rows) == 334 * 144:
        endpoints_ok = abs(endpoints["initial"] - cfg.initial_soc_kwh) <= cfg.displayed_tolerance and abs(endpoints["terminal"] - cfg.terminal_soc_kwh) <=
    cfg.displayed_tolerance
    else:
        endpoints_ok = False
    ok = (maximum["balance"] <= cfg.displayed_tolerance and maximum["soc_dynamic"] <= cfg.displayed_tolerance
          and maximum["continuity"] <= cfg.displayed_tolerance and minimum_soc >= cfg.storage_min_kwh - cfg.displayed_tolerance
          and maximum_soc <= cfg.storage_max_kwh + cfg.displayed_tolerance
          and maximum["step_power"] <= cfg.step_power_limit_kwh + cfg.displayed_tolerance
          and simultaneous == 0 and negative == 0 and endpoints_ok)
    audit.add(f"FINAL-{name}-PHYSICS", "PASS" if ok else "FAIL", "critical", name, actual,
             f"Balance/SOC residual <= {cfg.displayed_tolerance}; continuous SOC; bounds; no simultaneous charge/discharge; endpoints {cfg.initial_soc_kwh}-
             >{cfg.terminal_soc_kwh}."
             "independent recomputation from dispatch detail", "Physical infeasibility invalidates the strategy.")

```

```

def audit_source_mapping(audit: Audit, name: str, rows: Sequence[dict[str, Any]], input_dir: Path, cfg: AuditConfig) -> None:
    """Match every realized load, PV and settlement price to the raw attachments."""
    maxima = {"load_kwh": 0.0, "pv_kwh": 0.0, "price": 0.0}
    errors: list[str] = []
    if name == "result1.xlsx":
        wb = load_workbook(input_dir / "附件 1.xlsx", data_only=True, read_only=True)
        raw = list(wb.active.iter_rows(min_row=2, values_only=True))
        wb.close()
        if len(rows) != 144:
            errors.append(f"detail rows={len(rows)}")
        else:

```

```

        for row, source in zip(rows, raw):
            maxima["price"] = max(maxima["price"], abs(row["price"] - required_number(source[1], "附件 1 price")))
            maxima["load_kwh"] = max(maxima["load_kwh"], abs(row["load"] - required_number(source[2], "附件 1 load") * cfg.interval_hours))
            maxima["pv_kwh"] = max(maxima["pv_kwh"], abs(row["pv"] - required_number(source[3], "附件 1 pv") * cfg.interval_hours))
        else:
            wb2 = load_workbook(input_dir / "附件 2.xlsx", data_only=True, read_only=True)
            wb4 = load_workbook(input_dir / "附件 4.xlsx", data_only=True, read_only=True)

            load_rows = {norm_date(r[0]): r[1:145] for r in wb2["小区负载"].iter_rows(min_row=2, values_only=True)}
            pv_rows = {norm_date(r[0]): r[1:145] for r in wb2["光伏发电实际功率"].iter_rows(min_row=2, values_only=True)}
            price_rows = {norm_date(r[0]): r[1:145] for r in wb4.active.iter_rows(min_row=2, values_only=True)}
            wb2.close(); wb4.close()

            wb1 = load_workbook(input_dir / "附件 1.xlsx", data_only=True, read_only=True)

            fixed_price = [required_number(r[1], "附件 1 price") for r in wb1.active.iter_rows(min_row=2, values_only=True)]
            wb1.close()
            for row in rows:
                day, pos = row["date"], int(row["position"]) - 1
                if day not in load_rows or not 0 <= pos < 144:
                    errors.append(f"missing raw key {day} position {pos + 1}")
                    continue

                maxima["load_kwh"] = max(maxima["load_kwh"], abs(row["load"] - required_number(load_rows[day][pos], "附件 2 load") * cfg.interval_hours))
                maxima["pv_kwh"] = max(maxima["pv_kwh"], abs(row["pv"] - required_number(pv_rows[day][pos], "附件 2 pv") * cfg.interval_hours))
                expected_price = required_number(price_rows[day][pos], "附件 4 price") if name.startswith("result4") else fixed_price[pos]
                maxima["price"] = max(maxima["price"], abs(row["price"] - expected_price))
            ok = not errors and max(maxima.values()) <= cfg.numeric_tolerance
            audit.add(f"FINAL-{name}-SOURCE-MAPPING", "PASS" if ok else "FAIL", "critical", name,
                    {"maximum_errors": maxima, "errors": errors[:30]},
                    f"Every realized load/PV energy and settlement price equals the same date/position in attachments within {cfg.numeric_tolerance}.",
                    "dispatch detail vs 附件 1/2/4", "An off-by-one interval or wrong date invalidates all costs and balances.")

def display_round(value: float, decimals: int) -> Decimal:
    quantum = Decimal(1).scaleb(-decimals)
    return Decimal(str(value)).quantize(quantum, rounding=ROUND_HALF_UP)

def decimal_sum(values: Iterable[float]) -> Decimal:
    """Sum serialized full-precision values deterministically in base 10."""
    return sum((Decimal(str(value)) for value in values), Decimal("0"))

def emergency_segments(rows: Sequence[dict[str, Any]], cfg: AuditConfig) -> list[dict[str, Any]]:
    """Filter display-zero records, then group adjacent retained positions by day."""
    result: list[dict[str, Any]] = []
    for day in date_range(MAIN_START, MAIN_END):
        day_rows = [row for row in rows if row["date"] == day]
        visible = [
            row for row in day_rows
            if row["emergency"] >= cfg.emergency_display_threshold_kwh
            and display_round(row["emergency"], cfg.display_decimals) != 0
        ]
        group: list[dict[str, Any]] = []
        for row in visible:
            if group and int(row["position"]) != int(group[-1]["position"]) + 1:
                result.append(segment_record(day, group, cfg))
                group = []
            group.append(row)
        if group:
            result.append(segment_record(day, group, cfg))
    return result

def segment_record(day: date, group: Sequence[dict[str, Any]], cfg: AuditConfig) -> dict[str, Any]:
    start_pos = int(group[0]["position"])
    end_pos = int(group[-1]["position"])
    start = (start_pos - 1) * 10
    end = end_pos * 10
    label = f"{start // 60:02d}:{start % 60:02d}" + ("24:00" if end == 1440 else f"{end // 60:02d}:{end % 60:02d}")
    full_decimal = decimal_sum(row["emergency"] for row in group)
    full = float(full_decimal)
    return {"date": day, "start_position": start_pos, "end_position": end_pos, "interval": label,
            "full_precision_kwh": full, "display_kwh": float(full_decimal.quantize(
                Decimal(1).scaleb(-cfg.display_decimals), rounding=ROUND_HALF_UP))}

```

```

def time_range_key(label: Any) -> tuple[int, int] | None:
    text = str(label).strip().replace(" : ", ":")
    match = re.match(r"^(?!(\d{1,2}):)(?!(\d{2}):)(?!(\d{1,2}):)(?!(\d{2}):)$", text)
    if not match:
        return None
    sh, sm, eh, em = map(int, match.groups())
    return sh * 60 + sm, eh * 60 + em

def read_emergency_sheet(path: Path) -> tuple[list[dict[str, Any]], dict[str, Any]]:
    wb = load_workbook(path, data_only=True, read_only=True)
    if "紧急购电量" not in wb.sheetnames:
        wb.close()
        return [], {"error_count": 1, "samples": ["缺少紧急购电量工作表"]}
    ws = wb["紧急购电量"]
    rows: list[dict[str, Any]] = []
    errors: list[str] = []
    current: date | None = None
    for row_num, raw in enumerate(ws.iter_rows(min_row=2, values_only=True), 2):
        if all(value in (None, "") for value in raw[:3]):
            continue
        if raw[0] not in (None, ""):
            try:
                current = norm_date(raw[0])
            except ValueError as exc:
                errors.append(f"row {row_num}: {exc}")
                current = None
        if current is None:
            errors.append(f"row {row_num}: 日期为空且无可继承日期")
            continue
        key = time_range_key(raw[1])
        if key is None:
            errors.append(f"row {row_num}: 无效连续区间 {raw[1]!r}")
            continue
        try:
            amount = required_number(raw[2], f"紧急购电量!C{row_num}")
        except ValueError as exc:
            errors.append(str(exc))
            continue
        rows.append({"date": current, "time_key": key, "interval": str(raw[1]).strip(), "amount": amount, "row": row_num})
    wb.close()
    return rows, {"error_count": len(errors), "samples": errors[:30]}

def audit_emergency(audit: Audit, name: str, workbook_path: Path, rows: Sequence[dict[str, Any]], cfg: AuditConfig, out_dir: Path) -> None:
    expected = emergency_segments(rows, cfg)
    actual, scan = read_emergency_sheet(workbook_path)
    missing: list[dict[str, Any]] = []
    extra: list[dict[str, Any]] = []
    amount_errors: list[dict[str, Any]] = []
    actual_map = {(row["date"], row["time_key"]): row for row in actual}
    expected_map = {(row["date"], time_range_key(row["interval"])): row for row in expected}
    for key, exp in expected_map.items():
        got = actual_map.get(key)
        if got is None:
            missing.append({"date": str(exp["date"]), "interval": exp["interval"], "display_kwh": exp["display_kwh"]})
        elif abs(got["amount"] - exp["display_kwh"]) > 0.5 * 10 ** (-cfg.display_decimals) + 1e-12:
            amount_errors.append({"date": str(exp["date"]), "interval": exp["interval"],
                                   "actual": got["amount"], "expected_display": exp["display_kwh"],
                                   "full_precision": exp["full_precision_kwh"]})
    for key, got in actual_map.items():
        if key not in expected_map:
            extra.append({"date": str(got["date"]), "interval": got["interval"], "amount": got["amount"]})
    order_expected = [(row["date"], time_range_key(row["interval"])) for row in expected]
    order_actual = [(row["date"], row["time_key"]) for row in actual]
    ok = scan["error_count"] == 0 and not missing and not extra and not amount_errors and order_actual == order_expected
    audit.add(f"FINAL-{name}-EMERGENCY-SEGMENTS", "PASS" if ok else "FAIL", "critical", name,
              {
                  "threshold_kwh": cfg.emergency_display_threshold_kwh, "rounding_rule": f"ROUND_HALF_UP to {cfg.display_decimals} decimals",
                  "detail_source": str(rows[0].get("_source", "dispatch detail")) if rows else "dispatch detail",
                  "expected_segments": len(expected), "actual_segments": len(actual), "sheet_scan": scan,
                  "missing_count": len(missing), "missing_samples": missing[:30], "extra_count": len(extra),
                  "extra_samples": extra[:30], "amount_error_count": len(amount_errors),
                  "amount_error_samples": amount_errors[:30], "order_exact": order_actual == order_expected,
                  "Every displayed segment exactly equals the contiguous non-display-zero full-precision records, in chronological order.",
                  f"{workbook_path}:紧急购电量", "A total-only comparison can hide omitted or incorrectly split intervals."
              })
    csv_path = out_dir / f"{Path(name).stem}_emergency_rebuilt.csv"
    with csv_path.open("w", newline="", encoding="utf-8-sig") as handle:
        writer = csv.DictWriter(handle, fieldnames=["date", "start_position", "end_position", "interval", "full_precision_kwh", "display_kwh"])
        writer.writeheader()
        for row in expected:
            writer.writerow({"row": row, "date": row["date"].isoformat()})

```

```

def attachment3_release_keys(input_dir: Path) -> set[tuple[date, str]]:
    """Return the release date/clock pairs physically present in Attachment 3."""
    path = input_dir / "附件 3.xlsx"
    if not path.is_file():
        return set()
    wb = load_workbook(path, data_only=True, read_only=True)
    try:
        ws = wb["Sheet1"]
        current: date | None = None
        releases: set[tuple[date, str]] = set()
        for row in ws.iter_rows(min_row=2, values_only=True):
            if row[0] not in (None, ""):
                current = norm_date(row[0])
            if current is not None:
                releases.add((current, norm_clock(row[1])))
        return releases
    finally:
        wb.close()

def load_versions(results_dir: Path, name: str,
                 forecast_release_keys: set[tuple[date, str]] | None = None
                 ) -> tuple[dict[str, list[dict[str, Any]]], Path | None, dict[str, Any]]:
    stem = Path(name).stem
    aliases = {stem.replace("-", "_"), stem.replace("result", "q").replace("-", "_")}
    mode = "q3" if name == "result3.xlsx" else "q4_3"
    candidates = [
        p for p in results_dir.rglob("*.csv")
        if "version" in p.name.lower()
        and (p.name.lower() == "decision_versions.csv" or any(a in p.name.lower().replace("-", "_") for a in aliases))
    ] if results_dir.exists() else []
    for path in sorted(candidates):
        with path.open(newline="", encoding="utf-8-sig") as handle:
            reader = csv.DictReader(handle)
            fields = set(reader.fieldnames or [])
            old_field = "previous_commitment_kwh" if "previous_commitment_kwh" in fields else "old_commitment_kwh"
            if not {"date", "position", old_field, "new_commitment_kwh"}.issubset(fields):
                continue
            formal_rows: list[dict[str, Any]] = []
            sensitivity_rows: list[dict[str, Any]] = []
            errors = []
            preview_next_day_rows = 0
            preview_recourse_rows = 0
            preview_commitment_violations = 0
            preview_target_date_violations = 0
            preview_grid_violations = 0
            preview_records: list[tuple[date, int, date, int]] = []
            preview_samples: list[str] = []
            filtered_other_runs = 0
            future_history_references = 0
            basis_counts: Counter[str] = Counter()
            forecast_release_checked = 0
            forecast_release_missing = 0
            forecast_release_parse_errors = 0
            forecast_release_time_mismatches = 0
            forecast_release_attachment_missing = 0
            forecast_release_samples: list[str] = []
            selected_version_rows = 0
            for row_num, row in enumerate(reader, 2):
                if row.get("mode") not in (None, "", mode):
                    continue
                if row.get("scenario") not in (None, "", "S3") or row.get("model_arm") not in (None, "", "all_three"):
                    filtered_other_runs += 1
                    continue
                selected_version_rows += 1
            try:
                day = norm_date(row["date"])
                if row.get("decision_hour") not in (None, ""):
                    decision_hour = int(required_number(row["decision_hour"], f"row {row_num} decision_hour"))
                else:
                    match = re.search(r"\b(\d{2}):00$", str(row.get("decision_time", "")))
                    if not match:
                        raise ValueError(f"row {row_num}: cannot derive decision hour")
                    decision_hour = int(match.group(1))
                decision_text = str(row.get("decision_time", "")).strip()
                forecast_text = str(row.get("forecast_issue_time", "")).strip()
                if not forecast_text:
                    forecast_release_missing += 1
                    if len(forecast_release_samples) < 30:
                        forecast_release_samples.append(f"row {row_num}: missing forecast_issue_time")
                else:
                    try:
                        decision_dt = datetime.fromisoformat(decision_text)
                        forecast_dt = datetime.fromisoformat(forecast_text)
                    except ValueError:
                        forecast_release_parse_errors += 1
                        if len(forecast_release_samples) < 30:
                            forecast_release_samples.append(

```

```

        f"row {row_num}: invalid decision/forecast time {decision_text!r}/{forecast_text!r}"
    else:
        forecast_release_checked += 1
        valid_decision = (decision_dt.date() == day and decision_dt.hour in {0, 6, 12, 18}
                          and decision_dt.minute == 0 and decision_dt.second == 0
                          and decision_dt.microsecond == 0 and decision_dt.hour == decision_hour)
        if not valid_decision or forecast_dt != decision_dt:
            forecast_release_time_mismatches += 1
            if len(forecast_release_samples) < 30:
                forecast_release_samples.append(
                    f"row {row_num}: decision={decision_text!r}, forecast={forecast_text!r}, date={day}")
            release_key = (forecast_dt.date(), forecast_dt.strftime("%H:%M"))
            if forecast_release_keys is not None and release_key not in forecast_release_keys:
                forecast_release_attachment_missing += 1
                if len(forecast_release_samples) < 30:
                    forecast_release_samples.append(
                        f"row {row_num}: Attachment 3 has no release {release_key[0]} {release_key[1]}")
            position = int(required_number(row["position"], f"row {row_num} position"))
            preview_next = str(row.get("is_preview_next_day", "0")).strip() in {"1", "true", "True"}
            preview_recourse = str(row.get("is_preview_recourse", "0")).strip() in {"1", "true", "True"}
            preview_next_day_rows += int(preview_next)
            preview_recourse_rows += int(preview_recourse)
            for field in ("load_history_dates", "price_history_dates", "pv_bias_history_dates", "residual_scenario_dates"):
                for token in filter(None, str(row.get(field, "")).split("|")):
                    if norm_date(token) >= day:
                        future_history_references += 1
            formal_old_raw = row.get(old_field)
            formal_new_raw = row.get("new_commitment_kwh")
            sensitivity_old_raw = row.get("sensitivity_old_commitment_kwh")
            sensitivity_new_raw = row.get("sensitivity_new_commitment_kwh")
            decision_basis = str(row.get("decision_basis", "")).strip()
            basis_counts[decision_basis or "<missing>"] += 1
            allowed_basis = {"next_block_stochastic", "deterministic_day_ahead_backup",
                           "deterministic_point_forecast_sensitivity", "next_day_preview"}
            if decision_basis not in allowed_basis:
                errors.append(f"row {row_num}: invalid decision_basis {decision_basis!r}")
            if preview_next:
                if any(value not in (None, "") for value in (formal_old_raw, formal_new_raw, sensitivity_old_raw, sensitivity_new_raw)):
                    errors.append(f"row {row_num}: next-day preview contains a commitment")
                preview_commitment_violations += 1
                if len(preview_samples) < 30:
                    preview_samples.append(f"row {row_num}: preview contains formal/sensitivity commitment")
            target_day: date | None = None
            try:
                target_day = norm_date(row.get("target_date"))
            except ValueError as exc:
                preview_target_date_violations += 1
                if len(preview_samples) < 30:
                    preview_samples.append(f"row {row_num}: invalid target_date: {exc}")
            else:
                if target_day <= day or target_day != day + timedelta(days=1):
                    preview_target_date_violations += 1
                if len(preview_samples) < 30:
                    preview_samples.append(
                        f"row {row_num}: target_date {target_day} is not the next day after {day}")
            try:
                preview_grid = required_number(row.get("preview_grid_kwh"), f"row {row_num} preview_grid_kwh")
            except ValueError as exc:
                preview_grid_violations += 1
                if len(preview_samples) < 30:
                    preview_samples.append(str(exc))
            else:
                if preview_grid < 0:
                    preview_grid_violations += 1
                    if len(preview_samples) < 30:
                        preview_samples.append(f"row {row_num}: negative preview_grid_kwh {preview_grid}")
            if target_day is not None:
                preview_records.append((day, decision_hour, target_day, position))
                continue
            common = {"date": day, "decision_hour": decision_hour, "position": position,
                    "decision_basis": decision_basis}
            if formal_new_raw not in (None, ""):
                formal_rows.append(("common",
                                    "previous": None if decision_hour == 0 and formal_old_raw in (None, "") else required_number(formal_old_raw, f"row {row_num} formal
previous"),
                                    "new": required_number(formal_new_raw, f"row {row_num} formal new"))
            elif formal_old_raw not in (None, ""):
                errors.append(f"row {row_num}: formal old exists without formal new")
            if sensitivity_new_raw not in (None, ""):
                sensitivity_rows.append(("common",
                                        "previous": None if decision_hour == 0 and sensitivity_old_raw in (None, "") else required_number(sensitivity_old_raw, f"row {row_num}
sensitivity previous"),
                                        "new": required_number(sensitivity_new_raw, f"row {row_num} sensitivity new"))
            elif sensitivity_old_raw not in (None, ""):
                errors.append(f"row {row_num}: sensitivity old exists without sensitivity new")
            except (ValueError, TypeError) as exc:
                errors.append(str(exc))
            preview_counts = Counter((day, hour) for day, hour, _, _ in preview_records)
            expected_preview_per_decision = {0: 0, 6: 36, 12: 72, 18: 108}
            preview_count_errors = sum(
                preview_counts[(day, hour)] != count
                for day in date_range(MAIN_START, MAIN_END)
                for hour, count in expected_preview_per_decision.items()
            )
            if not (day == MAIN_END and count > 0)

```

```

)
preview_count_errors += sum(preview_counts[(MAIN_END, hour)] != 0 for hour in expected_preview_per_decision)
expected_preview_records = [
    (day, hour, day + timedelta(days=1), position)
    for day in date_range(MAIN_START, MAIN_END - timedelta(days=1))
    for hour, count in expected_preview_per_decision.items()
    for position in range(1, count + 1)
]
preview_sequence_exact = preview_records == expected_preview_records
preview_sequence_sample: dict[str, Any] | None = None
if not preview_sequence_exact:
    for index in range(min(len(preview_records), len(expected_preview_records))):
        if preview_records[index] != expected_preview_records[index]:
            preview_sequence_sample = {"index": index, "actual": preview_records[index],
                                      "expected": expected_preview_records[index]}
            break
    if preview_sequence_sample is None:
        preview_sequence_sample = {"actual_rows": len(preview_records),
                                   "expected_rows": len(expected_preview_records)}
return {"formal": formal_rows, "sensitivity": sensitivity_rows, "path", {
    "error_count": len(errors), "samples": errors[:30],
    "preview_next_day_rows": preview_next_day_rows,
    "preview_recourse_rows": preview_recourse_rows,
    "preview_next_day_integrity": {
        "rows": preview_next_day_rows,
        "parsed_records": len(preview_records),
        "expected_rows": 71928,
        "expected_per_decision": expected_preview_per_decision,
        "decision_count_errors": preview_count_errors,
        "sequence_exact": preview_sequence_exact,
        "sequence_mismatch_sample": preview_sequence_sample,
        "commitment_violations": preview_commitment_violations,
        "target_date_violations": preview_target_date_violations,
        "preview_grid_violations": preview_grid_violations,
        "samples": preview_samples,
    },
    "filtered_other_runs": filtered_other_runs,
    "future_history_references": future_history_references,
    "decision_basis_counts": dict(basis_counts),
    "forecast_release": {
        "selected_rows": selected_version_rows,
        "checked_rows": forecast_release_checked,
        "missing_field": forecast_release_missing,
        "parse_errors": forecast_release_parse_errors,
        "decision_time_mismatches": forecast_release_time_mismatches,
        "attachment_release_missing": forecast_release_attachment_missing,
        "samples": forecast_release_samples,
    }
}}
return {"formal": [], "sensitivity": [], None, {"error_count": 1, "samples": ["未找到逐次承诺版本 CSV"]}]

def audit_settlement(audit: Audit, name: str, rows: Sequence[dict[str, Any]], results_dir: Path,
                    input_dir: Path, cfg: AuditConfig) -> None:
    if not rows:
        return
    final_components = defaultdict(float)
    final_row_error = 0.0
    component_row_errors = defaultdict(float)
    for row in rows:
        baseline, adjusted, price, emergency = row["plan"], row["adjusted"], row["price"], row["emergency"]
        down = max(baseline - adjusted, 0.0)
        up = max(adjusted - baseline, 0.0)
        components = {
            "normal_yuan": price * min(baseline, adjusted),
            "down_yuan": 0.5 * price * down,
            "up_yuan": 1.5 * price * up,
            "emergency_yuan": 5.0 * price * emergency,
        }
        for key, value in components.items():
            final_components[key] += value
        expected = sum(components.values())
        signed_adjustment = -components["down_yuan"] + components["up_yuan"]
        stored_expectations = {
            "plan_cost": price * baseline,
            "adjustment_cost": signed_adjustment,
            "emergency_cost": components["emergency_yuan"],
            "final_cost": expected,
        }
        for field, value in stored_expectations.items():
            if field in row:
                component_row_errors[field] = max(component_row_errors[field], abs(row[field] - value))
    if "final_cost" in row:
        final_row_error = max(final_row_error, abs(row["final_cost"] - expected))
    final_components["total_yuan"] = sum(final_components[key] for key in ("normal_yuan", "down_yuan", "up_yuan", "emergency_yuan"))
    required_stored = ("plan_cost", "adjustment_cost", "emergency_cost", "final_cost")
    stored_complete = all(field in row for field in required_stored) for row in rows
    final_ok = stored_complete and max(component_row_errors.values(), default=0.0) <= cfg.numeric_tolerance
    audit.add(f"FINAL-{name}-SETTLEMENT-FINAL", "PASS" if final_ok else "FAIL", "critical", name,
            {"components": dict(final_components), "stored_components_complete": stored_complete,
             "component_row_max_errors_yuan": dict(component_row_errors), "row_max_error_yuan": final_row_error,
             "p min(B,A)+0.5p(B-A)^+ + 1.5p(A-B)^+ + 5pQ on every row, full precision.",

```

```

"dispatch detail", "The main reported cost uses this convention.")

release_keys = attachment3_release_keys(input_dir)
ledgers, path, scan = load_versions(results_dir, name, release_keys)
release_scan = scan.get("forecast_release", {})
release_violations = sum(int(release_scan.get(field, 0)) for field in
    ("missing_field", "parse_errors", "decision_time_mismatches", "attachment_release_missing"))
release_unexamined = max(0, int(release_scan.get("selected_rows", 0))
    - int(release_scan.get("checked_rows", 0))
    - int(release_scan.get("missing_field", 0))
    - int(release_scan.get("parse_errors", 0)))
release_ok = path is not None and bool(release_keys) and release_violations == 0 and release_unexamined == 0
audit.add(f"FINAL-{name}-FORECAST-RELEASE", "PASS" if release_ok else "FAIL", "critical", name,
    {"version_source": str(path) if path is not None else None,
     "attachment_release_count": len(release_keys),
     "violations": release_violations, "unexamined_rows": release_unexamined, **release_scan},
    "Every selected decision version has forecast_issue_time exactly equal to its same-day 00/06/12/18 decision_time, and Attachment 3 contains that release.",
    str(path or results_dir), "A missing or shifted release timestamp can conceal future-information leakage.")
preview_scan = scan.get("preview_next_day_integrity", {})
preview_ok = (path is not None and int(preview_scan.get("rows", -1)) == 71928
    and int(preview_scan.get("parsed_records", -1)) == 71928
    and int(preview_scan.get("decision_count_errors", -1)) == 0
    and bool(preview_scan.get("sequence_exact", False))
    and all(int(preview_scan.get(field, -1)) == 0 for field in
        ("commitment_violations", "target_date_violations", "preview_grid_violations")))
audit.add(f"FINAL-{name}-NEXT-DAY-PREVIEW", "PASS" if preview_ok else "FAIL", "critical", name,
    {"version_source": str(path) if path is not None else None, **preview_scan},
    "Per mode: 333 days x (36+72+108) = 71,928 ordered next-day preview rows; target_date is the following day, positions are 1..36/72/108 at 06/12/18, all four
    commitment fields are blank, and preview_grid_kwh is finite non-negative.",
    str(path or results_dir), "Missing, duplicated or executable preview rows invalidate the cross-day rolling contract.")
if path is None:
    audit.add(f"FINAL-{name}-SETTLEMENT-SEQUENTIAL", "FAIL", "critical", name, scan,
        "Timestamped commitment versions exist and independently reproduce sequential settlement.",
        str(results_dir), "Sequential sensitivity cannot be authenticated without every revision.")
    return
detail_by_key = {(row["date"], int(row["position"])): row for row in rows}
formal = ledgers["formal"]
sensitivity = ledgers["sensitivity"]
formal_by_key = dict(tuple([date, int], list(dict[str, Any])) = defaultdict(list)
sensitivity_by_key = dict(tuple([date, int], list(dict[str, Any])) = defaultdict(list)
formal_past = sensitivity_past = 0
for version in formal:
    formal_by_key[(version["date"], version["position"])] = append(version)
    formal_past += version["position"] < version["decision_hour"] * 6 + 1
for version in sensitivity:
    sensitivity_by_key[(version["date"], version["position"])] = append(version)
    sensitivity_past += version["position"] < version["decision_hour"] * 6 + 1

# Formal execution ledger: exactly one 36-interval block per decision,
# chained from the 00:00 baseline and ending at executed dispatch.
formal_chain_errors = 0
formal_final_mismatches = 0
for key, row in detail_by_key.items():
    current = row["plan"]
    for version in sorted(formal_by_key.get(key, []), key=lambda x: x["decision_hour"]):
        if version["decision_hour"] == 0:
            if abs(version["new"] - row["plan"]) > cfg.numeric_tolerance:
                formal_chain_errors += 1
            elif version["previous"] is None or abs(version["previous"] - current) > cfg.numeric_tolerance:
                formal_chain_errors += 1
            current = version["new"]
        if abs(current - row["adjusted"]) > cfg.numeric_tolerance:
            formal_final_mismatches += 1
formal_counts = Counter((row["date"], row["decision_hour"]) for row in formal)
expected_formal_by_hour = {0: 144, 6: 36, 12: 36, 18: 36}
formal_decision_count_errors = sum(formal_counts[(day, hour)] != count for day in date_range(MAIN_START, MAIN_END) for hour, count in
expected_formal_by_hour.items())
formal_ok = (len(formal) == 334 * 252 and formal_decision_count_errors == 0 and formal_past == 0
    and formal_chain_errors == 0 and formal_final_mismatches == 0)

# Sequential sensitivity ledger: 00:00 baseline, followed by every
# remaining same-day deterministic proposal at 06/12/18.
total = 0.0
row_errors = 0.0
sensitivity_chain_errors = 0
for key, row in detail_by_key.items():
    base = row["plan"]
    cost = row["price"] * base + 5 * row["price"] * row["emergency"]
    current = base
    for version in sorted(sensitivity_by_key.get(key, []), key=lambda x: x["decision_hour"]):
        if version["decision_hour"] == 0:
            if abs(version["new"] - base) > cfg.numeric_tolerance:
                sensitivity_chain_errors += 1
            current = version["new"]
            continue
        if version["previous"] is None or abs(version["previous"] - current) > cfg.numeric_tolerance:
            sensitivity_chain_errors += 1
        cost += -0.5 * row["price"] * max(current - version["new"], 0.0)
        cost += 1.5 * row["price"] * max(version["new"] - current, 0.0)
        current = version["new"]

```

```

total += cost
if "sequential_cost" in row:
    row_errors = max(row_errors, abs(row["sequential_cost"] - cost))
expected_sensitivity_count = 334 * 360
sensitivity_counts = Counter((row["date"], row["decision_hour"]) for row in sensitivity)
expected_by_hour = {0: 144, 6: 108, 12: 72, 18: 36}
sensitivity_decision_count_errors = sum(sensitivity_counts[(day, hour)] != count for day in date_range(MAIN_START, MAIN_END) for hour, count in
expected_by_hour.items())
ok = (scan["error_count"] == 0 and scan.get("future_history_references", 0) == 0
and formal_ok and len(sensitivity) == expected_sensitivity_count
and sensitivity_decision_count_errors == 0 and sensitivity_past == 0
and sensitivity_chain_errors == 0
and all("sequential_cost" in row for row in rows) and row_errors <= cfg.numeric_tolerance)
audit.add(f"FINAL-{name}-SETTLEMENT-SEQUENTIAL", "PASS" if ok else "FAIL", "critical", name,
{"version_source": str(path), "version_scan": scan,
"formal_chain": {"rows": len(formal), "expected_rows": 334 * 252,
"expected_per_decision": expected_formal_by_hour,
"decision_count_errors": formal_decision_count_errors,
"past_interval_rewrites": int(formal_past),
"commitment_chain_errors": formal_chain_errors,
"final_dispatch_mismatches": formal_final_mismatches},
"sensitivity_chain": {"rows": len(sensitivity), "expected_rows": expected_sensitivity_count,
"expected_per_decision": expected_by_hour,
"decision_count_errors": sensitivity_decision_count_errors,
"past_interval_rewrites": int(sensitivity_past),
"commitment_chain_errors": sensitivity_chain_errors},
"future_history_references": scan.get("future_history_references", 0), "recomputed_total_yuan": total,
"stored_row_cost_present": all("sequential_cost" in row for row in rows),
"row_max_error_yuan": row_errors},
"Formal ledger: 252 versions/day (144 at 00:00, then 36 next-block changes at 06/12/18). Sensitivity ledger: 360 same-day versions/day (144/108/72/36),
independently recomputing initial pB plus every transition and emergency 5pQ; no past or next-day commitment.",
str(path), "The required sensitivity total depends on the entire commitment chain.")

```

```

def audit_unadjusted_cost(audit: Audit, name: str, rows: Sequence[dict[str, Any]], cfg: AuditConfig) -> None:
    maxima = defaultdict(float)
    totals = defaultdict(float)
    complete = all(all(field in row for field in ("plan_cost", "emergency_cost", "final_cost")) for row in rows)
    for row in rows:
        plan = row["price"] * row["plan"]
        emergency = 5.0 * row["price"] * row["emergency"]
        total = plan + emergency
        totals["plan_yuan"] += plan; totals["emergency_yuan"] += emergency; totals["total_yuan"] += total
    if complete:
        maxima["plan"] = max(maxima["plan"], abs(row["plan_cost"] - plan))
        maxima["emergency"] = max(maxima["emergency"], abs(row["emergency_cost"] - emergency))
        maxima["total"] = max(maxima["total"], abs(row["final_cost"] - total))
    ok = complete and max(maxima.values(), default=0.0) <= cfg.numeric_tolerance
    audit.add(f"FINAL-{name}-COST", "PASS" if ok else "FAIL", "critical", name,
{"stored_components_complete": complete, "row_max_errors_yuan": dict(maxima), "totals": dict(totals)},
"Every row independently reproduces plan pB, emergency 5pQ and their total from full precision.",
"dispatch detail", "Q2/Q4-2 reported costs must include all realized settlement terms.")

```

```

def audit_plan_sheet(audit: Audit, name: str, path: Path, template: Path) -> None:
    wb = load_workbook(path, data_only=True, read_only=True)
    twb = load_workbook(template, data_only=True, read_only=True)
    sheets = ["计划购电量"] + (["调整购电量"] if "调整购电量" in twb.sheetnames else [])
    expected_dates = date_range(MAIN_START, MAIN_END)
    for sheet in sheets:
        ws = wb[sheet]
        tws = twb[sheet]
        header = tuple(next(ws.iter_rows(values_only=True)))
        expected_header = tuple(next(tws.iter_rows(values_only=True)))
        if name == "result1.xlsx":
            rows = list(ws.iter_rows(min_row=2, values_only=True))
            labels = [row[0] for row in rows]
            expected_labels = [row[0] for row in tws.iter_rows(min_row=2, values_only=True)]
            _, numeric = scan_required_numbers((f"{sheet}!B{i}", row[1]) for i, row in enumerate(rows, 2))
            ok = header == expected_header and labels == expected_labels and len(set(labels)) == 144 and numeric["error_count"] == 0
            actual = {"header_exact": header == expected_header, "rows": len(rows), "unique_labels": len(set(labels)),
"label_order_exact": labels == expected_labels, "numeric_scan": numeric}
        else:
            rows = list(ws.iter_rows(min_row=2, values_only=True))
            dates: list[date] = []
            date_errors: list[str] = []
            for idx, row in enumerate(rows, 2):
                try:
                    dates.append(norm_date(row[0]))
                except ValueError as exc:
                    date_errors.append(f"row {idx}: {exc}")
            _, numeric = scan_required_numbers(
(f"{sheet}!R{r}C{c}", value)
for r, row in enumerate(rows, 2) for c, value in enumerate(row[1:147], 2)
)
            interval_header = header[1:145]
            ok = (header == expected_header and dates == expected_dates and len(set(dates)) == 334
and len(interval_header) == 144 and len(set(interval_header)) == 144

```

```

        and not date_errors and numeric["error_count"] == 0)
    actual = {"header_exact": header == expected_header, "days": len(dates), "unique_dates": len(set(dates)),
            "date_order_exact": dates == expected_dates, "date_errors": date_errors[:20],
            "intervals": len(interval_header), "unique_interval_labels": len(set(interval_header)),
            "numeric_scan": numeric}
    audit.add(f"FINAL-{name}-{sheet}-COVERAGE", "PASS" if ok else "FAIL", "critical", f"{name}/{sheet}",
            actual, "Exact template header and label order; exact date population; all required result cells finite numeric.",
            f"{path} vs {template}", "Official-table gaps or schema drift invalidate the submission.")
wb.close()
twb.close()

```

```

def audit_plan_values(audit: Audit, name: str, path: Path, rows: Sequence[dict[str, Any]], cfg: AuditConfig) -> None:
    """Reconcile all official 10-minute cells, daily energy and daily cost."""
    by_date: dict[date, list[dict[str, Any]]] = defaultdict(list)
    for row in rows:
        by_date[row["date"]].append(row)
    wb = load_workbook(path, data_only=True, read_only=True)
    targets = [{"计划购电量", "plan"}]
    if name in {"result3.xlsx", "result4-3.xlsx"}:
        targets.append(("调整购电量", "adjusted"))
    for sheet, field in targets:
        ws = wb[sheet]
        cell_error = 0.0
        energy_error = 0.0
        cost_error = 0.0
        errors: list[str] = []
        for excel_row, values in enumerate(ws.iter_rows(min_row=2, values_only=True), 2):
            try:
                day = norm_date(values[0])
            except ValueError as exc:
                errors.append(f"row {excel_row}: {exc}")
                continue
            detail_rows = by_date.get(day, [])
            if len(detail_rows) != 144:
                errors.append(f"row {excel_row}: {day} detail rows={len(detail_rows)}")
                continue
            try:
                displayed = [required_number(value, f"{sheet}!R{excel_row}C{col}") for col, value in enumerate(values[1:145], 2)]
                daily_energy = required_number(values[145], f"{sheet}!R{excel_row}C146")
                daily_cost = required_number(values[146], f"{sheet}!R{excel_row}C147")
            except ValueError as exc:
                errors.append(str(exc))
                continue
            expected_cells = [float(display_round(row[field], cfg.display_decimals)) for row in detail_rows]
            cell_error = max(cell_error, max(abs(a - b) for a, b in zip(displayed, expected_cells)))
            energy_expected = float(decimal_sum(row[field] for row in detail_rows).quantize(
                Decimal(1).scaleb(-cfg.display_decimals, rounding=ROUND_HALF_UP)))
            energy_error = max(energy_error, abs(daily_energy - energy_expected))
            if field == "adjusted":
                cost_expected = math.fsum(
                    row["price"] * min(row["plan"], row["adjusted"])
                    + 0.5 * row["price"] * max(row["plan"] - row["adjusted"], 0.0)
                    + 1.5 * row["price"] * max(row["adjusted"] - row["plan"], 0.0)
                    + 5.0 * row["price"] * row["emergency"] for row in detail_rows)
            else:
                # The official plan sheet labels this as planned purchase cost;
                # Q2/Q4-2 have no adjustment sheet, so their daily total also
                # carries emergency settlement. Q3/Q4-3 report it on the
                # adjusted sheet and keep the initial-plan cost separate.
                if name in {"result2.xlsx", "result4-2.xlsx"}:
                    cost_expected = math.fsum(row["price"] * row["plan"] + 5.0 * row["price"] * row["emergency"] for row in detail_rows)
                else:
                    cost_expected = math.fsum(row["price"] * row["plan"] for row in detail_rows)
            cost_error = max(cost_error, abs(daily_cost - float(display_round(cost_expected, cfg.display_decimals))))
            limit = 0.5 * 10 ** (-cfg.display_decimals) + 1e-12
            ok = not errors and cell_error <= limit and energy_error <= limit and cost_error <= limit
            audit.add(f"FINAL-{name}-{sheet}-VALUES", "PASS" if ok else "FAIL", "critical", f"{name}/{sheet}",
                    {"detail_field": field, "cell_max_error": cell_error, "daily_energy_max_error": energy_error,
                     "daily_cost_max_error": cost_error, "errors": errors[:30]},
                    f"All 10-minute cells and each daily energy/cost total equal the {cfg.display_decimals}-decimal half-up rendering of full-precision detail.",
                    f"{path}:{sheet}", "Official cells, aggregates and paper totals must share one frozen result source.")
wb.close()

```

```

def audit_q1_values(audit: Audit, path: Path, rows: Sequence[dict[str, Any]], cfg: AuditConfig) -> None:
    wb = load_workbook(path, data_only=True, read_only=True)
    ws = wb["计划购电量"]
    values = list(ws.iter_rows(min_row=2, values_only=True))
    errors: list[str] = []
    maximum = 0.0
    if len(values) != 144 or len(rows) != 144:
        errors.append(f"official rows={len(values)}, detail rows={len(rows)}")
    else:
        for idx, (official, detail) in enumerate(zip(values, rows), 2):
            try:

```

```

        actual = required_number(official[1], f"计划购电量!B{idx}")
    except ValueError as exc:
        errors.append(str(exc))
        continue
    maximum = max(maximum, abs(actual - float(display_round(detail["plan"], cfg.display_decimals))))
limit = 0.5 * 10 ** (-cfg.display_decimals) + 1e-12
audit.add("FINAL-result1.xlsx-计划购电量-VALUES", "PASS" if not errors and maximum <= limit else "FAIL", "critical",
        "result1.xlsx/计划购电量", {"cell_max_error": maximum, "errors": errors[:30]},
        "All 144 official quantities equal the four-decimal rendering of full-precision detail.",
        f"{path}:计划购电量", "Q1 official cells must trace to the frozen LP result.")
wb.close()

```

```

def audit_four_hour_sheet(audit: Audit, name: str, path: Path, rows: Sequence[dict[str, Any]], cfg: AuditConfig) -> None:
    """Validate every supplied four-hour row and all required date blocks."""
    wb = load_workbook(path, data_only=True, read_only=True)
    ws = wb["充放电量"]
    header = tuple(next(ws.iter_rows(values_only=True)))
    expected_header = ("时间段", "充电量", "放电量", "时刻", "储电量") if name == "result1.xlsx" else ("日期", "时间段", "充电量", "放电量", "时刻", "储电量")
    detail_by_date: dict[date, list[dict[str, Any]]] = defaultdict(list)
    for row in rows:
        detail_by_date[row["date"]].append(row)
    current: date | None = MAIN_START if name == "result1.xlsx" else None
    block_labels: dict[date, list[str]] = defaultdict(list)
    maximum = {"charge": 0.0, "discharge": 0.0}
    errors: list[str] = []
    observed_dates: list[date] = []
    for row_num, row in enumerate(ws.iter_rows(min_row=2, values_only=True), 2):
        if row[0] == ":" or (name != "result1.xlsx" and row[1] == ":"):
            continue
        if name == "result1.xlsx":
            label, c_raw, d_raw = row[0], row[1], row[2]
        else:
            if row[0] not in (None, ""):
                try:
                    current = norm_date(row[0])
                    if not observed_dates or observed_dates[-1] != current:
                        observed_dates.append(current)
                except ValueError as exc:
                    errors.append(f"row {row_num}: {exc}")
                    current = None
            label, c_raw, d_raw = row[1], row[2], row[3]
        if current is None:
            errors.append(f"row {row_num}: no date")
            continue
        if label not in FOUR_HOUR_LABELS:
            errors.append(f"row {row_num}: invalid four-hour label {label!r}")
            continue
        block_labels[current].append(label)
    try:
        actual_c = required_number(c_raw, f"充放电量 row {row_num} charge")
        actual_d = required_number(d_raw, f"充放电量 row {row_num} discharge")
    except ValueError as exc:
        errors.append(str(exc))
        continue
    day_rows = detail_by_date.get(current, [])
    if len(day_rows) != 144:
        errors.append(f"row {row_num}: detail rows for {current}={len(day_rows)}")
        continue
    group = FOUR_HOUR_LABELS.index(label)
    expected_c = float(decimal_sum(r["charge"] for r in day_rows[group * 24:(group + 1) * 24])).quantize(
        Decimal(1).scaleb(-cfg.display_decimals), rounding=ROUND_HALF_UP))
    expected_d = float(decimal_sum(r["discharge"] for r in day_rows[group * 24:(group + 1) * 24])).quantize(
        Decimal(1).scaleb(-cfg.display_decimals), rounding=ROUND_HALF_UP))
    maximum["charge"] = max(maximum["charge"], abs(actual_c - expected_c))
    maximum["discharge"] = max(maximum["discharge"], abs(actual_d - expected_d))
    bad_blocks = {str(day): labels for day, labels in block_labels.items() if labels != list(FOUR_HOUR_LABELS)}
    required_dates = {MAIN_START} if name == "result1.xlsx" else {MAIN_START, MAIN_START + timedelta(days=1), MAIN_END}
    if name in ["result3.xlsx", "result4-3.xlsx"]:
        required_dates.add(date(2025, 3, 20))
    missing_required = sorted(str(day) for day in required_dates - set(block_labels))
    limit = 0.5 * 10 ** (-cfg.display_decimals) + 1e-12
    ok = header == expected_header and not errors and not bad_blocks and not missing_required and max(maximum.values()) <= limit
    audit.add(f"FINAL-{name}-4H", "PASS" if ok else "FAIL", "high", f"{name}/充放电量",
            {"header_exact": header == expected_header, "date_blocks": len(block_labels), "observed_dates": [str(x) for x in observed_dates],
             "missing_required_dates": missing_required, "bad_blocks": dict(list(bad_blocks.items()))[:30]},
            "maximum_errors": maximum, "errors": errors[:30]},
            "Every included date has six ordered unique four-hour groups; required dates are present; charge/discharge equal full-precision sums rendered to four decimals.",
            f"{path}:充放电量", "Four-hour summaries are required outputs and must reconcile to the same detail.")
    wb.close()

```

```

def legacy_snapshot(audit: Audit, legacy_dir: Path, out_dir: Path, source_root: Path, project_root: Path, cfg: AuditConfig) -> None:
    sources = ("audit_summary.json", "rolling_audit_summary.json")

```

```

payload: dict[str, Any] = {'note': "Immutable snapshot of legacy statuses; final results do not overwrite or clear these findings.", "sources": {}}
missing = []
status_counts: Counter[str] = Counter()
nonpass: list[str] = []
for name in sources:
    path = legacy_dir / name
    if not path.is_file():
        missing.append(name)
        continue
    source = json.loads(path.read_text(encoding="utf-8"))
    checks = source.get("checks", [])
    status_counts.update(check.get("status", "UNKNOWN") for check in checks)
    nonpass.extend(check.get("id", "UNKNOWN") for check in checks if check.get("status") != "PASS")
    payload["sources"].append({'path': str(path), "sha256": sha256(path), "status_counts": source.get("status_counts", {}), "checks": checks})
payload["status_counts"] = dict(status_counts)
payload["nonpass_check_ids"] = nonpass
payload["missing_sources"] = missing
# The v1 audit compared emergency totals only. Preserve the stricter
# retrospective findings separately so a new PASS cannot conceal them.
supplemental = Audit()
for filename in ("result3.xlsx", "result4-3.xlsx"):
    workbook_path = source_root / filename
    if not workbook_path.is_file():
        continue
    detail = load_xlsx_detail(workbook_path, True)
    if detail is None:
        continue
    rows, scan = parse_detail(detail, True)
    if scan["error_count"] == 0:
        for row in rows:
            row["source"] = str(detail.source)
            audit_emergency(supplemental, f"legacy-{filename}", workbook_path, rows, cfg, out_dir)
payload["supplemental_exact_emergency_checks"] = [asdict(check) for check in supplemental.checks]
safe_payload = sanitize_output(payload, project_root, source_root)
(out_dir / "legacy_audit_snapshot.json").write_text(json.dumps(safe_payload, ensure_ascii=False, indent=2), encoding="utf-8")
audit.add("LEGACY-AUDIT-PRESERVED", "PASS" if not missing else "FAIL", "high", "outputs/c_audit_v1",
        {'source_files': len(payload["sources"]), "missing": missing, "status_counts": dict(status_counts),
         "nonpass_count": len(nonpass)},
        "Legacy checks and all non-PASS statuses are copied verbatim with source hashes.",
        str(out_dir / "legacy_audit_snapshot.json"), "New passes must not erase historical failures or unverifiable claims.")

```

```

def find_workbook(results_dir: Path, name: str) -> Path | None:
    direct = results_dir / name
    if direct.is_file():
        return direct
    matches = [path for path in results_dir.glob(name) if "audit" not in path.parts] if results_dir.exists() else []
    return sorted(matches, key=lambda p: len(str(p)))[0] if matches else None

```

```

def audit_results(audit: Audit, results_dir: Path, template_dir: Path, input_dir: Path, cfg: AuditConfig, out_dir: Path) -> None:
    for name in RESULT_NAMES:
        path = find_workbook(results_dir, name)
        if path is None:
            audit.add(f"FINAL-{name}", "PENDING", "critical", name, {'exists': False},
                    "Formal result workbook exists; rerun the same audit after generation.", str(results_dir),
                    "Stage report only; final delivery gate is not yet evaluated.")
            continue
        template = template_dir / name
        audit_schema(audit, path, template, f"FINAL-{name}")
        formula_errors = workbook_formula_errors(path)
        audit.add(f"FINAL-{name}-FORMULA-ERRORS", "PASS" if formula_errors["error_count"] == 0 else "FAIL", "critical", name,
                formula_errors, "No formula-error cells in formula or cached-value views.", str(path),
                "Formula errors can be hidden by displayed totals.")
        audit_plan_sheet(audit, name, path, template)
        if name == "result1.xlsx":
            detail = locate_detail(results_dir, path, False)
            if detail is None:
                audit.add("FINAL-result1.xlsx-DETAIL", "FAIL", "critical", name, {"found": False},
                        "Full-precision Q1 dispatch detail exists.", str(results_dir),
                        "Q1 feasibility and official values cannot be independently traced.")
                continue
            source_ok = detail.source_kind == "full_precision_csv"
            audit.add("FINAL-result1.xlsx-DETAIL-SOURCE", "PASS" if source_ok else "FAIL", "critical", name,
                    {'source': str(detail.source), "source_kind": detail.source_kind},
                    "Independent full-precision dispatch CSV.", str(detail.source),
                    "Rounded workbook cells cannot prove LP feasibility at full precision.")
            rows, scan = parse_detail(detail, False)
            positions = [int(row["position"]) for row in rows]
            coverage_ok = scan["error_count"] == 0 and len(rows) == 144 and positions == list(range(1, 145))
            audit.add("FINAL-result1.xlsx-DETAIL-COVERAGE", "PASS" if coverage_ok else "FAIL", "critical", name,
                    {'source': str(detail.source), "scan": scan, "rows": len(rows),
                     "unique_positions": len(set(positions)), "ordered_positions": positions == list(range(1, 145))},
                    "144 valid rows with ordered unique positions 1..144.", str(detail.source),
                    "Q1 gaps invalidate feasibility and cost.")
            if coverage_ok:
                for row in rows:
                    row["date"] = MAIN_START
                audit_physics(audit, name, rows, cfg, False)

```

```

        audit_source_mapping(audit, name, rows, input_dir, cfg)
        audit_q1_values(audit, path, rows, cfg)
        audit_four_hour_sheet(audit, name, path, rows, cfg)
        audit_unadjusted_cost(audit, name, rows, cfg)
        continue
    adjusted = name in ["result3.xlsx", "result4-3.xlsx"]
    detail = locate_detail(results_dir, path, adjusted)
    if detail is None:
        audit.add(f"FINAL-{name}-DETAIL", "FAIL", "critical", name, {"found": False},
            "Full-precision dispatch detail exists with the required unified fields.", str(results_dir),
            "Cannot independently recompute feasibility, costs or aggregates.")
        continue
    source_ok = detail.source_kind == "full_precision_csv"
    audit.add(f"FINAL-{name}-DETAIL-SOURCE", "PASS" if source_ok else "FAIL", "critical", name,
        {"source": str(detail.source), "source_kind": detail.source_kind},
        "Independent full-precision dispatch CSV, not a rounded workbook-detail fallback",
        str(detail.source), "Rounded cells cannot authenticate full-precision interval grouping or costs.")
    rows, scan = parse_detail(detail, adjusted)
    for row in rows:
        row["_source"] = str(detail.source)
    twb = load_workbook(template, data_only=True, read_only=True)
    official_labels = [str(value) for value in list(next(twb["计划购电量"].iter_rows(values_only=True)))[1:145])]
    twb.close()
    if audit_detail_coverage(audit, name, rows, scan, detail, official_labels):
        audit_dispatch_causality(audit, name, rows)
        audit_physics(audit, name, rows, cfg, adjusted)
        audit_source_mapping(audit, name, rows, input_dir, cfg)
        audit_plan_values(audit, name, path, rows, cfg)
        audit_four_hour_sheet(audit, name, path, rows, cfg)
        audit_emergency(audit, name, path, rows, cfg, out_dir)
        if adjusted:
            audit_settlement(audit, name, rows, results_dir, input_dir, cfg)
        else:
            audit_unadjusted_cost(audit, name, rows, cfg)

def write_reports(audit: Audit, out_dir: Path, config_path: Path, cfg: AuditConfig, raw_config: dict[str, str], config_parser: str,
    source_root: Path, results_dir: Path) -> int:
    project_root = Path(__file__).resolve().parents[2]
    counts = Counter(check.status for check in audit.checks)
    final_checks = [check for check in audit.checks if check.id.startswith("FINAL-")]
    final_fail = [check.id for check in audit.checks if check.status == "FAIL"]
    final_pending = [check.id for check in final_checks if check.status == "PENDING"]
    gate = "PASS" if not final_fail and not final_pending else ("PENDING" if not final_fail else "FAIL")
    safe_checks = [sanitize_output(asdict(check), project_root, source_root) for check in audit.checks]
    payload = {
        "generated_at": datetime.now().astimezone().isoformat(),
        "audit_version": "c_final_v1-audit-1",
        "source_root": "<source_root>",
        "results_dir": sanitize_output(str(results_dir), project_root, source_root),
        "config_path": sanitize_output(str(config_path), project_root, source_root),
        "config_exists": config_path.is_file(),
        "config_parser": config_parser,
        "config_explicit_keys": sorted(raw_config),
        "effective_config": asdict(cfg),
        "emergency_display_rule": {
            "display_decimals": cfg.display_decimals,
            "threshold_kwh": cfg.emergency_display_threshold_kwh,
            "rule": "A ten-minute record is omitted only when ROUND_HALF_UP(quantity, display_decimals) equals zero; grouping occurs after this filter.",
        },
        "check_count": len(audit.checks),
        "status_counts": dict(counts),
        "formal_delivery_gate": gate,
        "formal_fail_check_ids": final_fail,
        "formal_pending_check_ids": final_pending,
        "checks": safe_checks,
    }
    (out_dir / "audit_summary.json").write_text(json.dumps(payload, ensure_ascii=False, indent=2, encoding="utf-8"))
    with (out_dir / "audit_checks.csv").open("w", newline="", encoding="utf-8-sig") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(asdict(audit.checks[0])))
        writer.writeheader()
        writer.writerows(safe_checks)
    safe_config_path = sanitize_output(str(config_path), project_root, source_root)
    safe_results_dir = sanitize_output(str(results_dir), project_root, source_root)
    safe_out_dir = sanitize_output(str(out_dir), project_root, source_root)
    lines = [
        "# C 题最终独立审计报告", "", f"- 正式交付门 : **{gate}**", f"- 检查数 : {len(audit.checks)}",
        f"- 状态计数 : {dict(counts)}", f"- 配置 : {safe_config_path}" ( '{存在' if config_path.is_file() else '不存在, 使用显式默认值'} ; 解析器 '{config_parser}' ),
        f"- 紧急购电显示阈值 : {cfg.emergency_display_threshold_kwh} kWh ; 四位小数 'ROUND_HALF_UP' 后为 0 才省略。",
        "- 审计策略 : 任何必填空值、非数值、非有限值或公式错误先记 FAIL , 不进入数值计算 ; 绝不静默补零。", "",
        "## 正式结果门", "",
    ]
    for check in final_checks:
        lines.append(f"- {check.status}` {check.id}` : {check.scope}")
    lines.extend(["", "## 全部非 PASS 项", ""])
    nonpass = [check for check in audit.checks if check.status != "PASS"]
    if nonpass:

```

```

        for check in nonpass:
            lines.append(f"- {check.status}` `{check.id}` `({check.severity})` : {check.impact}")
        else:
            lines.append("- 无。")
        lines.extend(["", "## 旧成果状态", "",
            "旧审计结果逐项保存在 `legacy_audit_snapshot.json`。新版正式结果通过不会删除、覆盖或改写旧 FAIL/WARN/NOT_VERIFIABLE。该文件还保存用新版逐段规则回放旧 result3/result4-3 的补充检查；旧表的 0.0009 kWh 漏项和区间拆分继续保留为失败证据。", "",
            "## 复跑命令", "", "```bash",
            "python3 -m src.audit.audit_final --config config/final.yaml --results-dir outputs/c_final_v1 --output-dir outputs/c_final_v1/audit",
            "```", ""])
        (out_dir / "audit_report.md").write_text("\n".join(lines), encoding="utf-8")
        return 0 if gate in {"PASS", "PENDING"} else 2

def parse_args(argv: Sequence[str] | None = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    project = Path(__file__).resolve().parents[2]
    parser.add_argument("--source-root", type=Path, default=None,
        help="Raw material directory; defaults to source_root in config or CUMCM_SOURCE_ROOT")
    parser.add_argument("--results-dir", type=Path, default=project / "outputs" / "c_final_v1")
    parser.add_argument("--output-dir", type=Path, default=project / "outputs" / "c_final_v1" / "audit")
    parser.add_argument("--config", type=Path, default=project / "config" / "final.yaml")
    parser.add_argument("--legacy-dir", type=Path, default=project / "outputs" / "c_audit_v1")
    return parser.parse_args(argv)

def main(argv: Sequence[str] | None = None) -> int:
    args = parse_args(argv)
    args.output_dir.mkdir(parents=True, exist_ok=True)
    cfg, raw_config, config_parser = load_config(args.config)
    project_root = Path(__file__).resolve().parents[2]
    raw_source_root = args.source_root or raw_config.get("source_root") or os.environ.get("CUMCM_SOURCE_ROOT")
    if not raw_source_root:
        raise ValueError("未指定 source_root；请写入配置、传 --source-root 或设置 CUMCM_SOURCE_ROOT")
    source_root = Path(raw_source_root)
    if not source_root.is_absolute():
        source_root = project_root / source_root
    audit = Audit()
    derived_threshold = 0.5 * 10 ** (-cfg.display_decimals)
    threshold_ok = abs(cfg.emergency_display_threshold_kwh - derived_threshold) <= max(1e-15, derived_threshold * 1e-12)
    audit.add("CONFIG-EMERGENCY-DISPLAY-THRESHOLD", "PASS" if threshold_ok else "FAIL", "critical", "config/final.yaml",
        {"configured_kwh": cfg.emergency_display_threshold_kwh, "derived_kwh": derived_threshold,
        "display_decimals": cfg.display_decimals, "config_exists": args.config.is_file()},
        "Threshold equals half one unit in the configured display precision.", str(args.config),
        "A larger threshold may omit a visible record; a smaller threshold changes declared grouping semantics.")
    input_dir = source_root / "C 题" / "附件"
    template_dir = input_dir / "附件 5"
    audit_attachment_inputs(audit, input_dir)
    audit_official_templates(audit, template_dir)
    legacy_snapshot(audit, args.legacy_dir, args.output_dir, source_root, project_root, cfg)
    audit_results(audit, args.results_dir, template_dir, input_dir, cfg, args.output_dir)
    code = write_reports(audit, args.output_dir, args.config, cfg, raw_config, config_parser, source_root, args.results_dir)
    print(json.dumps({"status_counts": dict(Counter(check.status for check in audit.checks)),
        "formal_delivery_gate": "FAIL" if code else ("PENDING" if any(check.status == "PENDING" for check in audit.checks if check.id.startswith("FINAL-")) else
        "PASS"),
        "output_dir": str(args.output_dir),
        "emergency_display_threshold_kwh": cfg.emergency_display_threshold_kwh}, ensure_ascii=False))
    return code

if __name__ == "__main__":
    raise SystemExit(main())

```

src/model/__init__.py

"""C-problem causal microgrid model and reproducible rolling experiments."""

```

from .config import load_config
from .data import DataBundle, load_data
from .rolling import RunSpec, simulate

```

```
__all__ = ["DataBundle", "RunSpec", "load_config", "load_data", "simulate"]
```

src/model/cli.py

```
from __future__ import annotations

import argparse
import json
from pathlib import Path

from .config import load_config
from .data import load_data
from .experiments import run_and_write_all, run_and_write_official, run_stability_parallel, run_pre_stability_parallel

def main() -> int:
    ap=argparse.ArgumentParser(description="Rebuild the C-problem causal rolling model from official attachments")
    ap.add_argument("--config",default="config/final.yaml")
    ap.add_argument("--skip-stability",action="store_true",help="skip the 36 full-year solver/tiebreak sensitivity runs")
    ap.add_argument("--scope",choices=("official","all","pre-parallel","stability"),default="all")
    args=ap.parse_args()
    cfg=load_config(args.config); data=load_data(cfg)
    if args.scope=="official": summary=run_and_write_official(data,cfg)
    elif args.scope=="pre-parallel":
        pre=run_pre_stability_parallel(data,cfg); summary={"results":pre["experiments"],"stability":[]}
    elif args.scope=="stability":
        stability=run_stability_parallel(data,cfg); summary={"results":[],"stability":stability}
    else: summary=run_and_write_all(data,cfg,include_stability=not args.skip_stability)
    print(json.dumps({"status":"ok","scope":args.scope,"output_root":cfg["_output_root"],"results":len(summary["results"]),"stability_runs":len(summary.get("stability",
    []))},ensure_ascii=False))
    return 0

if __name__=="__main__": raise SystemExit(main())
```

src/model/config.py

```
from __future__ import annotations

import json
import os
from pathlib import Path
from typing import Any

PROJECT_ROOT = Path(__file__).resolve().parents[2]

def load_config(path: str | Path) -> dict[str, Any]:
    """Load final.yaml.

    The file intentionally uses JSON syntax, which is a strict subset of YAML. This
    keeps the pipeline dependency-free while remaining readable by YAML tooling.
    """
    p = Path(path)
    if not p.is_absolute():
        p = PROJECT_ROOT / p
    cfg = json.loads(p.read_text(encoding="utf-8"))
    cfg["_config_path"] = str(p.resolve())
    source_value=os.environ.get("CUMCM_SOURCE_ROOT",cfg["source_root"])
    source=Path(source_value)
    if not source.is_absolute(): source=PROJECT_ROOT/source
    cfg["_source_root"]=str(source.resolve())
    out = Path(cfg["output_root"])
    if not out.is_absolute():
        out = PROJECT_ROOT / out
    cfg["_output_root"] = str(out.resolve())
    return cfg
```

src/model/data.py

```
from __future__ import annotations
```

```

from dataclasses import dataclass
from datetime import date, datetime, time, timedelta
from pathlib import Path
from typing import Any

import numpy as np
from openpyxl import load_workbook

def _date(value: Any) -> date:
    if isinstance(value, datetime):
        return value.date()
    if isinstance(value, date):
        return value
    return datetime.strptime(str(value).strip().replace(".", "-").replace("/", "-"), "%Y-%m-%d").date()

def _hm(value: Any) -> str:
    if isinstance(value, time):
        return f"{value.hour:02d}:{value.minute:02d}"
    return str(value).strip()

def _numeric(values: list[Any], label: str) -> np.ndarray:
    bad = [i for i, v in enumerate(values) if v is None or v == "" or isinstance(v, bool)]
    if bad:
        raise ValueError(f"{label}: blank/non-numeric cells at offsets {bad[:10]}")
    try:
        arr = np.asarray(values, dtype=float)
    except (TypeError, ValueError) as exc:
        raise ValueError(f"{label}: non-numeric cell present") from exc
    if not np.all(np.isfinite(arr)):
        raise ValueError(f"{label}: NaN/Inf present")
    return arr

@dataclass(frozen=True)
class DataBundle:
    dates: tuple[date, ...]
    source_timestamps: tuple[str, ...]
    official_labels: tuple[str, ...]
    fixed_price: np.ndarray
    q1_load_kw: np.ndarray
    q1_pv_kw: np.ndarray
    load_kw: np.ndarray
    pv_kw: np.ndarray
    realtime_price: np.ndarray
    pv_forecast_kw: dict[tuple[date, int], np.ndarray]

@property
def date_index(self) -> dict[date, int]:
    return {d: i for i, d in enumerate(self.dates)}

def time_rows(self) -> list[dict[str, Any]]:
    rows: list[dict[str, Any]] = []
    for di, day in enumerate(self.dates):
        for pos in range(144):
            start = pos * 10
            end = (pos + 1) * 10
            rows.append({
                "date": day.isoformat(),
                "position": pos + 1,
                "global_position": di * 144 + pos,
                "source_timestamp": self.source_timestamps[pos],
                "physical_interval": f"{start//60:02d}:{start%60:02d}-{end//60:02d}:{end%60:02d}",
                "official_label": self.official_labels[pos],
            })
    return rows

def _read_matrix(path: Path, sheet: str) -> tuple[tuple[str, ...], tuple[date, ...], np.ndarray]:
    ws = load_workbook(path, data_only=True, read_only=True)[sheet]
    rows = ws.iter_rows(values_only=True)

```

```

header = next(rows)
if len(header) < 145:
    raise ValueError(f"{path.name}/{sheet}: expected 145 columns")
headers = tuple(_hm(x) for x in header[1:145])
dates: list[date] = []
matrix: list[np.ndarray] = []
for ri, row in enumerate(rows, start=2):
    if row[0] in (None, ""):
        raise ValueError(f"{path.name}/{sheet}!A{ri}: blank date")
    dates.append(_date(row[0]))
    matrix.append(_numeric(list(row[1:145]), f"{path.name}/{sheet} row {ri}"))
out = np.vstack(matrix)
if out.shape != (365, 144) or len(set(dates)) != 365:
    raise ValueError(f"{path.name}/{sheet}: expected 365 unique days x 144 intervals, got {out.shape}")
return headers, tuple(dates), out

```

```

def load_data(cfg: dict[str, Any]) -> DataBundle:
    root = Path(cfg["_source_root"]) / "C 题" / "附件"

    ws1 = load_workbook(root / "附件 1.xlsx", data_only=True, read_only=True).active
    rows1 = list(ws1.iter_rows(values_only=True))
    if len(rows1) != 145:
        raise ValueError("附件 1: expected header + 144 intervals")
    source_timestamps = tuple(_hm(r[0]) for r in rows1[1:])
    fixed_price = _numeric([r[1] for r in rows1[1:]], "附件 1/电价")
    q1_load = _numeric([r[2] for r in rows1[1:]], "附件 1/负载")
    q1_pv = _numeric([r[3] for r in rows1[1:]], "附件 1/光伏")

    h1, dates, load = _read_matrix(root / "附件 2.xlsx", "小区负载")
    h2, dates2, pv = _read_matrix(root / "附件 2.xlsx", "光伏发电实际功率")
    h3, dates3, rt = _read_matrix(root / "附件 4.xlsx", "Sheet1")
    if not (dates == dates2 == dates3 and h1 == h2 == h3):
        raise ValueError("附件 2/4 date or timestamp axes differ")
    expected = tuple(date(2025, 1, 1) + timedelta(days=i) for i in range(365))
    if dates != expected:
        raise ValueError("附件 2/4 dates are not exactly 2025-01-01..2025-12-31")

    template = load_workbook(root / "附件 5" / "result2.xlsx", data_only=True, read_only=True)["计划购电量"]
    official = tuple(str(x.value) for x in next(template.iter_rows())[1:145])
    if len(official) != 144 or len(set(official)) != 144:
        raise ValueError("official result template does not contain 144 unique interval labels")

    ws3 = load_workbook(root / "附件 3.xlsx", data_only=True, read_only=True).active
    forecasts: dict[tuple[date, int], np.ndarray] = {}
    current: date | None = None
    for ri, row in enumerate(ws3.iter_rows(min_row=2, values_only=True), start=2):
        if row[0] not in (None, ""):
            current = _date(row[0])
        if current is None:
            raise ValueError(f"附件 3 row {ri}: missing carried date")
        issue_text = str(row[1]).strip()
        try:
            issue = int(issue_text.split(":", 1)[0])
        except Exception as exc:
            raise ValueError(f"附件 3 row {ri}: invalid issue time {row[1]!r}") from exc
        vals = _numeric(list(row[2:26]), f"附件 3 row {ri}")
        key = (current, issue)
        if key in forecasts:
            raise ValueError(f"附件 3 duplicate forecast {key}")
        forecasts[key] = vals
    expected_keys = {(d, h) for d in dates for h in (0, 6, 12, 18)}
    if set(forecasts) != expected_keys:
        raise ValueError(f"附件 3 forecast coverage mismatch: missing={len(expected_keys-set(forecasts))}, extra={len(set(forecasts)-expected_keys)}")
    return DataBundle(dates, source_timestamps, official, fixed_price, q1_load, q1_pv, load, pv, rt, forecasts)

```

src/model/experiments.py

```
from __future__ import annotations
```

```

import csv
from concurrent.futures import ProcessPoolExecutor, as_completed
from dataclasses import asdict
from datetime import date, timedelta
import hashlib

```

```
import json
from pathlib import Path
import platform
import sys
from typing import Any, Iterable
```

```
import numpy as np
import scipy
```

```
from .data import DataBundle
from .forecast import profile_horizon
from .lp import solve_deterministic
from .rolling import RunSpec, simulate, _solve
```

```
def _write_csv(path: Path, rows: list[dict[str, Any]]) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    if not rows: raise ValueError("refusing to write empty CSV {path}")
    with path.open("w", newline="", encoding="utf-8-sig") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)
```

```
def _sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for block in iter(lambda: f.read(1<<20), b''): h.update(block)
    return h.hexdigest()
```

```
def solve_q1(data: DataBundle, cfg: dict[str, Any]) -> list[dict[str, Any]]:
    dt = float(cfg["storage"]["interval_minutes"])/60; soc0 = float(cfg["study"]["initial_soc_kwh"])
    sol = solve_deterministic(data.q1_load_kw*dt, data.q1_pv_kw*dt, data.fixed_price, soc0, np.full(144, np.nan),
        "force_target", float(cfg["study"]["terminal_soc_kwh"]), 0.0, cfg)
    starts = np.r_[soc0, sol.soc_end[-1]]; rows = []
    for j in range(144):
        cost = data.fixed_price[j]*sol.grid[j]
        curtail_pv = min(float(sol.curtail[j]), float(data.q1_pv_kw[j]*dt)); planned_unused_nonpv = max(float(sol.curtail[j]) - float(data.q1_pv_kw[j]*dt), 0.0)
        rows.append({
            "date": "2025-01-01", "position": j+1, "source_timestamp": data.source_timestamps[j],
            "physical_interval": f"{j*10//60:02d}-{(j+1)*10//60:02d}-{(j+1)*10%60:02d}", "official_label": data.official_labels[j],
            "plan_00_grid_kwh": float(sol.grid[j]), "final_adjusted_grid_kwh": float(sol.grid[j]), "adjustment_kwh": 0.0,
            "charge_kwh": float(sol.charge[j]), "discharge_kwh": float(sol.discharge[j]), "soc_start_kwh": float(starts[j]),
            "soc_end_kwh": float(sol.soc_end[j]), "curtail_pv_kwh": curtail_pv, "planned_unused_nonpv_kwh": planned_unused_nonpv,
            "forecast_load_kwh": float(data.q1_load_kw[j]*dt), "forecast_pv_kwh": float(data.q1_pv_kw[j]*dt),
            "actual_load_kwh": float(data.q1_load_kw[j]*dt), "actual_pv_kwh": float(data.q1_pv_kw[j]*dt),
            "forecast_price": float(data.fixed_price[j]), "settlement_price": float(data.fixed_price[j]),
            "emergency_kwh": 0.0, "unused_surplus_kwh": 0.0, "plan_cost_yuan": float(cost), "adjustment_cost_yuan": 0.0,
            "emergency_cost_yuan": 0.0, "final_relative_cost_yuan": float(cost), "sequential_cost_yuan": float(cost),
            "decision_hour": 0, "history_cutoff_date": "", "model_arm": "deterministic_q1", "scenario": "Q1",
        })
    return rows
```

```
def perfect_information_controls(data: DataBundle, cfg: dict[str, Any]) -> list[dict[str, Any]]:
    """Fair Feb-Dec lower bounds with the same 6000 -> 6000 endpoints"""
    start = date.fromisoformat(cfg["study"]["official_start"]); i0 = data.date_index[start]
    dt = float(cfg["storage"]["interval_minutes"])/60; n = (len(data.dates) - i0)*144
    load = data.load_kw[i0:].reshape(-1)*dt; pv = data.pv_kw[i0:].reshape(-1)*dt
    fixed = np.tile(data.fixed_price, len(data.dates) - i0); rt = data.realtime_price[i0:].reshape(-1)
    out = []
    for mode, price in (("q2", fixed), ("q4_2", rt)):
        sol = solve_deterministic(load, pv, price, float(cfg["study"]["initial_soc_kwh"]), np.full(n, np.nan),
            "force_target", float(cfg["study"]["terminal_soc_kwh"]), 0.0, cfg)
        out.append({
            "mode": mode, "information": "perfect_information_lower_bound", "days": len(data.dates) - i0,
            "initial_soc_kwh": float(cfg["study"]["initial_soc_kwh"]), "terminal_soc_kwh": float(sol.soc_end[-1]),
            "purchase_cost_yuan": float(np.dot(price, sol.grid)), "grid_kwh": float(sol.grid.sum()),
            "solver_seconds": sol.elapsed_seconds, "max_balance_residual_kwh": sol.max_balance_residual,
            "max_soc_residual_kwh": sol.max_soc_residual
        })
    return out
```

```
def simulate_day_ahead(data: DataBundle, cfg: dict[str, Any], mode: str, method: str = "highs", tiebreak: float|None=None) -> dict[str, Any]:
    if mode not in {"q2", "q4_2"}: raise ValueError(mode)
    start = date.fromisoformat(cfg["study"]["official_start"]); end = date.fromisoformat(cfg["study"]["official_end"])
    dt = float(cfg["storage"]["interval_minutes"])/60; eta = float(cfg["storage"]["charge_efficiency"])
    soc = float(cfg["study"]["initial_soc_kwh"]); target = float(cfg["study"]["terminal_soc_kwh"]); count = int(cfg["forecast"]["same_weekday_profiles"])
    rows = []; daily = []; stats = []
```

```

for day in [d for d in data.dates if start<=d<=end]:
    di=data.date_index[day]; gi=di*144; soc0=soc
    load_hload=profile_horizon(data,gi,144,day,"load",count); pv_hpv=profile_horizon(data,gi,144,day,"pv",count)
    if mode=="q4_2": price,hprice=profile_horizon(data,gi,144,day,"price",count); actual_price=data.realtime_price[di]
    else: price=data.fixed_price; hprice=[]; actual_price=data.fixed_price
    terminal="force_target" if day==end else str(cfg["rolling"])+selected_terminal_rule")
    penalty=float(np.median(price))*float(cfg["rolling"])+selected_soft_penalty_multiplier")
    sol=solve_deterministic(load*dt,pv*dt,price,soc,np.full(144,np.nan),terminal,target,penalty,cfg,method,tiebreak)
    starts=np.r_[soc,sol.soc_end[-1]]; soc=float(sol.soc_end[-1])
    actual_load=data.load_kw[di]*dt; actual_pv=data.pv_kw[di]*dt
    net=sol.grid+actual_pv+sol.discharge-actual_load-sol.charge
    emergency=np.maximum(-net,0); unused=np.maximum(net,0)
    plan_cost=actual_price*sol.grid; emergency_cost=float(cfg["cost"]+emergency_multiplier))*actual_price*emergency; total=plan_cost+emergency_cost
    hist_dates=sorted(set(hload+hpv+hprice)); hist="".join(x.isoformat() for x in hist_dates)
    for j in range(144):
        curtail_pv=min(float(sol.curtail[j]),float(pv[j]*dt)); planned_unused_nonpv=max(float(sol.curtail[j])-float(pv[j]*dt),0.0)
        rows.append({
            "date":day.isoformat(),"position":j+1,"source_timestamp":data.source_timestamps[j],
            "physical_interval":f"{j*10//60:02d}:{j*10%60:02d}-{(j+1)*10//60:02d}:{(j+1)*10%60:02d}",official_label":data.official_labels[j],
            "plan_00_grid_kwh":float(sol.grid[j]),"final_adjusted_grid_kwh":float(sol.grid[j]),"adjustment_kwh":0.0,
            "charge_kwh":float(sol.charge[j]),"discharge_kwh":float(sol.discharge[j]),"soc_start_kwh":float(starts[j]),"soc_end_kwh":float(sol.soc_end[j]),
            "curtail_pv_kwh":curtail_pv,"planned_unused_nonpv_kwh":planned_unused_nonpv,"forecast_load_kwh":float(load[j]*dt),"forecast_pv_kwh":float(pv[j]*dt),
            "actual_load_kwh":float(actual_load[j]),"actual_pv_kwh":float(actual_pv[j]),"forecast_price":float(price[j]),"settlement_price":float(actual_price[j]),
            "emergency_kwh":float(emergency[j]),"unused_surplus_kwh":float(unused[j]),"plan_cost_yuan":float(plan_cost[j]),
            "adjustment_cost_yuan":0.0,"emergency_cost_yuan":float(emergency_cost[j]),"final_relative_cost_yuan":float(total[j]),
            "sequential_cost_yuan":float(total[j]),"decision_hour":0,"history_cutoff_date":(day-timedelta(days=1)).isoformat(),
            "model_arm":causal_day_ahead,"scenario":mode.upper(),
        })
    daily.append({"date":day.isoformat(),"mode":mode,"start_soc_kwh":soc0,"end_soc_kwh":soc,
        "plan_cost_yuan":float(plan_cost.sum()),"emergency_cost_yuan":float(emergency_cost.sum()),
        "final_relative_cost_yuan":float(total.sum()),"sequential_cost_yuan":float(total.sum()),
        "emergency_kwh":float(emergency.sum()),"unused_surplus_kwh":float(unused.sum())})
    stats.append(sol)
return [{"dispatch":rows,"daily":daily,"summary":{
    "mode":mode,"scenario":mode.upper(),"model_arm":causal_day_ahead,"days":len(daily),
    "initial_soc_kwh":daily[0]["start_soc_kwh"],"terminal_soc_kwh":daily[-1]["end_soc_kwh"],
    "plan_cost_yuan":sum(x["plan_cost_yuan"] for x in daily),"adjustment_cost_yuan":0.0,
    "emergency_cost_yuan":sum(x["emergency_cost_yuan"] for x in daily),"final_relative_cost_yuan":sum(x["final_relative_cost_yuan"] for x in daily),
    "sequential_cost_yuan":sum(x["sequential_cost_yuan"] for x in daily),"emergency_kwh":sum(x["emergency_kwh"] for x in daily),
    "unused_surplus_kwh":sum(x["unused_surplus_kwh"] for x in daily),"solver_seconds":sum(x.elapsed_seconds for x in stats),"solver_runs":len(stats),
    "max_balance_residual_kwh":max(x.max_balance_residual for x in stats),"max_soc_residual_kwh":max(x.max_soc_residual for x in stats),
    "constraint_violation_count":int(sum(x.max_balance_residual>float(cfg["audit"])+feasibility_tolerance)) or x.max_soc_residual>float(cfg["audit"]
+feasibility_tolerance)) for x in stats)}}]

```

```

def select_terminal_rule(data:DataBundle,cfg:dict[str,Any]) -> list[dict[str,Any]]:
    original_rule=cfg["rolling"]+selected_terminal_rule"; original_mult=cfg["rolling"]+selected_soft_penalty_multiplier"
    rows=[]; start=date(2025,1,8); end=date(2025,1,31)
    try:
        for rule in cfg["rolling"]+terminal_candidates":
            multipliers=cfg["rolling"]+soft_penalty_multipliers" if rule=="soft_target" else [None]
            for mult in multipliers:
                cfg["rolling"]+selected_terminal_rule"]=rule
                if mult is not None: cfg["rolling"]+selected_soft_penalty_multiplier"]=mult
                arm="raw_baseline" if rule=="daily_closed" else "crossday_only"
                cost=0.0; emergency=0.0
                for mode in ("q3","q4_3"):
                    r=simulate(data,cfg,RunSpec(mode,"S3",arm,start_date=start,end_date=end))
                    cost+=r["summary"]+final_relative_cost_yuan"; emergency+=r["summary"]+emergency_kwh"
                rows.append({"terminal_rule":rule,"soft_penalty_multiplier":"" if mult is None else mult,
                    "validation_start":start.isoformat(),"validation_end":end.isoformat(),"combined_cost_yuan":cost,"combined_emergency_kwh":emergency})
    finally:
        cfg["rolling"]+selected_terminal_rule"]=original_rule; cfg["rolling"]+selected_soft_penalty_multiplier"]=original_mult
    return rows

```

```

def moving_block_ci(differences:np.ndarray,block:int,reprs:int,seed:int) -> tuple[float,float]:
    x=np.asarray(differences,float); n=len(x); rng=np.random.default_rng(seed)
    starts=np.arange(max(1,n-block+1)); vals=np.empty(reprs)
    for r in range(reprs):
        sample=[]
        while len(sample)<n:
            s=int(rng.choice(starts)); sample.extend(x[s:min(s+block,n)])
        vals[r]=np.mean(sample[:n])
    return float(np.quantile(vals,.025)),float(np.quantile(vals,.975))

```

```

def run_and_write_all(data:DataBundle,cfg:dict[str,Any],include_stability:bool=True) -> dict[str,Any]:
    root=Path(cfg["output_root"]); frozen=root/"frozen"; evidence=root/"evidence"; frozen.mkdir(parents=True,exist_ok=True);
    evidence.mkdir(parents=True,exist_ok=True)
    _write_csv(frozen/"time_index.csv",data.time_rows())
    q1=solve_q1(data,cfg); _write_csv(frozen/"q1_dispatch.csv",q1)
    validation=select_terminal_rule(data,cfg); _write_csv(evidence/"terminal_validation.csv",validation)
    selected=min(validation,key=lambda x:x["combined_cost_yuan"])
    cfg["rolling"]+selected_terminal_rule"]=selected["terminal_rule"]
    if selected["soft_penalty_multiplier"]!="": cfg["rolling"]+selected_soft_penalty_multiplier"]=float(selected["soft_penalty_multiplier"])

```

```

resolved={k:v for k,v in cfg.items() if not k.startswith("_")}
(frozen/"resolved_config.json").write_text(json.dumps(resolved,ensure_ascii=False,indent=2,encoding="utf-8"))
perfect=perfect_information_controls(data,cfg); _write_csv(frozen/"perfect_information_summary.csv",perfect)
q2=simulate_day_ahead(data,cfg,q2"); q42=simulate_day_ahead(data,cfg,q4_2")
_write_csv(frozen/"q2_dispatch.csv",q2["dispatch"]); _write_csv(frozen/"q4_2_dispatch.csv",q42["dispatch"])
exp=[]; daily_by=dict[tuple[str,str].list[dict[str,Any]]]={}; exp_results=dict[tuple[str,str].dict[str,Any]]={}
for mode in ("q3","q4_3"):
    for arm in cfg["experiments"]["arms"]:
        detail=arm==cfg["experiments"]["official_arm"]
        result=simulate(data,cfg,RunSpec(mode,"S3",arm,collect_detail=detail)); exp.append(result["summary"]); daily_by[(mode,arm)]=result["daily"];
exp_results[(mode,arm)]=result
print(json.dumps({"stage":"ablation","mode":mode,"arm":arm,"cost":result["summary"]["final_relative_cost_yuan"]},ensure_ascii=False),flush=True)
official={}
for mode in ("q3","q4_3"):
    for scenario in cfg["experiments"]["schedules"]:
        if scenario==cfg["experiments"]["official_scenario"]:
            result=exp_results[(mode,cfg["experiments"]["official_arm"])]
        else:
            result=simulate(data,cfg,RunSpec(mode,scenario,cfg["experiments"]["official_arm"],collect_detail=False))
            official[(mode,scenario)]=result
            print(json.dumps({"stage":"schedule","mode":mode,"scenario":scenario,"cost":result["summary"]["final_relative_cost_yuan"]},ensure_ascii=False),flush=True)
            key=(mode,cfg["experiments"]["official_scenario"]); path="q3_dispatch.csv" if mode=="q3" else "q4_3_dispatch.csv"
            _write_csv(frozen/path,official[key]["dispatch"])
q3_versions=official[("q3",cfg["experiments"]["official_scenario"])]["decision_versions"]
q43_versions=official[("q4_3",cfg["experiments"]["official_scenario"])]["decision_versions"]
_write_csv(frozen/"q3_decision_versions.csv",q3_versions); _write_csv(frozen/"q4_3_decision_versions.csv",q43_versions)
_write_csv(frozen/"decision_versions.csv",q3_versions+q43_versions)
experiment_rows=exp+[official[k]["summary"] for k in official if k[1]!=cfg["experiments"]["official_scenario"]]
_write_csv(frozen/"experiment_summary.csv",experiment_rows)
monthly=[]
for mode in ("q3","q4_3"):
    base=[x["date"] for x in daily_by[(mode,"raw_baseline")]]; combo=official[(mode,"S3")]["daily"]
    for month in range(2,13):
        br=[base[x["date"]] for x in combo if int(x["date"][5:7])==month]; cr=[x for x in combo if int(x["date"][5:7])==month]
        dif=np.asarray([c["final_relative_cost_yuan"]-b["final_relative_cost_yuan"] for b,c in zip(br,cr)])
        lo,hi=moving_block_ci(dif,int(cfg["experiments"]["moving_block_days"]),int(cfg["experiments"]["bootstrap_replicates"]),int(cfg["experiments"]["bootstrap_seed"])+month)
        monthly.append({"mode":mode,"month":month,"days":len(dif),"baseline_cost_yuan":sum(x["final_relative_cost_yuan"] for x in br),
            "combination_cost_yuan":sum(x["final_relative_cost_yuan"] for x in cr),"difference_yuan":float(dif.sum()),"block_bootstrap_95_low_yuan":lo,"block_bootstrap_95_high_yuan":hi})
        _write_csv(frozen/"monthly_effects.csv",monthly)
    comparison=[]; monthly_schedules=[]
    for mode in ("q3","q4_3"):
        for scenario in cfg["experiments"]["schedules"]:
            schedule=official[(mode,scenario)]["daily"]
            for month in range(2,13):
                selected=[r for r in schedule if int(r["date"][5:7])==month]
                monthly_schedules.append({"mode":mode,"scenario":scenario,"month":month,"days":len(selected),
                    "final_relative_cost_yuan":sum(r["final_relative_cost_yuan"] for r in selected),
                    "sequential_cost_yuan":sum(r["sequential_cost_yuan"] for r in selected),
                    "emergency_kwh":sum(r["emergency_kwh"] for r in selected),
                    "unused_surplus_kwh":sum(r["unused_surplus_kwh"] for r in selected),
                    "month_end_soc_kwh":selected[-1]["end_soc_kwh"]})
            s2=official[(mode,"S2")]["daily"]; s3=official[(mode,"S3")]["daily"]
            dif=np.asarray([b["final_relative_cost_yuan"]-a["final_relative_cost_yuan"] for a,b in zip(s2,s3)])
            lo,hi=moving_block_ci(dif,int(cfg["experiments"]["moving_block_days"]),int(cfg["experiments"]["bootstrap_replicates"]),int(cfg["experiments"]["bootstrap_seed"])+100)
            cents=0.005
            comparison.append({"mode":mode,"comparison":f"S3_minus_S2_final_relative","days":len(dif),"point_difference_yuan":float(dif.sum()),
                "block_ci_low_yuan":lo,"block_ci_high_yuan":hi,"positive_days":int(np.sum(dif>cents)),
                "negative_days":int(np.sum(dif<-cents)),zero_days":int(np.sum(np.abs(dif)<=cents))})
            ds=np.asarray([b["sequential_cost_yuan"]-a["sequential_cost_yuan"] for a,b in zip(s2,s3)])
            lo,hi=moving_block_ci(ds,int(cfg["experiments"]["moving_block_days"]),int(cfg["experiments"]["bootstrap_replicates"]),int(cfg["experiments"]["bootstrap_seed"])+101)
            comparison.append({"mode":mode,"comparison":f"S3_minus_S2_sequential","days":len(ds),"point_difference_yuan":float(ds.sum()),
                "block_ci_low_yuan":lo,"block_ci_high_yuan":hi,"positive_days":int(np.sum(ds>cents)),
                "negative_days":int(np.sum(ds<-cents)),zero_days":int(np.sum(np.abs(ds)<=cents))})
            _write_csv(frozen/"comparison_stats.csv",comparison); _write_csv(frozen/"monthly_schedules.csv",monthly_schedules)
    stability=[]
    if include_stability:
        for mode in ("q3","q4_3"):
            for scenario in ("S2","S3"):
                for method in cfg["experiments"]["solver_methods"]:
                    for tb in cfg["experiments"]["tiebreak_coefficients"]:
                        if method=="highs" and float(tb)==float(cfg["cost"])/["throughput_tiebreak_yuan_per_kwh"]):
                            r=official[(mode,scenario)]
                        else:
                            r=simulate(data,cfg,RunSpec(mode,scenario,cfg["experiments"]["official_arm"],method,float(tb),collect_detail=False))
                        stability.append({"r":r["summary"],"solver_method":method,"tiebreak":tb})
                        print(json.dumps({"stage":"stability","mode":mode,"scenario":scenario,"method":method,"tiebreak":tb,"cost":r["summary"]
                            ["final_relative_cost_yuan"]},ensure_ascii=False),flush=True)
                        _write_csv(frozen/"stability_summary.csv",stability)
            assessment=[]
            for mode in ("q3","q4_3"):
                by=[(r["solver_method"],float(r["tiebreak"]),r["scenario"]) for r in stability if r["mode"]==mode]
                for metric,settlement in (("final_relative_cost_yuan","final_relative_to_00_plan"),
                    ("sequential_cost_yuan","sequential_sensitivity")):
                    dif=[]
                    for method in cfg["experiments"]["solver_methods"]:
                        for tb in cfg["experiments"]["tiebreak_coefficients"]:
                            dif.append(by[(method,float(tb),"S3"])[metric]-by[(method,float(tb),"S2"])[metric])
                    signs=[0 if abs(x)<.005 else (1 if x>0 else -1) for x in dif]; cents={round(x,2) for x in dif}
                    stable=len(signs)==1 and 0 not in signs and len(cents)==1
                    assessment.append({"mode":mode,"settlement":settlement,"comparison":f"S3_minus_S2","replicates":len(dif),

```

```

        "difference_min_yuan":min(dif),"difference_max_yuan":max(dif),
        "rounded_cent_values":["".join(str(x) for x in sorted(cents)),direction_values:""].join(str(x) for x in sorted(signs)),
        "conclusion":"stable" if stable else "numerically_indistinguishable"))
    _write_csv(frozen/"stability_assessment.csv",assessment)
    summaries=[q2["summary"],q42["summary"]]+experiment_rows
    summary={"generated_at": _import_ ("datetime").datetime.now().astimezone().isoformat(),"config":"config/final.yaml",
        "environment":{"python":sys.version,"platform":platform.platform(),"numpy":np.__version__,"scipy":scipy.__version__},
        "causality": "Every historical reference is strictly before the decision date; actual values enter only realized settlement.",
        "terminal_validation":validation,"selected_terminal":selected,"perfect_information_controls":perfect,"results":summaries,"stability":stability}
    (frozen/"model_run_summary.json").write_text(json.dumps(summary,ensure_ascii=False,indent=2,encoding="utf-8"))
    files=sorted(p for p in frozen.iterdir() if p.is_file() and p.name!="frozen_manifest.json")
    manifest={"schema_version":cfg["schema_version"],"generated_at":summary["generated_at"],"source_root":cfg["source_root"],
        "command":"./run_pipeline.sh --config config/final.yaml","files":[{"path":p.name,bytes:p.stat().st_size,sha256:_sha256(p) for p in files]}
    (frozen/"frozen_manifest.json").write_text(json.dumps(manifest,ensure_ascii=False,indent=2,encoding="utf-8"))
    return summary

def run_and_write_official(data:DataBundle,cfg:dict[str,Any]) -> dict[str,Any]:
    """Build only the five frozen dispatches and their shared decision ledger."""
    root=Path(cfg["_output_root"]); frozen=root/"frozen"; evidence=root/"evidence"; frozen.mkdir(parents=True,exist_ok=True);
    evidence.mkdir(parents=True,exist_ok=True)
    _write_csv(frozen/"time_index.csv",data.time_rows())
    _write_csv(frozen/"q1_dispatch.csv",solve_q1(data,cfg))
    validation=select_terminal_rule(data,cfg); _write_csv(evidence/"terminal_validation.csv",validation)
    selected=min(validation,key=lambda x:x["combined_cost_yuan"])
    cfg["rolling"]["selected_terminal_rule"]=selected["terminal_rule"]
    if selected["soft_penalty_multiplier"]!="": cfg["rolling"]["selected_soft_penalty_multiplier"]=float(selected["soft_penalty_multiplier"])
    resolved={k:v for k,v in cfg.items() if not k.startswith("_")}
    (frozen/"resolved_config.json").write_text(json.dumps(resolved,ensure_ascii=False,indent=2,encoding="utf-8"))
    perfect=perfect_information_controls(data,cfg); _write_csv(frozen/"perfect_information_summary.csv",perfect)
    q2=simulate_day_ahead(data,cfg,q2); q42=simulate_day_ahead(data,cfg,q4_2)
    q3=simulate(data,cfg,RunSpec("q3",cfg["experiments"])[0]["official_scenario"],cfg["experiments"])[0]["official_arm"])
    q43=simulate(data,cfg,RunSpec("q4_3",cfg["experiments"])[0]["official_scenario"],cfg["experiments"])[0]["official_arm"])
    _write_csv(frozen/"q2_dispatch.csv",q2["dispatch"]); _write_csv(frozen/"q4_2_dispatch.csv",q42["dispatch"])
    _write_csv(frozen/"q3_dispatch.csv",q3["dispatch"]); _write_csv(frozen/"q4_3_dispatch.csv",q43["dispatch"])
    _write_csv(frozen/"q3_decision_versions.csv",q3["decision_versions"]); _write_csv(frozen/"q4_3_decision_versions.csv",q43["decision_versions"])
    _write_csv(frozen/"decision_versions.csv",q3["decision_versions"]+q43["decision_versions"])
    summaries=[q2["summary"],q3["summary"],q42["summary"],q43["summary"]]
    _write_csv(frozen/"official_summary.csv",summaries)
    summary={"generated_at": _import_ ("datetime").datetime.now().astimezone().isoformat(),"config":"config/final.yaml",
        "environment":{"python":sys.version,"platform":platform.platform(),"numpy":np.__version__,"scipy":scipy.__version__},
        "causality": "Every historical reference is strictly before the decision date; actual values enter only realized settlement.",
        "terminal_validation":validation,"selected_terminal":selected,"perfect_information_controls":perfect,"results":summaries}
    (frozen/"model_run_summary.json").write_text(json.dumps(summary,ensure_ascii=False,indent=2,encoding="utf-8"))
    schemas={
        "time_index":["date","position","global_position","source_timestamp","physical_interval","official_label"],
        "decision_version":
        ["date","target_date","mode","scenario","model_arm","decision_time","forecast_issue_time","effective_start","effective_end","history_cutoff_date","position","physical_interval","o
        fficial_label","old_commitment_kwh","new_commitment_kwh","sensitivity_old_commitment_kwh","sensitivity_new_commitment_kwh","preview_grid_kwh","decision_basis
        ","soc_at_decision_kwh","load_history_dates","price_history_dates","pv_bias_history_dates","residual_scenario_dates","is_preview_next_day","is_preview_recourse"],
        "dispatch_detail":list(q3["dispatch"])[0]),
    }
    (frozen/"schemas.json").write_text(json.dumps(schemas,ensure_ascii=False,indent=2,encoding="utf-8"))
    files=sorted(p for p in frozen.iterdir() if p.is_file() and p.name!="frozen_manifest.json")
    manifest={"schema_version":cfg["schema_version"],"generated_at":summary["generated_at"],"source_root":cfg["source_root"],
        "command":"./run_pipeline.sh --config config/final.yaml","files":[{"path":p.name,bytes:p.stat().st_size,sha256:_sha256(p) for p in files]}
    (frozen/"frozen_manifest.json").write_text(json.dumps(manifest,ensure_ascii=False,indent=2,encoding="utf-8"))
    return summary

def finalize_existing_official(data:DataBundle,cfg:dict[str,Any]) -> dict[str,Any]:
    """Refresh portable metadata and perfect-information controls without rerunning rolling LPs"""
    frozen=Path(cfg["_output_root"])/"frozen"
    summary_path=frozen/"model_run_summary.json"
    if not summary_path.is_file(): raise FileNotFoundError(summary_path)
    summary=json.loads(summary_path.read_text(encoding="utf-8"))
    summary["config"]="config/final.yaml"
    perfect=perfect_information_controls(data,cfg)
    _write_csv(frozen/"perfect_information_summary.csv",perfect)
    summary["perfect_information_controls"]=perfect
    tol=float(cfg["audit"])[0]["feasibility_tolerance"])
    for row in summary.get("results",[]):
        row["constraint_violation_count"]=int(row.get("max_balance_residual_kwh",0)>tol or row.get("max_soc_residual_kwh",0)>tol)
    resolved={k:v for k,v in cfg.items() if not k.startswith("_")}
    (frozen/"resolved_config.json").write_text(json.dumps(resolved,ensure_ascii=False,indent=2,encoding="utf-8"))
    summary_path.write_text(json.dumps(summary,ensure_ascii=False,indent=2,encoding="utf-8"))
    _write_csv(frozen/"official_summary.csv",summary["results"])
    files=sorted(p for p in frozen.iterdir() if p.is_file() and p.name!="frozen_manifest.json")
    manifest={"schema_version":cfg["schema_version"],"generated_at":summary["generated_at"],"source_root":cfg["source_root"],
        "command":"./run_pipeline.sh --config config/final.yaml","files":[{"path":p.name,bytes:p.stat().st_size,sha256:_sha256(p) for p in files]}
    (frozen/"frozen_manifest.json").write_text(json.dumps(manifest,ensure_ascii=False,indent=2,encoding="utf-8"))
    return summary

def migrate_existing_unused_split(cfg:dict[str,Any]) -> None:
    """Losslessly split legacy total planned unused into PV and non-PV fields."""

```

```

frozen=Path(cfg["_output_root"])/"frozen"
for name in ("q1","q2","q3","q4_2","q4_3"):
    path=frozen/f"{name}_dispatch.csv"; tmp=frozen/f"{name}_dispatch.migrate.csv"
    with path.open(encoding="utf-8-sig",newline="") as src:
        reader=csv.DictReader(src); fields=list(reader.fieldnames or [])
        if "planned_unused_nonpv_kwh" in fields: continue
        insert=fields.index("curtail_pv_kwh")+1; fields.insert(insert,"planned_unused_nonpv_kwh")
        with tmp.open("w",encoding="utf-8-sig",newline="") as dst:
            writer=csv.DictWriter(dst,fieldnames=fields); writer.writeheader()
            for row in reader:
                total=float(row["curtail_pv_kwh"]); pv=float(row["forecast_pv_kwh"])
                row["curtail_pv_kwh"]=repr(min(total,pv)); row["planned_unused_nonpv_kwh"]=repr(max(total-pv,0.0)); writer.writerow(row)
    tmp.replace(path)
    schemas_path=frozen/"schemas.json"; schemas=json.loads(schemas_path.read_text(encoding="utf-8"))
    with (frozen/"q3_dispatch.csv").open(encoding="utf-8-sig",newline="") as f: schemas["dispatch_detail"]=next(csv.reader(f))
    schemas_path.write_text(json.dumps(schemas,ensure_ascii=False,indent=2),encoding="utf-8")
    summary=json.loads((frozen/"model_run_summary.json").read_text(encoding="utf-8"))
    files=sorted(p for p in frozen.iterdir() if p.is_file() and p.name!="frozen_manifest.json")
    manifest={"schema_version":cfg["schema_version"],"generated_at":summary["generated_at"],"source_root":cfg["source_root"],
        "command":"./run_pipeline.sh --config config/final.yaml","files":[{"path":p.name,"bytes":p.stat().st_size,"sha256":_sha256(p)} for p in files]}
    (frozen/"frozen_manifest.json").write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding="utf-8")

```

```

def backfill_full_next_day_previews(data:DataBundle,cfg:dict[str,Any]) -> None:
    """Rebuild full non-binding next-day previews using deterministic point forecasts."""
    frozen=Path(cfg["_output_root"])/"frozen"; combined=[]
    for mode in ("q3","q4_3"):
        dispatch={}
        with (frozen/f"{mode}_dispatch.csv").open(encoding="utf-8-sig",newline="") as f:
            for r in csv.DictReader(f): dispatch[{r["date"],int(r["position"])}]=r
        path=frozen/f"{mode}_decision_versions.csv"
        with path.open(encoding="utf-8-sig",newline="") as f:
            rows=list(csv.DictReader(f)); old_fields=list(rows[0])
            rows=[r for r in rows if r.get("is_preview_next_day")!="1"]
            for r in rows:
                r.setdefault("target_date",r["date"]); r.setdefault("preview_grid_kwh","")
            dates=sorted({d for d_ in dispatch})
            for ds in dates:
                day=date.fromisoformat(ds); di=data.date_index[day]
                if di==len(data.dates)-1: continue
                baseline=np.asarray([float(dispatch[{ds,j}]["plan_00_grid_kwh"]) for j in range(1,145)])
                for issue in (6,12,18):
                    pos=issue*6; length=min(144,len(data.dates)*144-(di*144+pos)); cur_len=144-pos
                    b=np.r_[baseline[pos:],np.full(length-cur_len,np.nan)]
                    soc=float(dispatch[{ds,pos+1}]["soc_start_kwh"])
                    terminal="force_target" if di*144+pos+length==len(data.dates)*144 else str(cfg["rolling"])[f"selected_terminal_rule"]
                    sol,_ = solve(data.cfg,day,issue,length,soc,b,True,False,terminal,36,mode,"highs",None)
                    for k in range(cur_len.length):
                        target=data.dates[(di*144+pos+k)//144]; tpos=(di*144+pos+k)%144
                        rows.append({
                            "date":ds,"target_date":target.isoformat(),"mode":mode,"scenario":"S3","model_arm":"all_three",
                            "decision_time":f"{ds} {issue:02d}:00","forecast_issue_time":f"{ds} {issue:02d}:00",
                            "effective_start":f"{ds} {issue:02d}:00","effective_end":f"{(day+timedelta(days=1)).isoformat()} {issue:02d}:00",
                            "history_cutoff_date":(day-timedelta(days=1)).isoformat(),"position":tpos+1,
                            "physical_interval":f"{tpos*10//60:02d}-{tpos*10%60:02d}-{(tpos+1)*10//60:02d}-{(tpos+1)*10%60:02d}",
                            "official_label":data.official_labels[tpos],"old_commitment_kwh":"","new_commitment_kwh":",",
                            "sensitivity_old_commitment_kwh":"","sensitivity_new_commitment_kwh":"","preview_grid_kwh":float(sol.grid[k]),
                            "decision_basis":"next_day_preview","soc_at_decision_kwh":soc,"load_history_dates":"","price_history_dates":"","",
                            "pv_bias_history_dates":"","residual_scenario_dates":"","is_preview_next_day":1,"is_preview_recourse":1,
                        })
    desired=["date","target_date","mode","scenario","model_arm","decision_time","forecast_issue_time","effective_start","effective_end","history_cutoff_date","position","physical_int
erval","official_label","old_commitment_kwh","new_commitment_kwh","sensitivity_old_commitment_kwh","sensitivity_new_commitment_kwh","preview_grid_kwh","decisi
on_basis","soc_at_decision_kwh","load_history_dates","price_history_dates","pv_bias_history_dates","residual_scenario_dates","is_preview_next_day","is_preview_recourse"]
    rows.sort(key=lambda r:(r["date"],r["decision_time"],r["target_date"],int(r["position"])))
    with path.open("w",encoding="utf-8-sig",newline="") as f:
        w=csv.DictWriter(f,fieldnames=desired); w.writeheader(); w.writerows(rows)
    combined.extend(rows)
    _write_csv(frozen/"decision_versions.csv",combined)
    schemas_path=frozen/"schemas.json"; schemas=json.loads(schemas_path.read_text(encoding="utf-8")); schemas["decision_version"]=list(combined[0])
    schemas_path.write_text(json.dumps(schemas,ensure_ascii=False,indent=2),encoding="utf-8")
    summary=json.loads((frozen/"model_run_summary.json").read_text(encoding="utf-8"))
    files=sorted(p for p in frozen.iterdir() if p.is_file() and p.name!="frozen_manifest.json")
    manifest={"schema_version":cfg["schema_version"],"generated_at":summary["generated_at"],"source_root":cfg["source_root"],
        "command":"./run_pipeline.sh --config config/final.yaml","files":[{"path":p.name,"bytes":p.stat().st_size,"sha256":_sha256(p)} for p in files]}
    (frozen/"frozen_manifest.json").write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding="utf-8")

```

```

_WORKER_DATA:DataBundle|None=None
_WORKER_CFG:dict[str,Any]|None=None

```

```

def _init_stability_worker(data:DataBundle,cfg:dict[str,Any])>None:
    global _WORKER_DATA,_WORKER_CFG
    _WORKER_DATA=data; _WORKER_CFG=cfg

```

```

def _stability_worker(task:tuple[str,str,float])>->dict[str,Any]:
    if _WORKER_DATA is None or _WORKER_CFG is None: raise RuntimeError("stability worker not initialized")
    mode,scenario,method,tb=task
    r=simulate(_WORKER_DATA,_WORKER_CFG,RunSpec(mode,scenario,_WORKER_CFG["experiments"])[_official_arm"],method,tb,collect_detail=False)
    return {"r":["summary"],"solver_method":method,"tiebreak":tb}

def run_stability_parallel(data:DataBundle,cfg:dict[str,Any])>->list[dict[str,Any]]:
    """Run all 36 full-year solver/tiebreak checks in isolated worker processes."""
    tasks=[(mode,scenario,method,float(tb)) for mode in ("q3","q4_3") for scenario in ("S2","S3")
            for method in cfg["experiments"][_solver_methods] for tb in cfg["experiments"][_tiebreak_coefficients]]
    results=[]; workers=int(cfg["experiments"].get("stability_workers",4))
    with ProcessPoolExecutor(max_workers=workers,initializer=_init_stability_worker,initargs=(data,cfg)) as pool:
        future_map=[pool.submit(_stability_worker,t) for t in tasks]
    try:
        for fut in as_completed(future_map):
            row=fut.result(); results.append(row)
            print(json.dumps({"stage":"stability_parallel","completed":len(results),"total":len(tasks),"mode":row["mode"],
                             "scenario":row["scenario"],"method":row["solver_method"],"tiebreak":row["tiebreak"],
                             "cost":row["final_relative_cost_yuan"]},ensure_ascii=False),flush=True)
    except BaseException:
        for fut in future_map: fut.cancel()
        raise
    order=["q3":0,"q4_3":1]; sc={"S2":0,"S3":1}; meth={m:i for i,m in enumerate(cfg["experiments"][_solver_methods])}
    results.sort(key=lambda r:(order[r["mode"]],sc[r["scenario"]],meth[r["solver_method"]],float(r["tiebreak"])))
    frozen=Path(cfg["_output_root"])/"frozen"; _write_csv(frozen/"stability_summary.csv",results)
    assessment=[]
    for mode in ("q3","q4_3"):
        by={r["solver_method"],float(r["tiebreak"]),r["scenario"]}:r for r in results if r["mode"]==mode}
        for metric,settlement in (("final_relative_cost_yuan","final_relative_to_00_plan"),
                                   ("sequential_cost_yuan","sequential_sensitivity")):
            dif=[]
            for method in cfg["experiments"][_solver_methods]:
                for tb in cfg["experiments"][_tiebreak_coefficients]:
                    dif.append(by[(method,float(tb),"S3"])[metric]-by[(method,float(tb),"S2"])[metric])
            signs={0 if abs(x)<.005 else (1 if x>0 else -1) for x in dif}; cents={round(x,2) for x in dif}; stable=len(signs)==1 and 0 not in signs and len(cents)==1
            assessment.append({"mode":mode,"settlement":settlement,"comparison":"S3_minus_S2","replicates":len(dif),
                              "difference_min_yuan":min(dif),"difference_max_yuan":max(dif),
                              "rounded_cent_values":",".join(str(x) for x in sorted(cents)),
                              "direction_values":",".join(str(x) for x in sorted(signs)),
                              "conclusion":"stable" if stable else "numerically_indistinguishable"})
    _write_csv(frozen/"stability_assessment.csv",assessment)
    summary_path=frozen/"model_run_summary.json"; summary=json.loads(summary_path.read_text(encoding="utf-8"))
    summary["stability"]=results; summary["stability_assessment"]=assessment
    summary_path.write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding="utf-8")
    files=sorted(p for p in frozen.iterdir() if p.is_file() and p.name!="frozen_manifest.json")
    manifest={"schema_version":cfg["schema_version"],"generated_at":summary["generated_at"],"source_root":cfg["source_root"],
              "command":"./run_pipeline.sh --config config/final.yaml","files":[{"path":p.name,"bytes":p.stat().st_size,"sha256":sha256(p)} for p in files]}
    (frozen/"frozen_manifest.json").write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding="utf-8")
    return results

def _pre_worker(task:tuple[str,str,str])>->dict[str,Any]:
    if _WORKER_DATA is None or _WORKER_CFG is None: raise RuntimeError("pre worker not initialized")
    kind,mode,label=task
    spec=RunSpec(mode,"S3",label,collect_detail=False) if kind=="arm" else RunSpec(mode,label,_WORKER_CFG["experiments"][_official_arm"],collect_detail=False)
    r=simulate(_WORKER_DATA,_WORKER_CFG,spec)
    return {"kind":kind,"mode":mode,"label":label,"summary":r["summary"],"daily":r["daily"]}

def _official_s3_from_frozen(cfg:dict[str,Any],mode:str)>->dict[str,Any]:
    frozen=Path(cfg["_output_root"])/"frozen"; grouped:dict[str,list[dict[str,str]]]={}
    with (frozen/f"{mode}_dispatch.csv").open(encoding="utf-8-sig",newline="") as f:
        for row in csv.DictReader(f): grouped.setdefault(row["date"],[]).append(row)
    daily=[]
    for day,rows in sorted(grouped.items()):
        daily.append({"date":day,"mode":mode,"scenario":"S3","model_arm":"all_three",
                      "start_soc_kwh":float(rows[0][_soc_start_kwh]),"end_soc_kwh":float(rows[-1][_soc_end_kwh]),
                      "plan_grid_kwh":sum(float(r[_plan_00_grid_kwh]) for r in rows),"final_grid_kwh":sum(float(r[_final_adjusted_grid_kwh]) for r in rows),
                      "emergency_kwh":sum(float(r[_emergency_kwh]) for r in rows),"unused_surplus_kwh":sum(float(r[_unused_surplus_kwh]) for r in rows),
                      "plan_cost_yuan":sum(float(r[_plan_cost_yuan]) for r in rows),"adjustment_cost_yuan":sum(float(r[_adjustment_cost_yuan]) for r in rows),
                      "emergency_cost_yuan":sum(float(r[_emergency_cost_yuan]) for r in rows),"final_relative_cost_yuan":sum(float(r[_final_relative_cost_yuan]) for r in rows),
                      "sequential_cost_yuan":sum(float(r[_sequential_cost_yuan]) for r in rows)})
    model=json.loads((frozen/"model_run_summary.json").read_text(encoding="utf-8"))
    summary=next(r for r in model["results"] if r["mode"]==mode)
    return {"summary":summary,"daily":daily}

def run_pre_stability_parallel(data:DataBundle,cfg:dict[str,Any])>->dict[str,Any]:

```

```

"""Parallelize all non-official ablations and S0-S2; workers never write files"""
official_arm=cfg["experiments"]["official_arm"]
tasks=[("arm",mode,arm) for mode in ("q3","q4_3") for arm in cfg["experiments"]["arms"] if arm!=official_arm]
tasks += [("schedule",mode,s) for mode in ("q3","q4_3") for s in ("S0","S1","S2")]
got=[]; workers=int(cfg["experiments"].get("stability_workers",4))
with ProcessPoolExecutor(max_workers=workers,initializer=_init_stability_worker,initargs=(data,cfg)) as pool:
    future_map=[pool.submit(_pre_worker,t) for t in tasks]
    try:
        for fut in as_completed(future_map):
            row=fut.result(); got.append(row)
            print(json.dumps({'stage':"pre_parallel","completed":len(got),"total":len(tasks),"kind":row["kind"],"mode":row["mode"],
                "label":row["label"],"cost":row["summary"]['final_relative_cost_yuan']},"ensure_ascii=False),flush=True)
    except BaseException:
        for fut in future_map: fut.cancel()
        raise
arms=[(r["mode"],r["label"]):r for r in got if r["kind"]=="arm"]
schedules=[(r["mode"],r["label"]):r for r in got if r["kind"]=="schedule"]
for mode in ("q3","q4_3"):
    official= official_s3 from frozen(cfg,mode)
    arms[(mode,official_arm)]={"kind":"arm","mode":mode,"label":official_arm,"official"}
    schedules[(mode,"S3")]=["kind":"schedule","mode":mode,"label":"S3","official"]
frozen=Path(cfg["output_root"])/"frozen"
exp_rows=[arms[(m,a)]["summary"] for m in ("q3","q4_3") for a in cfg["experiments"]["arms"]]
exp_rows += [schedules[(m,s)]["summary"] for m in ("q3","q4_3") for s in ("S0","S1","S2")]
_write_csv(frozen/"experiment_summary.csv",exp_rows)
monthly=[]
for mode in ("q3","q4_3"):
    base=[x["date"]:x for x in arms[(mode,"raw_baseline")]["daily"]]; combo=schedules[(mode,"S3")]["daily"]
    for month in range(2,13):
        cr=[x for x in combo if int(x["date"][5:7])==month]; br=[base[x["date"]] for x in cr]
        dif=np.asarray([c["final_relative_cost_yuan"]-b["final_relative_cost_yuan"] for b,c in zip(br,cr)])
        lo,hi=moving_block_ci(dif,int(cfg["experiments"]["moving_block_days"]),int(cfg["experiments"]["bootstrap_replicates"]),int(cfg["experiments"]["bootstrap_seed"])+month)
        monthly.append({"mode":mode,"month":month,"days":len(dif),"baseline_cost_yuan":sum(x["final_relative_cost_yuan"] for x in br),
            "combination_cost_yuan":sum(x["final_relative_cost_yuan"] for x in cr),"difference_yuan":float(dif.sum()),
            "block_bootstrap_95_low_yuan":lo,"block_bootstrap_95_high_yuan":hi})
_write_csv(frozen/"monthly_effects.csv",monthly)
comparison=[]; monthly_schedules=[]
for mode in ("q3","q4_3"):
    for scenario in ("S0","S1","S2","S3"):
        schedule=schedules[(mode,scenario)]["daily"]
        for month in range(2,13):
            selected=[r for r in schedule if int(r["date"][5:7])==month]
            monthly_schedules.append({"mode":mode,"scenario":scenario,"month":month,"days":len(selected),
                "final_relative_cost_yuan":sum(r["final_relative_cost_yuan"] for r in selected),
                "sequential_cost_yuan":sum(r["sequential_cost_yuan"] for r in selected),"emergency_kwh":sum(r["emergency_kwh"] for r in selected),
                "unused_surplus_kwh":sum(r["unused_surplus_kwh"] for r in selected),"month_end_soc_kwh":selected[-1]["end_soc_kwh"]})
            s2=schedules[(mode,"S2")]["daily"]; s3=schedules[(mode,"S3")]["daily"]
            for metric,label,seedoff in (("final_relative_cost_yuan","S3 minus S2 final_relative",100),("sequential_cost_yuan","S3 minus S2 sequential",101)):
                dif=np.asarray([b[metric]-a[metric] for a,b in zip(s2,s3)]); lo,hi=moving_block_ci(dif,int(cfg["experiments"]["moving_block_days"]),int(cfg["experiments"]
["bootstrap_replicates"]),int(cfg["experiments"]["bootstrap_seed"])+seedoff)
                comparison.append({'mode':mode,'comparison':label,'days':len(dif),'point_difference_yuan':float(dif.sum()),'block_ci_low_yuan':lo,'block_ci_high_yuan':hi,
                    'positive_days':int(np.sum(dif>.005)),'negative_days':int(np.sum(dif<-.005)),'zero_days':int(np.sum(np.abs(dif)<=.005))})
_write_csv(frozen/"comparison_stats.csv",comparison); _write_csv(frozen/"monthly_schedules.csv",monthly_schedules)
summary_path=frozen/"model_run_summary.json"; summary=json.loads(summary_path.read_text(encoding="utf-8")); summary["experiments"]=exp_rows
summary_path.write_text(json.dumps(summary,ensure_ascii=False,indent=2),encoding="utf-8")
files=sorted(p for p in frozen.iterdir() if p.is_file() and p.name!="frozen_manifest.json")
manifest={"schema_version":cfg["schema_version"],"generated_at":summary["generated_at"],"source_root":cfg["source_root"],
    "command":"./run_pipeline.sh --config config/final.yaml","files":[{"path":p.name,"bytes":p.stat().st_size,"sha256":_sha256(p)} for p in files]}
(frozen/"frozen_manifest.json").write_text(json.dumps(manifest,ensure_ascii=False,indent=2),encoding="utf-8")
return {"experiments":exp_rows,"comparison":comparison}

```

src/model/forecast.py

```
from __future__ import annotations
```

```
from datetime import date
from typing import Any
```

```
import numpy as np
```

```
from .data import DataBundle
```

```
def history_days(data: DataBundle, target_day: date, cutoff_day: date, count: int = 4) -> list[int]:
    """Indices of the most recent same-weekday actual profiles, strictly before cutoff."""
    candidates = [i for i, d in enumerate(data.dates) if d < cutoff_day and d.weekday() == target_day.weekday()]
    if not candidates:
        candidates = [i for i, d in enumerate(data.dates) if d < cutoff_day]
    if not candidates:
        raise ValueError(f"no causal history before {cutoff_day}")
    return candidates[-count:]

```

```

def profile_horizon(data: DataBundle, start_global: int, length: int, cutoff_day: date, kind: str, count: int) -> tuple[np.ndarray, list[date]]:
    if kind == "load": matrix = data.load_kw
    elif kind == "pv": matrix = data.pv_kw
    elif kind == "price": matrix = data.realtime_price
    else: raise ValueError(f"unknown profile kind {kind}")
    out = np.empty(length, dtype=float)
    used: set[int] = set()
    for k in range(length):
        gi = start_global + k
        di, pos = divmod(gi, 144)
        target = data.dates[di]
        hist = history_days(data, target, cutoff_day, count)
        used.update(hist)
        out[k] = float(np.mean(matrix[hist, pos]))
    return out, [data.dates[i] for i in sorted(used)]

```

```

def published_pv_horizon(data: DataBundle, day: date, issue_hour: int, length: int) -> np.ndarray:
    hourly = data.pv_forecast_kw[(day, issue_hour)]
    return np.repeat(hourly, 6)[:length].astype(float, copy=True)

```

```

def pv_cap_horizon(data: DataBundle, day: date, issue_hour: int, length: int, cfg: dict[str, Any]) -> np.ndarray:
    fcfg = cfg["forecast"]; cutoff = data.date_index[day]; hist0 = max(0, cutoff - int(fcfg["cap_window_days"])); hist = data.pv_kw[hist0:cutoff]
    caps = np.full(24, np.inf)
    if len(hist):
        for lead in range(24):
            target_pos = ((issue_hour + lead) % 24) * 6
            caps[lead] = float(np.quantile(hist[:, target_pos:target_pos + 6].reshape(-1), float(fcfg["cap_quantile"])))
    return np.repeat(caps, 6)[:length]

```

```

def _past_issue_residuals(data: DataBundle, day: date, issue_hour: int, max_days: int) -> tuple[np.ndarray, list[date]]:
    idx = data.date_index
    candidates = [d for d in data.dates if d < day and (d, issue_hour) in data.pv_forecast_kw]
    candidates = candidates[-max_days:]
    rows: list[np.ndarray] = []
    kept: list[date] = []
    for old in candidates:
        start = idx[old] * 144 + issue_hour * 6
        if start + 144 > data.pv_kw.size:
            continue
        actual_hourly = data.pv_kw.reshape(-1)[start:start+144].reshape(24, 6).mean(axis=1)
        rows.append(actual_hourly - data.pv_forecast_kw[(old, issue_hour)])
        kept.append(old)
    return (np.vstack(rows) if rows else np.empty((0, 24))), kept

```

```

def corrected_pv_horizon(data: DataBundle, day: date, issue_hour: int, length: int, cfg: dict[str, Any]) -> tuple[np.ndarray, list[date]]:
    fcfg = cfg["forecast"]
    base = data.pv_forecast_kw[(day, issue_hour)].astype(float)
    residuals, dates = _past_issue_residuals(data, day, issue_hour, int(fcfg["bias_window_days"]))
    bias = residuals.mean(axis=0) if len(residuals) else np.zeros(24)
    corrected = np.maximum(base + bias, 0.0)
    expanded = np.repeat(corrected, 6)[:length]
    return np.minimum(expanded, pv_cap_horizon(data, day, issue_hour, length, cfg)), dates

```

```

def residual_scenarios(data: DataBundle, day: date, issue_hour: int, base: np.ndarray, cfg: dict[str, Any], centered: bool = False) -> tuple[np.ndarray, list[date]]:
    fcfg = cfg["forecast"]
    residuals, dates = _past_issue_residuals(data, day, issue_hour, int(fcfg["scenario_history_days"]))
    n = min(int(fcfg["scenario_count"]), len(residuals))
    if n == 0:
        return np.minimum(base[None, :], pv_cap_horizon(data, day, issue_hour, len(base), cfg)[None, :]), []
    residuals = residuals[-n:]
    if centered:
        residuals = residuals - residuals.mean(axis=0, keepdims=True)
    dates = dates[-n:]
    expanded = np.repeat(residuals, 6, axis=1)[:, :len(base)]
    scenarios = np.maximum(base[None, :] + expanded, 0.0)
    caps = pv_cap_horizon(data, day, issue_hour, len(base), cfg)
    return np.minimum(scenarios, caps[None, :]), dates

```

src/model/lp.py

```
from __future__ import annotations

from dataclasses import dataclass
import time
from typing import Any

import numpy as np
from scipy.optimize import linprog
from scipy.sparse import coo_matrix

@dataclass
class DispatchSolution:
    grid: np.ndarray
    charge: np.ndarray
    discharge: np.ndarray
    curtail: np.ndarray
    emergency: np.ndarray
    soc_end: np.ndarray
    objective: float
    elapsed_seconds: float
    iterations: int
    max_balance_residual: float
    max_soc_residual: float

def _one_way(charge: np.ndarray, discharge: np.ndarray, unused: np.ndarray, eta: float) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """Remove simultaneous cycling while preserving SOC change and energy balance."""
    delta = eta * charge - discharge / eta
    c = np.maximum(delta, 0) / eta
    d = np.maximum(-delta, 0) * eta
    u = unused + (d - discharge) - (c - charge)
    u[np.abs(u) < 1e-10] = 0.0
    if np.min(u) < -1e-7: raise AssertionError(f"one-way normalization produced negative unused energy {np.min(u)}")
    return c, d, np.maximum(u, 0.0)

def _method_options(method: str, tolerance: float) -> dict[str, Any]:
    options: dict[str, Any] = {"presolve": True}
    if method in {"highs", "highs-ds":
        options.update({"primal_feasibility_tolerance": tolerance, "dual_feasibility_tolerance": tolerance})
    if method in {"highs", "highs-ipm":
        options["ipm_optimality_tolerance"] = max(tolerance, 1e-10)
    return options

def solve_deterministic(
    load_kwh: np.ndarray,
    pv_kwh: np.ndarray,
    price: np.ndarray,
    initial_soc: float,
    baseline: np.ndarray,
    terminal_rule: str,
    terminal_target: float,
    terminal_penalty: float,
    cfg: dict[str, Any],
    method: str = "highs",
    tiebreak: float | None = None,
) -> DispatchSolution:
    load = np.asarray(load_kwh, float); pv = np.asarray(pv_kwh, float); price = np.asarray(price, float)
    base = np.asarray(baseline, float)
    T = len(load)
    if not (len(pv) == len(price) == len(base) == T):
        raise ValueError("LP input lengths differ")
    G = slice(0, T); C = slice(T, 2*T); D = slice(2*T, 3*T); U = slice(3*T, 4*T)
    E = slice(4*T, 5*T); DN = slice(5*T, 6*T); UP = slice(6*T, 7*T)
    ZP = 7*T; ZN = 7*T+1; n = 7*T+2
    committed = np.isfinite(base)
    c = np.zeros(n)
    c[G] = np.where(committed, 0.0, price)
    c[DN] = np.where(committed, -cfg["cost"]["down_adjustment_multiplier"] * price, 0.0)
    c[UP] = np.where(committed, cfg["cost"]["up_adjustment_multiplier"] * price, 0.0)
    tb = float(cfg["cost"]["throughput_tiebreak_yuan_per_kwh"]) if tiebreak is None else tiebreak
    c[C] = tb; c[D] = tb
    if terminal_rule == "soft_target": c[ZP] = terminal_penalty; c[ZN] = terminal_penalty
    rr: list[int] = []; cc: list[int] = []; vv: list[float] = []; rhs: list[float] = []; row = 0
    for t in range(T):
        rr += [row]*4; cc += [G.start+t, C.start+t, D.start+t, U.start+t]; vv += [1, -1, 1, -1]
```

```

    rhs.append(float(load[t]-pv[t])); row+=1
eta=float(cfg["storage"]*["charge_efficiency"])
for t in range(T):
    rr += [row]*3; cc += [C.start+t,D.start+t,E.start+t]; vv += [-eta,1/eta,1]
    if t: rr.append(row); cc.append(E.start+t-1); vv.append(-1); rhs.append(0.0)
    else: rhs.append(float(initial_soc))
    row+=1
for t in np.flatnonzero(committed):
    rr += [row]*3; cc += [G.start+t,DN.start+t,UP.start+t]; vv += [1,1,-1]
    rhs.append(float(base[t])); row+=1
if terminal_rule == "soft_target":
    rr += [row]*3; cc += [E.stop-1,ZP,ZN]; vv += [1,-1,1]; rhs.append(float(terminal_target)); row+=1
Aeq=coo_matrix((vv,rr,cc),shape=(row,n)).tocsr()
st=cfg["storage"]; step=float(st["power_kw"])*float(st["interval_minutes"])/60
bounds=(
    [(0,None)]*T+[(0,step)]*T+[(0,step)]*T+[(0,float(x)) for x in pv]
    +[(float(st["soc_min_kwh"]),float(st["soc_max_kwh"]))]*T
    +[(0,float(base[t])) if committed[t] else (0,0) for t in range(T)]
    +[(0,None) if committed[t] else (0,0) for t in range(T)]
    +[(0,None),(0,None)] if terminal_rule == "soft_target" else [(0,0),(0,0))
)
if terminal_rule in {"daily_closed","state_cycle","force_target":
    target = initial_soc if terminal_rule == "state_cycle" else terminal_target
    bounds[E.stop-1]=(float(target),float(target))
    started=time.perf_counter()
res=linprog(c,A_eq=Aeq,b_eq=np.asarray(rhs),bounds=bounds,method=method,
    options=_method_options(method,float(cfg["audit"]*["feasibility_tolerance"])))
elapsed=time.perf_counter()-started
if not res.success: raise RuntimeError(f"LP infeasible ({method}): {res.message}")
x=res.x; charge,discharge,unused=_one_way(x[C],x[D],x[U],eta); e0=np.r_[initial_soc,x[E][-1]]
bal=x[G]+pv+discharge-load-charge-unused
soc=x[E]-e0-eta*charge+discharge/eta
return DispatchSolution(x[G],charge,discharge,unused,np.zeros(T),x[E],float(res.fun),elapsed,int(res.nit),float(np.max(np.abs(bal))),float(np.max(np.abs(soc))))

def solve_stochastic(
    load_kwh: np.ndarray,
    pv_scenarios_kwh: np.ndarray,
    price: np.ndarray,
    initial_soc: float,
    baseline: np.ndarray,
    execute_intervals: int,
    terminal_rule: str,
    terminal_target: float,
    terminal_penalty: float,
    cfg: dict[str, Any],
    method: str = "highs",
    tiebreak: float | None = None,
) -> DispatchSolution:
    """Two-stage extensive-form LP; current executable block is nonanticipative."""
    load=np.asarray(load_kwh,float); pvs=np.asarray(pv_scenarios_kwh,float); price=np.asarray(price,float); base=np.asarray(baseline,float)
    S,T=pvs.shape
    if len(load)!=T or len(price)!=T or len(base)!=T: raise ValueError("stochastic LP input lengths differ")
    # Per scenario: grid, charge, discharge, curtail/unused, emergency, SOC, down, up.
    per=8*T; soft=terminal_rule=="soft_target"; n=S*per+(2*S if soft else 0)
    committed=np.isfinite(base); c=np.zeros(n); prob=1.0/S
    st=cfg["storage"]; eta=float(st["charge_efficiency"]); step=float(st["power_kw"])*float(st["interval_minutes"])/60
    down_mult=float(cfg["cost"]*["down_adjustment_multiplier"]); up_mult=float(cfg["cost"]*["up_adjustment_multiplier"]); em_mult=float(cfg["cost"]
["emergency_multiplier"])
    tb=float(cfg["cost"]*["throughput_tiebreak_yuan_per_kwh"]) if tiebreak is None else tiebreak
    bounds=list(tuple([float,float|None])=[])
    def slices(s:int):
        o=s*per
        return tuple(slice(o+i*T,o+(i+1)*T) for i in range(8))
    for s in range(S):
        G,C,D,U,H,E,DN,UP=slices(s)
        c[G]=prob*np.where(committed,0.0,price); c[C]=prob*tb; c[D]=prob*tb
        c[H]=prob*em_mult*price
        c[DN]=prob*np.where(committed,-down_mult*price,0.0); c[UP]=prob*np.where(committed,up_mult*price,0.0)
        bounds += [(0,None)]*T+[(0,step)]*T+[(0,step)]*T+[(0,None)]*T+[(0,None)]*T
        bounds += [(float(st["soc_min_kwh"]),float(st["soc_max_kwh"]))]*T
        bounds += [(0,float(base[t])) if committed[t] else (0,0) for t in range(T)]
        bounds += [(0,None) if committed[t] else (0,0) for t in range(T)]
    if soft:
        for s in range(S):
            c[S*per+2*S:S*per+2*S+2]=prob*terminal_penalty
            bounds += [(0,None),(0,None)]
    rr=list[int]=[]; cc=list[int]=[]; vv=list[float]=[]; rhs=list[float]=[]; row=0
    for s in range(S):
        G,C,D,U,H,E,DN,UP=slices(s)
        for t in range(T):
            rr += [row]*5; cc += [G.start+t,C.start+t,D.start+t,U.start+t,H.start+t]; vv += [1,-1,1,-1,1]
            rhs.append(float(load[t]-pvs[s,t])); row+=1
        for t in range(T):
            rr += [row]*3; cc += [C.start+t,D.start+t,E.start+t]; vv += [-eta,1/eta,1]
            if t: rr.append(row); cc.append(E.start+t-1); vv.append(-1); rhs.append(0.0)
            else: rhs.append(float(initial_soc))
            row+=1
        for t in np.flatnonzero(committed):
            rr += [row]*3; cc += [G.start+t,DN.start+t,UP.start+t]; vv += [1,1,-1]
            rhs.append(float(base[t])); row+=1

```

```

if soft:
    rr += [row]*3; cc += [E.stop-1,S*per+2*s,S*per+2*s+1]; vv += [1,-1,1]
    rhs.append(float(terminal_target)); row+=1
elif terminal_rule in {'daily_closed',"state_cycle","force_target"}:
    target=initial_soc if terminal_rule=="state_cycle" else terminal_target
    bounds[E.stop-1]=(float(target),float(target))
k=min(int(execute_intervals),T)
for s in range(1,S):
    a=slices(0); b=slices(s)
    for block in (0,1,2):
        for t in range(k):
            rr += [row,row]; cc += [a[block].start+t,b[block].start+t]; vv += [1,-1]; rhs.append(0.0); row+=1
Aeq=coo_matrix((vv,(rr,cc)),shape=(row,n)).tocsr()
started=time.perf_counter()
res=linprog(c,A_eq=Aeq,b_eq=np.asarray(rhs),bounds=bounds,method=method,
            options=_method_options(method,float(cfg["audit"] ["feasibility_tolerance"])))
elapsed=time.perf_counter()-started
if not res.success: raise RuntimeError(f"stochastic LP infeasible ({method}): {res.message}")
x=res.x; G,C,D,U,H,E,_=slices(0); charge,discharge,unused=_one_way(x[C],x[D],x[U],eta); e0=np.r_[initial_soc,x[E][:1]]
bal=x[G]+pvs[0]+discharge+x[H]-load-charge-unused
soc=x[E]-e0-eta*charge+discharge/eta
return DispatchSolution(x[G],charge,discharge,unused,x[H],x[E],float(res.fun),elapsed,int(res.nit),float(np.max(np.abs(bal))),float(np.max(np.abs(soc))))

```

src/model/rolling.py

```

from __future__ import annotations

```

```

from dataclasses import dataclass
from datetime import date, timedelta
from typing import Any

```

```

import numpy as np

```

```

from .data import DataBundle
from .forecast import corrected_pv_horizon, profile_horizon, published_pv_horizon, residual_scenarios
from .lp import DispatchSolution, solve_deterministic, solve_stochastic

```

```

@dataclass(frozen=True)
class RunSpec:
    mode: str          # q3 or q4_3
    scenario: str       # S0..S3
    arm: str
    method: str = "highs"
    tiebreak: float | None = None
    start_date: date | None = None
    end_date: date | None = None
    collect_detail: bool = True

```

```

def _arm_flags(arm: str) -> tuple[bool, bool, bool]:
    mapping = {
        "raw_baseline": (False, False, False),
        "crossday_only": (True, False, False),
        "bias_only": (False, True, False),
        "bias_crossday": (True, True, False),
        "scenario_crossday": (True, False, True),
        "all_three": (True, True, True),
    }
    if arm not in mapping: raise ValueError(f"unknown experiment arm {arm}")
    return mapping[arm]

```

```

def _interval(pos: int) -> str:
    a=pos*10; b=(pos+1)*10
    return f"{a//60:02d}:{a%60:02d}-{b//60:02d}:{b%60:02d}"

```

```

def _solve(
    data: DataBundle, cfg: dict[str, Any], day: date, issue: int, length: int,
    initial_soc: float, baseline: np.ndarray, use_bias: bool, use_scenarios: bool,
    terminal_rule: str, execute_intervals: int, mode: str, method: str, tiebreak: float | None,
) -> tuple[DispatchSolution, dict[str, Any]]:
    di=data.date_index[day]; start=di*144+issue*6
    count=int(cfg["forecast"] ["same_weekday_profiles"])
    load_kw, load_hist=profile_horizon(data,start,length,day,"load",count)

```

```

if mode=="q4_3": price, price_hist=profile_horizon(data,start,length,day,"price",count)
else:
    price=np.resize(data.fixed_price[issue*6:],length) if length <= 144-issue*6 else np.resize(np.r_[data.fixed_price[issue*6:],data.fixed_price],length)
    price_hist=[]
if use_bias:
    pv_kw,bias_hist=corrected_pv_horizon(data,day,issue,length,cfg)
else:
    pv_kw=published_pv_horizon(data,day,issue,length); bias_hist=[]
dt=float(cfg["storage"]["interval_minutes"])/60
target=float(cfg["study"]["terminal_soc_kwh"])
penalty=float(np.median(price))*float(cfg["rolling"]["selected_soft_penalty_multiplier"])
if use_scenarios:
    pv_scen,scenario_dates=residual_scenarios(data,day,issue,pv_kw,cfg,centered=use_bias)
    sol=solve_stochastic(load_kw*dt,pv_scen*dt,price,initial_soc,baseline,execute_intervals,
        terminal_rule,target,penalty,cfg,method,tiebreak)
else:
    scenario_dates=[]
    sol=solve_deterministic(load_kw*dt,pv_kw*dt,price,initial_soc,baseline,terminal_rule,
        target,penalty,cfg,method,tiebreak)
meta={
    "forecast_load_kwh":load_kw*dt,"forecast_pv_kwh":pv_kw*dt,"forecast_price":price,
    "load_history_dates":','.join(d.isoformat() for d in load_hist),
    "price_history_dates":','.join(d.isoformat() for d in price_hist),
    "pv_bias_history_dates":','.join(d.isoformat() for d in bias_hist),
    "residual_scenario_dates":','.join(d.isoformat() for d in scenario_dates),
}
return sol,meta

def simulate(data: DataBundle, cfg: dict[str, Any], spec: RunSpec) -> dict[str, Any]:
    crossday,use_bias,use_scenarios=_arm_flags(spec.arm)
    study=cfg["study"]
    start=spec.start_date or date.fromisoformat(study["official_start"])
    end=spec.end_date or date.fromisoformat(study["official_end"])
    days=[d for d in data.dates if start<=d<=end]
    if not days: raise ValueError("empty simulation period")
    allowed=list(cfg["experiments"]["schedules"][spec.scenario])
    eta=float(cfg["storage"]["charge_efficiency"]); dt=float(cfg["storage"]["interval_minutes"])/60
    final_target=float(study["terminal_soc_kwh"]); total_intervals=len(data.dates)*144
    soc=float(study["initial_soc_kwh"]); first_initial=soc
    dispatch_rows:list[dict[str,Any]]=[]; version_rows:list[dict[str,Any]]=[]; daily_rows:list[dict[str,Any]]=[]
    solve_stats:list[dict[str,Any]]=[]; physical_violation_count=0
    for day in days:
        di=data.date_index[day]
        baseline=np.full(144,np.nan); current_grid=np.full(144,np.nan); current_c=np.zeros(144); current_d=np.zeros(144); current_u=np.zeros(144)
        f_load=np.zeros(144); f_pv=np.zeros(144); f_price=np.zeros(144); decision_owner=np.zeros(144,dtype=int)
        sequential_delta=np.zeros(144); sensitivity_grid=np.full(144,np.nan)
        day_soc_start=soc
        for ai,issue in enumerate(allowed):
            pos=issue*6; global_start=di*144+pos
            if crossday:
                length=min(int(cfg["rolling"]["horizon_hours"])*6,total_intervals-global_start)
            else:
                length=144-pos
            next_issue=allowed[ai+1] if ai+1<len(allowed) else 24
            execute=(next_issue-issue)*6
            b=np.full(length,np.nan)
            if issue>0: b[:144-pos]=baseline[pos:]
            if global_start+length==total_intervals:
                terminal="force_target"
            elif crossday:
                terminal=str(cfg["rolling"]["selected_terminal_rule"])
            else:
                terminal="daily_closed"
            old=current_grid[pos:].copy() if issue>0 else np.full(144-pos,np.nan)
            backup=None
            if use_scenarios:
                # The extensive form governs the executable first block. A causal
                # deterministic point-forecast solve supplies the 00:00 plan and
                # the explicitly hypothetical sequential-settlement path. No
                # random scenario branch is ever recorded as a future commitment.
                backup,_=solve(data,cfg,day,issue,length,soc,b,use_bias,False,terminal,execute,
                    spec.mode,spec.method,spec.tiebreak)
            sol,meta=solve(data,cfg,day,issue,length,soc,b,use_bias,use_scenarios,terminal,execute,
                spec.mode,spec.method,spec.tiebreak)
            solve_stats.append({"date":day.isoformat(),"issue_hour":issue,"elapsed_seconds":sol.elapsed_seconds,
                "iterations":sol.iterations,"max_balance_residual_kwh":sol.max_balance_residual,
                "max_soc_residual_kwh":sol.max_soc_residual,"scenario_count":len(meta["residual_scenario_dates"].split(",")) if meta["residual_scenario_dates"] else
            })
        cur_len=144-pos
        if issue==0:
            if backup is None:
                baseline[:]=sol.grid[:144]
                current_grid[:]=sol.grid[:144]; current_c[:]=sol.charge[:144]; current_d[:]=sol.discharge[:144]; current_u[:]=sol.curtail[:144]
            else:
                baseline[:]=backup.grid[:144]
                first=min(execute,144)
                baseline[:first]=sol.grid[:first]
                current_grid[:]=baseline
                current_c[:]=backup.charge[:144]; current_d[:]=backup.discharge[:144]; current_u[:]=backup.curtail[:144]
                current_c[:first]=sol.charge[:first]; current_d[:first]=sol.discharge[:first]; current_u[:first]=sol.curtail[:first]

```

```

        sensitivity_grid[:]=baseline
    else:
        price_actual=data.realtime_price[di] if spec.mode=="q4_3" else data.fixed_price
        apply_len=min(cur_len,execute) if use_scenarios else cur_len
        old_sensitivity=sensitivity_grid[pos:].copy()
        sensitivity_proposal=backup_grid[:cur_len] if backup is not None else sol.grid[:cur_len]
        down_s=np.maximum(old_sensitivity-sensitivity_proposal,0); up_s=np.maximum(sensitivity_proposal-old_sensitivity,0)
        sequential_delta[pos:] += -float(cfg["cost"])*["down_adjustment_multiplier"]*price_actual[pos:]*down_s + float(cfg["cost"]
["up_adjustment_multiplier"])*price_actual[pos:]*up_s
        sensitivity_grid[pos:]=sensitivity_proposal
        sl=slice(pos,pos+apply_len)
        current_grid[sl]=sol.grid[apply_len]; current_c[sl]=sol.charge[apply_len]; current_d[sl]=sol.discharge[apply_len]; current_u[sl]=sol.curtail[apply_len]
        forecast_apply=min(cur_len,execute) if use_scenarios else cur_len
        sl=slice(pos,pos+forecast_apply)
        f_load[sl]=meta["forecast_load_kwh"][:forecast_apply]; f_pv[sl]=meta["forecast_pv_kwh"][:forecast_apply]; f_price[sl]=meta["forecast_price"][:forecast_apply];
    decision_owner[sl]=issue
    old_commit = baseline[pos:] if issue==0 else old
    sensitivity_old=np.full(cur_len,np.nan) if issue==0 else old_sensitivity
    sensitivity_new=baseline[pos:] if issue==0 else sensitivity_proposal
    for k in range(length):
        gi=global_start+k; tdi,tpos=divmod(gi,144); target_day=data.dates[tdi]
        is_preview=target_day!=day
        preview_recourse=bool(is_preview or (use_scenarios and issue>0 and k>=execute))
        formal=bool(not is_preview and (issue==0 or not use_scenarios or k<execute))
        preview_grid=(backup_grid[k] if backup is not None else sol.grid[k]) if is_preview else ""
        if not spec.collect_detail:
            continue
        version_rows.append({
            "date":day.isoformat(),"target_date":target_day.isoformat(),"mode":spec.mode,"scenario":spec.scenario,"model_arm":spec.arm,
            "decision_time":f"{day.isoformat()} {issue:02d}:00","forecast_issue_time":f"{day.isoformat()} {issue:02d}:00",
            "effective_start":f"{day.isoformat()} {issue:02d}:00","effective_end":f"{(day+timedelta(days=1)).isoformat()} {issue:02d}:00",
            "history_cutoff_date":(day-timedelta(days=1)).isoformat(),"position":tpos+1,
            "physical_interval": interval(tpos),"official_label":data.official_labels[tpos],
            "old_commitment_kwh":"" if not formal or issue==0 else float(old_commit[k]),
            "new_commitment_kwh":float(baseline[k]) if formal and issue==0 else (float(sol.grid[k]) if formal else ""),
            "sensitivity_old_commitment_kwh":"" if is_preview or issue==0 else float(sensitivity_old[k]),
            "sensitivity_new_commitment_kwh":"" if is_preview else float(sensitivity_new[k]),
            "preview_grid_kwh":float(preview_grid) if preview_grid!="" else "",
            "decision_basis":"next_block_stochastic" if formal and use_scenarios and k<execute else ("deterministic_day_ahead_backup" if formal and issue==0 else
("deterministic_point_forecast_sensitivity" if not is_preview else "next_day_preview")),
            "soc_at_decision_kwh":float(soc),
            "load_history_dates":meta["load_history_dates"] if k==0 else "", "price_history_dates":meta["price_history_dates"] if k==0 else "",
            "pv_bias_history_dates":meta["pv_bias_history_dates"] if k==0 else "", "residual_scenario_dates":meta["residual_scenario_dates"] if k==0 else "",
            "is_preview_next_day":int(is_preview),"is_preview_recourse":int(preview_recourse),
        })
        # Execute only until the next enabled decision (or midnight).
        ex=min(execute,144-pos)
        if ex:
            soc=float(sol.soc_end[ex-1])
            actual_load=data.load_kw[di]*dt; actual_pv=data.pv_kw[di]*dt
            price_actual=data.realtime_price[di] if spec.mode=="q4_3" else data.fixed_price
            soc_end=np.empty(144); s=day_soc_start
            for j in range(144):
                s=s+eta*current_c[j]-current_d[j]/eta; soc_end[j]=s
            soc_start=np_r_[day_soc_start,soc_end[-1]]; soc=float(soc_end[-1])
            physical_violation_count+=int(np.sum((soc_end<float(cfg["storage"])*["soc_min_kwh"])-float(cfg["audit"])*["feasibility_tolerance"]))(soc_end>float(cfg["storage"]
["soc_max_kwh"])+float(cfg["audit"])*["feasibility_tolerance"])))
            net=current_grid+actual_pv+current_d-actual_load-current_c
            emergency=np.maximum(-net,0); unused=np.maximum(net,0)
            down=np.maximum(baseline-current_grid,0); up=np.maximum(current_grid-baseline,0)
            plan_cost=price_actual*baseline
            adj_cost=-float(cfg["cost"])*["down_adjustment_multiplier"]*price_actual*down+float(cfg["cost"])*["up_adjustment_multiplier"]*price_actual*up
            emergency_cost=float(cfg["cost"])*["emergency_multiplier"]*price_actual*emergency
            total=plan_cost+adj_cost+emergency_cost
            sequential=plan_cost+sequential_delta+emergency_cost
            for j in range(144):
                if not spec.collect_detail:
                    continue
                curtail_pv=min(float(current_u[j]),float(f_pv[j])); planned_unused_nonpv=max(float(current_u[j])-float(f_pv[j]),0.0)
                dispatch_rows.append({
                    "date":day.isoformat(),"position":j+1,"source_timestamp":data.source_timestamps[j],
                    "physical_interval": interval(j),"official_label":data.official_labels[j],
                    "plan_00_grid_kwh":float(baseline[j]),"final_adjusted_grid_kwh":float(current_grid[j]),
                    "adjustment_kwh":float(current_grid[j]-baseline[j]),"charge_kwh":float(current_c[j]),
                    "discharge_kwh":float(current_d[j]),"soc_start_kwh":float(soc_start[j]),"soc_end_kwh":float(soc_end[j]),
                    "curtail_pv_kwh":curtail_pv,"planned_unused_nonpv_kwh":planned_unused_nonpv,"forecast_load_kwh":float(f_load[j]),"forecast_pv_kwh":float(f_pv[j]),
                    "actual_load_kwh":float(actual_load[j]),"actual_pv_kwh":float(actual_pv[j]),"forecast_price":float(f_price[j]),
                    "settlement_price":float(price_actual[j]),"emergency_kwh":float(emergency[j]),"unused_surplus_kwh":float(unused[j]),
                    "plan_cost_yuan":float(plan_cost[j]),"adjustment_cost_yuan":float(adj_cost[j]),"emergency_cost_yuan":float(emergency_cost[j]),
                    "final_relative_cost_yuan":float(total[j]),"sequential_cost_yuan":float(sequential[j]),"decision_hour":int(decision_owner[j]),
                    "history_cutoff_date":(day-timedelta(days=1)).isoformat(),"model_arm":spec.arm,"scenario":spec.scenario,
                })
            daily_rows.append({
                "date":day.isoformat(),"mode":spec.mode,"scenario":spec.scenario,"model_arm":spec.arm,
                "start_soc_kwh":float(day_soc_start),"end_soc_kwh":float(soc),"plan_grid_kwh":float(np.sum(baseline)),
                "final_grid_kwh":float(np.sum(current_grid)), "emergency_kwh":float(np.sum(emergency)),
                "unused_surplus_kwh":float(np.sum(unused)), "plan_cost_yuan":float(np.sum(plan_cost)),
                "adjustment_cost_yuan":float(np.sum(adj_cost)), "emergency_cost_yuan":float(np.sum(emergency_cost)),
                "final_relative_cost_yuan":float(np.sum(total)), "sequential_cost_yuan":float(np.sum(sequential)),
            })
        tol=float(cfg["audit"])*["numeric_tolerance"])
    if abs(first_initial-float(study["initial_soc_kwh"]))>tol:
        raise AssertionError("first day SOC differs from contract")
    if end==date.fromisoformat(study["official_end"]) and abs(soc-final_target)>tol:

```

```

        raise AssertionError(f"terminal SOC {soc} differs from {final_target}")
    tol=float(cfg["audit"]["feasibility_tolerance"]); st=cfg["storage"]
    violation_count=sum(int(x["max_balance_residual_kwh"]>tol or x["max_soc_residual_kwh"]>tol) for x in solve_stats)
    violation_count+=physical_violation_count
    return {"spec":spec,"dispatch":dispatch_rows,"decision_versions":.version_rows,"daily":daily_rows,"solve_stats":solve_stats,
            "summary":{
                "mode":spec.mode,"scenario":spec.scenario,"model_arm":spec.arm,"days":len(days),
                "initial_soc_kwh":float(daily_rows[0]["start_soc_kwh"]);terminal_soc_kwh":float(daily_rows[-1]["end_soc_kwh"]),
                "plan_cost_yuan":float(sum(x["plan_cost_yuan"] for x in daily_rows)),
                "adjustment_cost_yuan":float(sum(x["adjustment_cost_yuan"] for x in daily_rows)),
                "emergency_cost_yuan":float(sum(x["emergency_cost_yuan"] for x in daily_rows)),
                "final_relative_cost_yuan":float(sum(x["final_relative_cost_yuan"] for x in daily_rows)),
                "sequential_cost_yuan":float(sum(x["sequential_cost_yuan"] for x in daily_rows)),
                "emergency_kwh":float(sum(x["emergency_kwh"] for x in daily_rows)),
                "unused_surplus_kwh":float(sum(x["unused_surplus_kwh"] for x in daily_rows)),
                "solver_seconds":float(sum(x["elapsed_seconds"] for x in solve_stats));solver_runs":len(solve_stats),
                "max_balance_residual_kwh":float(max(x["max_balance_residual_kwh"] for x in solve_stats)),
                "max_soc_residual_kwh":float(max(x["max_soc_residual_kwh"] for x in solve_stats)),
                "constraint_violation_count":int(violation_count),
            }}

```

scripts/build_documents.py

```

#!/usr/bin/env python3
"""Inject frozen results into the paper and create the final DOCX sources."""
from __future__ import annotations

import csv
import json
import re
import shutil
from pathlib import Path

import numpy as np
import pandas as pd
from docx import Document
from docx.enum.section import WD_SECTION
from docx.enum.table import WD_ALIGN_VERTICAL, WD_TABLE_ALIGNMENT
from docx.enum.text import WD_ALIGN_PARAGRAPH, WD_BREAK, WD_LINE_SPACING
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
from docx.shared import Cm, Pt, RGBColor

ROOT = Path(__file__).resolve().parents[1]
FINAL = ROOT / "outputs" / "c_final_v1"
FROZEN = FINAL / "frozen"
PAPER = FINAL / "paper"
CHARTS = FINAL / "charts"
WORK = ROOT / "work" / "paper_build"
# Arial Unicode MS is available as a TrueType font to the bundled headless
# converter and contains the complete Simplified Chinese glyph set. macOS TTC
# family names such as Songti SC/Heiti SC are visible to desktop applications
# but are not resolved reliably by this conversion runtime.
BODY_FONT = "Arial Unicode MS"
HEADING_FONT = "Arial Unicode MS"

def csv_rows(path: Path) -> list[dict[str, str]]:
    with path.open(encoding="utf-8-sig", newline="") as f:
        return list(csv.DictReader(f))

def fmt_value(key: str, value: object) -> str:
    if isinstance(value, str):
        return value
    if any(x in key for x in ("COUNT", "N_DAYS", "ROWS")):
        return f"{int(value):d}"
    x = float(value)
    if "RESIDUAL" in key:
        return f"{x:.3e}"
    if any(xk in key for xk in ("KWH", "SOC_MIN", "SOC_MAX")):
        return f"{x:.4f}"
    if "PERCENT" in key:
        return f"{x:.2f}"
    return f"{x:.2f}"

```

```

def find(rows: list[dict[str, str]], **terms: str) -> dict[str, str]:
    found = [r for r in rows if all(str(r.get(k)) == str(v) for k, v in terms.items())]
    if not found:
        raise KeyError(terms)
    return found[-1]

def comparison_sentence(mode: str, exp: list[dict[str, str]], arm: str, baseline: str = "raw_baseline") -> str:
    a = find(exp, mode=mode, scenario="S3", model_arm=arm)
    b = find(exp, mode=mode, scenario="S3", model_arm=baseline)
    diff = float(a["final_relative_cost_yuan"]) - float(b["final_relative_cost_yuan"])
    em = float(a["emergency_kwh"]) - float(b["emergency_kwh"])
    verb = "减少" if diff < 0 else "增加"
    emverb = "减少" if em < 0 else "增加"
    return f"{mode.upper().replace('_', '-')} 中，三项组合相对原始因果基线费用{verb}{abs(diff):.2f}元，紧急购电量{emverb}{abs(em):.4f}kWh。"

def ablation_table(exp: list[dict[str, str]]) -> str:
    labels = {
        "raw_baseline": "原始预测基线", "crossday_only": "仅跨日终端", "bias_only": "仅偏差校正",
        "bias_crossday": "偏差校正+跨日", "scenario_crossday": "场景+跨日", "all_three": "三项组合",
    }
    lines = ["模式 | 实验组 | 总费用 元 | 紧急购电 kWh | 未用能量 kWh |", "|---|---|---|---|"]
    seen = set()
    for r in exp:
        key = (r["mode"], r["scenario"], r["model_arm"])
        if r["scenario"] != "S3" or r["model_arm"] not in labels or key in seen:
            continue
        seen.add(key)
        lines.append(f"| {r['mode'].upper().replace('_', '-')} | {labels[r['model_arm']] | {float(r['final_relative_cost_yuan']):.2f} | {float(r['emergency_kwh']):.4f} | {float(r['unused_surplus_kwh']):.4f} |")
    return "\n".join(lines)

def group_emergency(frame: pd.DataFrame, threshold: float) -> list[tuple[str, str, float]]:
    out = []
    for day, part in frame.groupby("date", sort=True):
        part = part.sort_values("position")
        current = None
        for row in part.itertuples():
            q = float(row.emergency_kwh); pos = int(row.position)
            if q < threshold:
                if current is not None: out.append((str(day), interval(current[0], current[1]), current[2])); current = None
                continue
            if current is None: current = [pos, pos, q]
            elif pos == current[1] + 1: current[1] = pos; current[2] += q
            else: out.append((str(day), interval(current[0], current[1]), current[2])); current = [pos, pos, q]
            if current is not None: out.append((str(day), interval(current[0], current[1]), current[2]))
    return out

def interval(start_pos: int, end_pos: int) -> str:
    def hm(minutes: int) -> str:
        return f"{minutes//60}:{minutes%60:02d}"
    return f"{hm((start_pos-1)*10)}-{hm(end_pos*10)}"

def appendix_tables(cfg: dict) -> str:
    selected = {"2025-03-20", "2025-06-21", "2025-09-23", "2025-12-21"}
    chunks = ["### 指定日期结果表", "", "以下各表由全精度执行明细自动生成；紧急区间按配置的四位显示阈值重建。"]
    q1 = pd.read_csv(FROZEN / "q1_dispatch.csv").sort_values("position")
    wanted = ["10:00-10:10", "12:00-12:10", "14:00-14:10", "16:00-16:10", "18:00-18:10", "20:00-20:10"]
    chunks += ["", "#### Q1 指定时段购电量", "", "| 时间段 | 购电量 kWh |", "|---|---|"]
    for label in wanted:
        r = q1[q1["official_label"] == label]
        if len(r): chunks.append(f"| {label} | {float(r.iloc[0]['plan_00_grid_kwh']):.4f} |")
    chunks += ["", "| 全天购电量 kWh | 全天购电费 元 |", "|---|---|", f"| {q1['plan_00_grid_kwh'].sum():.4f} | {q1['plan_cost_yuan'].sum():.2f} |", "", "#### Q1 四小时充放电", "", "| 时间段 | 充电量 kWh | 放电量 kWh |", "|---|---|"]
    for block in range(6):
        p = q1.iloc[block*24:(block+1)*24]
        chunks.append(f"| {block*4}:00-{(block+1)*4}:00 | {p['charge_kwh'].sum():.4f} | {p['discharge_kwh'].sum():.4f} |")
    chunks += ["", "0:00 与 24:00 储电量均为 6000.0000 kWh。"]

```

```

for name, title in (("q2_dispatch.csv", "Q2"), ("q3_dispatch.csv", "Q3"), ("q4_2_dispatch.csv", "Q4-2"), ("q4_3_dispatch.csv", "Q4-3")):
    d = pd.read_csv(FROZEN / name)
    d = d[d["date"].isin(selected)]

    chunks += ["", f"#### {title} 指定日期汇总", "", "| 日期 | 0 点计划量 kWh | 最终购电量 kWh | 总费用 元 | 紧急购电量 kWh |", "|---|---|---|---|"]
    for day, p in d.groupby("date", sort=True):
        chunks.append(f"| {day} | {p['plan_00_grid_kwh'].sum():.4f} | {p['final_adjusted_grid_kwh'].sum():.4f} | {p['final_relative_cost_yuan'].sum():.2f} | {p['emergency_kwh'].sum():.4f} |")

    chunks += ["", f"#### {title} 指定日期紧急购电区间", "", "| 日期 | 紧急购电时间段 | 购电量 kWh |", "|---|---|---|"]
    groups = group_emergency(d, float(cfg["audit"])["emergency_display_omit_threshold_kwh"])
    if groups:
        for day, span, amount in groups:
            chunks.append(f"| {day} | {span} | {amount:.4f} |")
    else:
        chunks.append("| 四个指定日期 | 无 | 0.0000 |")
return "\n".join(chunks)

def source_code_appendix() -> str:
    """Embed the complete runnable delivery source without machine-specific paths."""
    paths = [ROOT / "config" / "final.yaml", ROOT / "requirements.txt", ROOT / "run_pipeline.sh"]
    paths += sorted(ROOT / "src").rglob("*.py")
    paths += sorted(ROOT / "scripts").glob("*.py")
    paths += sorted(ROOT / "scripts").glob("*.mjs")
    paths += sorted(ROOT / "tests").glob("*.py")

    chunks = ["### 完整可运行源程序", "", "以下代码与支撑材料中的同名文件一致；运行入口为 run_pipeline.sh。"]
    for path in paths:
        rel = path.relative_to(ROOT).as_posix()
        chunks.extend(["", f"#### {rel}", "", "``text", path.read_text(encoding="utf-8"), "``"])
    return "\n".join(chunks)

def expand_paper() -> str:
    registry = json.loads(FROZEN / "frozen_results.json").read_text(encoding="utf-8"))["values"]
    exp = csv_rows(FROZEN / "experiment_summary.csv")
    monthly = csv_rows(FROZEN / "monthly_effects.csv")
    draft = (ROOT / "work" / "paper_agent" / "paper_outline_and_draft.md").read_text(encoding="utf-8")
    for key, value in registry.items():
        token = "{" + f"RESULT:{key}" + "}"
        draft = draft.replace(token, fmt_value(key, value))

    q3label = str(registry["Q3.S2_S3.STABILITY_LABEL"]); q43label = str(registry["Q4_3.S2_S3.STABILITY_LABEL"])
    q3diff = float(registry["Q3.S2_S3.DIFF_YUAN"]); q43diff = float(registry["Q4_3.S2_S3.DIFF_YUAN"])
    complex_values = {
        "Q3.ABLATION_NARRATIVE": comparison_sentence("q3", exp, "all_three"),
        "ABLATION.COMBINED_ANALYSIS": comparison_sentence("q3", exp, "all_three") + " " + comparison_sentence("q4_3", exp, "all_three") +
    }

    "\n\n[[FIGURE:fig1_ablation_cost.png|六组增强消融的总费用比较]]\n\n[[FIGURE:fig2_update_schedule.png|最终组合模型的 S0-S3 更新频率比较]],
    "Q3.S2_S3.CONCLUSION_SENTENCE": ("S2 与 S3 的差异在多求解器复算中数值不可区分，不据此主张 18 点更新有稳定收益。" if "不可区分" in q3label else
    f"在本数据和口径下，S3 相对 S2 的费用差稳定为 {q3diff:.2f} 元。"),
    "Q4_3.S2_S3.CONCLUSION_SENTENCE": ("S2 与 S3 的差异在多求解器复算中数值不可区分，不据此主张 18 点更新有稳定收益。" if "不可区分" in q43label else
    f"在本数据和口径下，S3 相对 S2 的费用差稳定为 {q43diff:.2f} 元。"),
    "MONTHLY_AND_BLOCK_ANALYSIS": "逐月差异未采用跨月平均替代，各月均从逐日配对费用计算。完整点估计和 7 天移动块区间见冻结比较表；区间跨
    0 时不作显著性表述。" "\n\n[[FIGURE:fig4_monthly_effects.png|三项组合相对原始因果基线的逐月费用差和移动块区间]],
    "SETTLEMENT_SENSITIVITY_ANALYSIS": f"Q3 S3 的逐次结算比主口径高{float(registry["Q3.S3.SEQUENTIAL_COST_YUAN"])-
    float(registry["Q3.S3.TOTAL_COST_YUAN"]):.2f} 元；Q4-3 S3 相差{float(registry["Q4_3.S3.SEQUENTIAL_COST_YUAN"])-
    float(registry["Q4_3.S3.TOTAL_COST_YUAN"]):.2f} 元。" "\n\n[[FIGURE:fig3_cost_components.png|最终组合策略的费用分项]],
    "PERFECT_INFORMATION_GAP_ANALYSIS": f"Q2 因果策略比完全信息对照高{float(registry["Q2.CAUSAL.TOTAL_COST_YUAN"])-
    float(registry["Q2.PERFECT_INFO.TOTAL_COST_YUAN"]):.2f} 元；Q4-2 的对应差额为{float(registry["Q4_2.CAUSAL.TOTAL_COST_YUAN"])-
    float(registry["Q4_2.PERFECT_INFO.TOTAL_COST_YUAN"]):.2f} 元。",
    "SOLVER_STABILITY_TABLE": "frozen/stability_summary.csv",
    "FINAL_CONCLUSIONS_NUMBERED": "1. 确定性 LP 能在题设边界内完成日内削峰填谷。" "\n2. 因果日前结果与完全信息结果存在明确的信息差距，正式结果
    不使用未来实际量。" "\n3. 24 小时跨日窗口、偏差校正和历史残差场景的作用由同口径消融确定，不能由单次费用差单独归因。" "\n4. S2-S3 结论严格服从多求解器
    和分币稳定性门。",
    "REPRODUCIBILITY_APPENDIX": "model_run_summary.json、resolved_config.json、frozen_manifest.json 和 SHA256SUMS.json",
    "PROVENANCE_TABLE": "evidence/result_provenance.csv",
    }
    for key, value in complex_values.items():
        token = "{" + f"RESULT:{key}" + "}"
        draft = draft.replace(token, value)
    draft = draft.replace("六组消融的完整表由 `Q3.ABLATION_TABLE` 注入。", "六组消融结果如下。" "\n\n" + ablation_table(exp))
    draft = draft.replace("论文最终版还应自动插入题面表 1 和表 2 的指定时段结果，数据键分别为 `Q1.SPECIFIED_PURCHASE_ROWS` 和
    `Q1.FOUR_HOUR_STORAGE_ROWS`。", "题面指定时段和四小时储能汇总列入附录指定日期结果表。")
    draft = draft.replace("各次求解均保存全精度目标和可行性残差。", "各次求解均保存全精度目标和可行性残差。" "\n\n[[FIGURE:fig6_stability.png|S2 与 S3 的多求解
    器和并列破除系数复算]]")

```

```

draft = draft.replace("统一流水线按以下顺序运行", "[[FIGURE:fig5_representative_day.png|代表日的计划购电、最终执行、紧急购电与 SOC 轨迹]]\n\n统一流水线
按以下顺序运行")
cfg = json.loads((FROZEN / "resolved_config.json").read_text(encoding="utf-8"))
draft += "\n\n" + appendix_tables(cfg) + "\n\n" + source_code_appendix() + "\n"
unresolved = sorted(set(re.findall(r"\{RESULT:[^}+\}\}", draft)))
if unresolved:
    raise RuntimeError("Unresolved paper placeholders: " + ", ".join(unresolved))
return draft

def clean_inline(text: str) -> str:
    text = re.sub(r"([^\]]+)\((https?://[^\)]+)\)", r"\1 ( \2 )", text)
    return text.replace(" ", "")

def set_cell_shading(cell, fill: str) -> None:
    tc_pr = cell._tc.get_or_add_tcPr(); shd = tc_pr.find(qn("w:shd"))
    if shd is None: shd = XmlElement("w:shd"); tc_pr.append(shd)
    shd.set(qn("w:fill"), fill)

def set_cell_borders(cell, color: str = "D9D9D9") -> None:
    tc_pr = cell._tc.get_or_add_tcPr(); borders = tc_pr.first_child_found_in("w:tcBorders")
    if borders is None: borders = XmlElement("w:tcBorders"); tc_pr.append(borders)
    for edge in ("top", "left", "bottom", "right", "insideH", "insideV"):
        node = borders.find(qn(f"w:{edge}"))
        if node is None: node = XmlElement(f"w:{edge}"); borders.append(node)
        node.set(qn("w:val"), "single"); node.set(qn("w:sz"), "4"); node.set(qn("w:color"), color)

def repeat_header(row) -> None:
    tr_pr = row._tr.get_or_add_trPr(); tbl_header = XmlElement("w:tblHeader"); tbl_header.set(qn("w:val"), "true"); tr_pr.append(tbl_header)

def add_page_number(paragraph) -> None:
    paragraph.alignment = WD_ALIGN_PARAGRAPH.CENTER
    run = paragraph.add_run(); begin = XmlElement("w:fldChar"); begin.set(qn("w:fldCharType"), "begin")
    instr = XmlElement("w:instrText"); instr.set(qn("xml:space"), "preserve"); instr.text = " PAGE "
    separate = XmlElement("w:fldChar"); separate.set(qn("w:fldCharType"), "separate")
    end = XmlElement("w:fldChar"); end.set(qn("w:fldCharType"), "end")
    run._r.extend([begin, instr, separate, end])

def configure(doc: Document) -> None:
    section = doc.sections[0]; section.page_width = Cm(21); section.page_height = Cm(29.7)
    section.top_margin = Cm(2.5); section.bottom_margin = Cm(2.5); section.left_margin = Cm(2.5); section.right_margin = Cm(2.5)
    styles = doc.styles
    normal = styles["Normal"]; normal.font.name = BODY_FONT; normal.font.size = Pt(10.5)
    normal._element.rPr.rFonts.set(qn("w:eastAsia"), BODY_FONT)
    normal.paragraph_format.line_spacing = 1.25; normal.paragraph_format.space_after = Pt(3)
    for style_name, size, font in (("Title", 18, HEADING_FONT), ("Heading 1", 14, HEADING_FONT), ("Heading 2", 12, HEADING_FONT), ("Heading 3", 11,
    HEADING_FONT)):
        st = styles[style_name]; st.font.name = font; st.font.size = Pt(size); st.font.color.rgb = RGBColor(0, 0, 0); st.font.bold = True
        st._element.rPr.rFonts.set(qn("w:eastAsia"), font); st.paragraph_format.keep_with_next = True
        styles["Title"].paragraph_format.space_after = Pt(12)
        add_page_number(section.footer.paragraphs[0])

def add_table(doc: Document, lines: list[str]) -> None:
    data = [[clean_inline(x.strip()) for x in line.strip().strip("|").split("|")] for line in lines if not re.match(r"^[^s*]?s*:-+", line)]
    if not data: return
    cols = max(map(len, data)); table = doc.add_table(rows=len(data), cols=cols); table.alignment = WD_TABLE_ALIGNMENT.CENTER
    table.autofit = True
    for i, row in enumerate(data):
        for j in range(cols):
            cell = table.cell(i, j); cell.text = row[j] if j < len(row) else ""; cell.vertical_alignment = WD_ALIGN_VERTICAL.CENTER; set_cell_borders(cell)
        for p in cell.paragraphs:
            p.paragraph_format.space_after = Pt(0); p.alignment = WD_ALIGN_PARAGRAPH.CENTER if j == 0 or i == 0 else WD_ALIGN_PARAGRAPH.LEFT
            for run in p.runs:
                run.font.name = BODY_FONT; run.font.size = Pt(8.5); run._element.rPr.rFonts.set(qn("w:eastAsia"), BODY_FONT)
                if i == 0: run.font.bold = True; run.font.color.rgb = RGBColor(255, 255, 255)
            if i == 0: set_cell_shading(cell, "1F4E78")
            elif i % 2 == 0: set_cell_shading(cell, "F4F7FA")

```

```
repeat_header(table.rows[0])
doc.add_paragraph().paragraph_format.space_after = Pt(1)
```

```
def markdown_to_docx(text: str, output: Path) -> None:
    doc = Document(); configure(doc)
    lines = text.splitlines(); i = 0; in_code = False
    while i < len(lines):
        line = lines[i]
        if line.startswith("` ` ` ` ") or line.startswith("` ` ` `"):
            in_code = not in_code; i += 1; continue
        if in_code:
            p = doc.add_paragraph()
            p.paragraph_format.space_after = Pt(0)
            p.paragraph_format.line_spacing = 1.0
            run = p.add_run(line if line else " ")
            run.font.name = "Consolas"; run.font.size = Pt(6.5)
            run._element.rPr.rFonts.set(qn("w: eastAsia"), BODY_FONT)
            i += 1; continue
        if line.startswith("[") and i + 1 < len(lines) and lines[i + 1].startswith("["):
            block = []
            while i < len(lines) and lines[i].startswith("["):
                block.append(lines[i]); i += 1
            add_table(doc, block); continue
        fig = re.fullmatch(r"\[([FIGURE:([^\]]+))\]", line.strip())
        if fig:
            p = doc.add_paragraph(); p.alignment = WD_ALIGN_PARAGRAPH.CENTER
            p.add_run().add_picture(str(CHARTS / fig.group(1)), width=Cm(15.2))
            cap = doc.add_paragraph(f"图 {fig.group(2)}"); cap.alignment = WD_ALIGN_PARAGRAPH.CENTER
            for run in cap.runs: run.font.size = Pt(9); run.font.name = BODY_FONT; run._element.rPr.rFonts.set(qn("w: eastAsia"), BODY_FONT)
            i += 1; continue
        if line.startswith("# "):
            p = doc.add_paragraph(clean_inline(line[2:]), style="Title"); p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        elif line.startswith("## "):
            heading = clean_inline(line[3:])
            if heading.strip() == "附录":
                doc.add_page_break()
                doc.add_paragraph(heading, style="Heading 1")
            elif line.startswith("### "):
                doc.add_paragraph(clean_inline(line[4:]), style="Heading 2")
            elif line.startswith("#### "):
                doc.add_paragraph(clean_inline(line[5:]), style="Heading 3")
            elif re.match(r"^(d+\.s+)", line):
                doc.add_paragraph(clean_inline(re.sub(r"^(d+\.s+", "", line)), style="List Number")
            elif line.startswith("- "):
                doc.add_paragraph(clean_inline(line[2:]), style="List Bullet")
            elif not line.strip():
                pass
            else:
                p = doc.add_paragraph(); p.paragraph_format.first_line_indent = Pt(21)
                p.paragraph_format.keep_together = False
                run = p.add_run(clean_inline(line));
                i += 1
    output.parent.mkdir(parents=True, exist_ok=True); doc.save(output)
```

```
def expand_ai() -> str:
    text = (ROOT / "work" / "paper_agent" / "AI 工具使用详情_内容稿.md").read_text(encoding="utf-8")
    text = text.replace("状态：内容模板", "状态：机器核验完成；提交前仍须团队人工签核")
    text = text.replace(
        "核心建模选择、题意解释、参数冻结、结果取舍和最终提交由参赛队审核后决定。参赛队对拟提交的模型、程序、数值、引文和文字逐项进行人工审查与核实。",
        "核心建模选择、题意解释、参数冻结、结果取舍和最终提交须由参赛队审核决定；提交前还须对模型、程序、数值、引文和文字逐项人工审查与核实。",
    )
    text = text.replace("GPT-5 系列；最终以任务界面显示名称补录", "GPT-5 ( Codex 任务运行环境 ) ")
    text = text.replace(
        "下表必须根据实际会话界面、导出日志或账户记录补齐。未知型号不能猜填；应写“平台未显示具体型号”，并保留可核验的截图或导出记录。",
        "下表根据现有会话环境和留存文件如实登记；无法从旧文件验证的型号明确标注，不作猜测。",
    )
    text = text.replace(
        "[ A2 | 先前参与本题的其他 AI 工具 | {{AI_VENDOR:A2}} | {{AI_MODEL:A2}} | {{AI_DATE:A2}} | {{AI_LOG:A2}} | {{AI_USED:A2}} |",
        "[ A2 | 先前参与本题的 AI 智能体 | 无法由现有文件验证 | 无法由现有文件验证 | 旧成果生成阶段 | outputs/c_audit_v1 与原结果文件 | 是，作为旧成果与审计对象 |",
    )
    text = text.replace("[AI_LOG:A1]", "model_run_summary.json、audit_run.log 与 SHA-256 清单")
    text = "\n".join(line for line in text.splitlines() if not any(tag in line for tag in ("{{AI_MODEL:A3}}", "{{AI_DATE:A3}}", "{{AI_USED:A3}}")))
    replacements = {
        "{{EVIDENCE:TIME_MAPPING}}": "frozen/time_index.csv", "{{EVIDENCE:CAUSAL_AUDIT}}": "audit/audit_report.md 与 frozen/decision_versions.csv",
        "{{EVIDENCE:SETTLEMENT}}": "audit/settlement_recalculation.csv", "{{EVIDENCE:ROLLING}}": "frozen/q3_decision_versions.csv 与 q4_3_decision_versions.csv",
        "{{EVIDENCE:PAPER_QA}}": "evidence/result_provenance.csv 与 SHA256SUMS.json",
    }
```

```

    }
    for k, v in replacements.items(): text = text.replace(k, v)
    text = text.split("## 发布前必须删除或替换的标记", 1)[0].rstrip() + "\n"
    result_example = "" + "{{" + "RESULT:..." + ""
    text = text.replace(result_example, "冻结结果键")
    leftovers = re.findall(r"\{\{[^}]+\}\}", text)
    if leftovers: raise RuntimeError(f"Unresolved AI detail placeholders: {leftovers}")
    return text

def main() -> None:
    PAPER.mkdir(parents=True, exist_ok=True); WORK.mkdir(parents=True, exist_ok=True)
    expanded = expand_paper(); source = FINAL / "evidence" / "paper_source_frozen.md"; source.parent.mkdir(parents=True, exist_ok=True); source.write_text(expanded,
encoding="utf-8")
    markdown_to_docx(expanded, PAPER / "C 题论文.docx")
    ai = expand_ai(); (FINAL / "evidence" / "AI 工具使用详情_冻结稿.md").write_text(ai, encoding="utf-8")
    markdown_to_docx(ai, WORK / "AI 工具使用详情.docx")
    print(json.dumps({"paper_docx": str(PAPER / "C 题论文.docx"), "ai_docx": str(WORK / "AI 工具使用详情.docx")}, ensure_ascii=False))

if __name__ == "__main__":
    main()

```

scripts/freeze_results.py

```

#!/usr/bin/env python3
"""Build the single scalar registry consumed by the paper and final checks."""
from __future__ import annotations

```

```

import csv
import hashlib
import json
from collections import Counter
from datetime import date
from pathlib import Path

```

```

import numpy as np
import pandas as pd

```

```

ROOT = Path(__file__).resolve().parents[1]
FINAL = ROOT / "outputs" / "c_final_v1"
FROZEN = FINAL / "frozen"

```

```

def csv_rows(name: str) -> list[dict[str, str]]:
    with (FROZEN / name).open(encoding="utf-8-sig", newline="") as f:
        return list(csv.DictReader(f))

```

```

def sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for block in iter(lambda: f.read(1 << 20), b''):
            h.update(block)
    return h.hexdigest()

```

```

def find(rows: list[dict[str, str]], **terms: str) -> dict[str, str]:
    found = [r for r in rows if all(str(r.get(k)) == str(v) for k, v in terms.items())]
    if not found:
        raise KeyError(f"No result row for {terms}")
    return found[-1]

```

```

def stable_label(rows: list[dict[str, str]], mode: str) -> tuple[str, list[float]]:

```

```

group = [r for r in rows if r["mode"] == mode and r["model_arm"] == "all_three"]
by = {(r["solver_method"], r["tiebreak"], r["scenario"]): r for r in group}
diffs = []
for method in sorted({r["solver_method"] for r in group}):
    for tb in sorted({r["tiebreak"] for r in group}, key=float):
        if (method, tb, "S2") in by and (method, tb, "S3") in by:
            diffs.append(float(by[(method, tb, "S3")]["final_relative_cost_yuan"]) - float(by[(method, tb, "S2")]["final_relative_cost_yuan"]))
cents = {round(v, 2) for v in diffs}
signs = {int(np.sign(v)) for v in diffs if abs(v) >= 0.005}
stable = len(diffs) == 9 and len(cents) == 1 and len(signs) == 1 and 0.0 not in cents
label = f"稳定到分币 ( S3-S2={diffs[0]:.2f}元 )" if stable else "数值不可区分"
return label, diffs

def audit_metrics() -> dict[str, float | int]:
    max_balance = 0.0; max_soc = 0.0; soc_min = float("inf"); soc_max = -float("inf"); simultaneous = 0
    eta = 0.9
    for name in ("q1_dispatch.csv", "q2_dispatch.csv", "q3_dispatch.csv", "q4_2_dispatch.csv", "q4_3_dispatch.csv"):
        d = pd.read_csv(FROZEN / name)
        balance = (d["final_adjusted_grid_kwh"] + d["emergency_kwh"] + d["actual_pv_kwh"] + d["discharge_kwh"]
                    - d["actual_load_kwh"] - d["charge_kwh"] - d["unused_surplus_kwh"])
        soc = d["soc_end_kwh"] - d["soc_start_kwh"] - eta*d["charge_kwh"] + d["discharge_kwh"]/eta
        max_balance = max(max_balance, float(balance.abs().max()))
        max_soc = max(max_soc, float(soc.abs().max()))
        soc_min = min(soc_min, float(d["soc_start_kwh", "soc_end_kwh"].min().min()))
        soc_max = max(soc_max, float(d["soc_start_kwh", "soc_end_kwh"].max().max()))
        simultaneous += int(((d["charge_kwh"] > 1e-7) & (d["discharge_kwh"] > 1e-7)).sum())
    return {"max_balance": max_balance, "max_soc": max_soc, "soc_min": soc_min, "soc_max": soc_max, "simultaneous": simultaneous}

def audit_actual(audit: dict, check_id: str) -> dict:
    matches = {item.get("actual", {} for item in audit.get("checks", []) if item.get("id") == check_id}
    if not matches:
        raise KeyError(f"Audit check not found: {check_id}")
    return matches[-1]

def main() -> None:
    exp = csv_rows("experiment_summary.csv")
    official = csv_rows("official_summary.csv") if (FROZEN / "official_summary.csv").exists() else exp
    stability = csv_rows("stability_summary.csv")
    comparison = csv_rows("comparison_stats.csv") if (FROZEN / "comparison_stats.csv").exists() else []
    perfect = csv_rows("perfect_information_summary.csv") if (FROZEN / "perfect_information_summary.csv").exists() else []
    audit = json.loads((FINAL / "audit" / "audit_summary.json").read_text(encoding="utf-8"))
    legacy = json.loads((FINAL / "audit" / "legacy_audit_snapshot.json").read_text(encoding="utf-8"))
    q1 = pd.read_csv(FROZEN / "q1_dispatch.csv")
    q1_grid = float(q1["plan_00_grid_kwh"].sum()); q1_cost = float(q1["plan_cost_yuan"].sum())
    no_storage = float((np.maximum(q1["actual_load_kwh"] - q1["actual_pv_kwh"], 0) * q1["settlement_price"]).sum())
    phys = audit_metrics()
    cfg = json.loads((FROZEN / "resolved_config.json").read_text(encoding="utf-8")) if (FROZEN / "resolved_config.json").exists() else json.loads((ROOT / "config" /
"final.yaml").read_text(encoding="utf-8"))

    reg: dict[str, object] = {
        "META.N_DAYS": 334,
        "Q1.TOTAL_GRID_KWH": q1_grid,
        "Q1.TOTAL_COST_YUAN": q1_cost,
        "Q1.COST_WITHOUT_STORAGE_YUAN": no_storage,
        "Q1.COST_SAVING_PERCENT": 100 * (no_storage - q1_cost) / no_storage,
        "Q3.SELECTED_ARM": cfg["experiments"]["official_arm"],
        "Q4_3.SELECTED_ARM": cfg["experiments"]["official_arm"],
        "Q3.SELECTED_TERMINAL_RULE": cfg["rolling"]["selected_terminal_rule"],
    }
    for mode, prefix in (("q2", "Q2.CAUSAL"), ("q4_2", "Q4_2.CAUSAL")):
        r = find(official, mode=mode)
        reg[f"{prefix}.TOTAL_COST_YUAN"] = float(r["final_relative_cost_yuan"])
        reg[f"{prefix}.EMERGENCY_KWH"] = float(r["emergency_kwh"])
    for mode, prefix in (("q2", "Q2.PERFECT_INFO"), ("q4_2", "Q4_2.PERFECT_INFO")):
        r = find(perfect, mode=mode)
        reg[f"{prefix}.TOTAL_COST_YUAN"] = float(r["purchase_cost_yuan"])
        reg[f"{prefix}.EMERGENCY_KWH"] = float(r.get("emergency_kwh", 0) or 0)
    for mode, prefix in (("q3", "Q3"), ("q4_3", "Q4_3")):
        s0 = find(exp, mode=mode, scenario="S0", model_arm="all_three")
        for scen in ("S0", "S1", "S2", "S3"):
            r = find(exp, mode=mode, scenario=scen, model_arm="all_three")
            reg[f"{prefix}{scen}.TOTAL_COST_YUAN"] = float(r["final_relative_cost_yuan"])
            reg[f"{prefix}{scen}.SEQUENTIAL_COST_YUAN"] = float(r["sequential_cost_yuan"])
            reg[f"{prefix}{scen}.EMERGENCY_KWH"] = float(r["emergency_kwh"])
            reg[f"{prefix}{scen}.DIFF_VS_S0_YUAN"] = float(r["final_relative_cost_yuan"]) - float(s0["final_relative_cost_yuan"])
        label, diffs = stable_label(stability, mode)
        reg[f"{prefix}.S2_S3.STABILITY_LABEL"] = label
        reg[f"{prefix}.S2_S3.DIFF_YUAN"] = float(find(exp, mode=mode, scenario="S3", model_arm="all_three")["final_relative_cost_yuan"]) - float(find(exp, mode=mode,
scenario="S2", model_arm="all_three")["final_relative_cost_yuan"])
        if comparison:
            c = find(comparison, mode=mode, comparison="S3_minus_S2_final_relative")

```

```

reg[f"{prefix}.S2_S3.BLOCK_CI95_LOW_YUAN"] = float(c["block_ci_low_yuan"])
reg[f"{prefix}.S2_S3.BLOCK_CI95_HIGH_YUAN"] = float(c["block_ci_high_yuan"])

counts = audit.get("status_counts", {})
legacy_counts = legacy.get("status_counts", {})
reg.update({
    "AUDIT.PASS_COUNT_NEW": int(counts.get("PASS", 0)),
    "AUDIT.WARN_COUNT_NEW": int(counts.get("WARN", 0)),
    "AUDIT.FAIL_COUNT_NEW": int(counts.get("FAIL", 0)),
    "AUDIT.LEGACY_FAIL_COUNT": int(legacy_counts.get("FAIL", 0)),
    "AUDIT.LEGACY_NOT_VERIFIABLE_COUNT": int(legacy_counts.get("NOT_VERIFIABLE", 0)),
    "AUDIT.MAX_BALANCE_RESIDUAL_KWH": phys["max_balance"],
    "AUDIT.MAX_SOC_RESIDUAL_KWH": phys["max_soc"],
    "AUDIT.SOC_MIN_KWH": phys["soc_min"],
    "AUDIT.SOC_MAX_KWH": phys["soc_max"],
    "AUDIT.SIMULTANEOUS_COUNT": phys["simultaneous"],
})
versions = pd.read_csv(FROZEN / "decision_versions.csv")
reg["AUDIT.DECISION_VERSION_ROWS"] = int(len(versions))
future_history = release_violations = past_rewrites = commitment_mismatches = 0
for workbook in ("result3.xlsx", "result4-3.xlsx"):
    seq = audit_actual(audit, f"FINAL-{workbook}-SETTLEMENT-SEQUENTIAL")
    rel = audit_actual(audit, f"FINAL-{workbook}-FORECAST-RELEASE")
    future_history += int(seq.get("future_history_references", 0))
    release_violations += int(rel.get("violations", 0))
    for chain_name in ("formal_chain", "sensitivity_chain"):
        chain = seq.get(chain_name, {})
        past_rewrites += int(chain.get("past_interval_rewrites", 0))
        commitment_mismatches += int(chain.get("commitment_chain_errors", 0))
        commitment_mismatches += int(seq.get("formal_chain", {}).get("final_dispatch_mismatches", 0))
reg["AUDIT.FUTURE_HISTORY_REFERENCE_COUNT"] = future_history
reg["AUDIT.FORECAST_RELEASE_VIOLATION_COUNT"] = release_violations
reg["AUDIT.PAST_REWRITE_COUNT"] = past_rewrites
reg["AUDIT.COMMITMENT_MISMATCH_COUNT"] = commitment_mismatches

out = {"schema_version": "1.0", "values": reg}
result_path = FROZEN / "frozen_results.json"
result_path.write_text(json.dumps(out, ensure_ascii=False, indent=2), encoding="utf-8")
files = sorted(p for p in FROZEN.iterdir() if p.is_file() and p.name != "frozen_manifest.json")
manifest = {
    "schema_version": "1.0",
    "command": "bash run_pipeline.sh --config config/final.yaml",
    "files": [{"path": p.name, "bytes": p.stat().st_size, "sha256": sha256(p)} for p in files],
}
(FROZEN / "frozen_manifest.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
print(json.dumps({"values": len(reg), "manifest_files": len(files), ensure_ascii=False}))

if __name__ == "__main__":
    main()

```

scripts/make_charts.py

```

#!/usr/bin/env python3
"""Generate the paper figures exclusively from the frozen result files."""
from __future__ import annotations

```

```

import csv
from collections import defaultdict
from pathlib import Path

```

```

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

```

```

ROOT = Path(__file__).resolve().parents[1]
FINAL = ROOT / "outputs" / "c_final_v1"
FROZEN = FINAL / "frozen"
CHARTS = FINAL / "charts"

```

```

COLORS = {"q3": "#1F4E78", "q4_3": "#C55A11"}

```

```

def save(fig: plt.Figure, stem: str) -> None:
    fig.tight_layout()
    fig.savefig(CHARTS / f"{stem}.png", dpi=220, bbox_inches="tight", facecolor="white")
    fig.savefig(CHARTS / f"{stem}.svg", bbox_inches="tight", facecolor="white")
    plt.close(fig)

def main() -> None:
    CHARTS.mkdir(parents=True, exist_ok=True)
    plt.rcParams.update({'font.family': 'DejaVu Sans', 'font.size': 9, 'axes.spines.top': False, 'axes.spines.right': False})
    exp = pd.read_csv(FROZEN / "experiment_summary.csv").drop_duplicates(
        subset=["mode", "scenario", "model_arm"], keep="last"
    )
    monthly = pd.read_csv(FROZEN / "monthly_effects.csv")
    stab = pd.read_csv(FROZEN / "stability_summary.csv")

    # 1. Pre-registered ablation at S3.
    arms = ["raw_baseline", "crossday_only", "bias_only", "bias_crossday", "scenario_crossday", "all_three"]
    labels = ["Raw", "Cross-day", "Bias", "Bias + cross-day", "Scenarios + cross-day", "All three"]
    fig, ax = plt.subplots(figsize=(9.0, 4.6))
    x = np.arange(len(arms)); width = 0.36
    for offset, mode in [(-width/2, "q3"), (width/2, "q4_3")]:
        d = exp[(exp["mode"] == mode) & (exp["scenario"] == "S3")].set_index("model_arm")
        vals = [float(d.loc[a, "final_relative_cost_yuan"]) / 1e6 if a in d.index else np.nan for a in arms]
        ax.bar(x + offset, vals, width, label=mode.upper().replace("_", "-"), color=COLORS[mode])
    ax.set_xticks(x, labels, rotation=18, ha="right")
    ax.set_ylabel("Settled cost (million yuan)")
    ax.set_title("Ablation under the same causal information boundary")
    ax.legend(frameon=False, ncol=2)
    ax.grid(axis="y", color="#D9D9D9", linewidth=.6)
    save(fig, "fig1_ablation_cost")

    # 2. S0-S3 schedule comparison for the selected model.
    fig, ax = plt.subplots(figsize=(7.4, 4.2))
    schedules = ["S0", "S1", "S2", "S3"]
    for mode in ["q3", "q4_3"]:
        d = exp[(exp["mode"] == mode) & (exp["model_arm"] == "all_three")].set_index("scenario")
        vals = [float(d.loc[s, "final_relative_cost_yuan"]) / 1e6 if s in d.index else np.nan for s in schedules]
        ax.plot(schedules, vals, marker="o", linewidth=2, label=mode.upper().replace("_", "-"), color=COLORS[mode])
    ax.set_ylabel("Settled cost (million yuan)")
    ax.set_xlabel("Forecast update schedule")
    ax.set_title("Value of intraday forecast updates")
    ax.grid(color="#D9D9D9", linewidth=.6)
    ax.legend(frameon=False)
    save(fig, "fig2_update_schedule")

    # 3. Cost composition for the selected S3 strategy.
    d = exp[(exp["scenario"] == "S3") & (exp["model_arm"] == "all_three")].set_index("mode")
    fig, ax = plt.subplots(figsize=(6.7, 4.3))
    modes = [m for m in ["q3", "q4_3"] if m in d.index]
    bottom = np.zeros(len(modes))
    for col, label, color in [
        ("plan_cost_yuan", "00:00 plan", "#5B9BD5"),
        ("adjustment_cost_yuan", "Net adjustment", "#ED7D31"),
        ("emergency_cost_yuan", "Emergency", "#A5A5A5"),
    ]:
        vals = d.loc[modes, col].astype(float).to_numpy() / 1e6
        ax.bar([m.upper().replace("_", "-") for m in modes], vals, bottom=bottom, label=label, color=color)
        bottom += vals
    ax.set_ylabel("Cost component (million yuan)")
    ax.set_title("Cost composition of the selected causal strategy")
    ax.legend(frameon=False)
    ax.grid(axis="y", color="#D9D9D9", linewidth=.6)
    save(fig, "fig3_cost_components")

    # 4. Month-level paired effects and uncertainty intervals.
    fig, axes = plt.subplots(2, 1, figsize=(8.0, 6.5), sharex=True)
    for ax, mode in zip(axes, ["q3", "q4_3"]):
        d = monthly[monthly["mode"] == mode].sort_values("month")
        y = d["difference_yuan"].astype(float).to_numpy() / 1000
        lo = d["block_bootstrap_95_low_yuan"].astype(float).to_numpy() / 1000
        hi = d["block_bootstrap_95_high_yuan"].astype(float).to_numpy() / 1000
        ax.errorbar(d["month"].astype(str), y, yerr=np.vstack([y-lo, hi-y]), marker="o", capsized=3, color=COLORS[mode])
        ax.axhline(0, color="#444444", linewidth=.8)
    ax.set_ylabel("Difference (thousand yuan)")
    ax.set_title(mode.upper().replace("_", "-"))
    ax.grid(axis="y", color="#D9D9D9", linewidth=.6)
    axes[-1].set_xlabel("Month")
    fig.suptitle("All-three strategy minus raw causal baseline")
    save(fig, "fig4_monthly_effects")

    # 5. Representative summer-day operation from the selected Q3 dispatch.
    q3 = pd.read_csv(FROZEN / "q3_dispatch.csv")
    sample_date = "2025-06-21" if (q3["date"] == "2025-06-21").any() else str(q3.iloc[len(q3)//2]["date"])

```

```

day = q3[q3["date"] == sample_date].sort_values("position")
hours = (day["position"].astype(float).to_numpy() - .5) / 6
fig, ax1 = plt.subplots(figsize=(9.0, 4.5))
ax1.plot(hours, day["plan_00_grid_kwh"], label="00:00 plan", color="#7F7F7F", linewidth=1.2)
ax1.plot(hours, day["final_adjusted_grid_kwh"], label="Executed grid purchase", color="#1F4E78", linewidth=1.6)
ax1.plot(hours, day["emergency_kwh"], label="Emergency", color="#C00 000", linewidth=1.0)
ax1.set_xlabel("Hour")
ax1.set_ylabel("Energy per 10 min (kWh)")
ax2 = ax1.twinx()
ax2.plot(hours, day["soc_end_kwh"], label="SOC", color="#70AD47", linewidth=1.5)
ax2.set_ylabel("Stored energy (kWh)")
lines, labels1 = ax1.get_legend_handles_labels(); lines2, labels2 = ax2.get_legend_handles_labels()
ax1.legend(lines + lines2, labels1 + labels2, frameon=False, ncol=2, loc="upper center")
ax1.set_title(f"Representative rolling operation on {sample_date}")
ax1.grid(color="#D9D9D9", linewidth=.6)
save(fig, "fig5_representative_day")

```

```

# 6. Pair each S2 run with the exactly matching S3 solver/tie-break run.
fig, ax = plt.subplots(figsize=(9.0, 4.6))
stable = stab[(stab["model_arm"] == "all_three") & (stab["scenario"].isin(["S2", "S3"]))].copy()
methods = ["highs", "highs-ds", "highs-ipm"]
tiebreaks = sorted(stable["tiebreak"].astype(float).unique())
pairs = [(m, tb) for m in methods for tb in tiebreaks]
labels6 = [f"{m}\n{tb:.0e}" for m, tb in pairs]
x6 = np.arange(len(pairs))
for mode in ["q3", "q4_3"]:
    dm = stable[stable["mode"] == mode]
    vals = []
    for method, tb in pairs:
        same_tb = (dm["tiebreak"].astype(float) - tb).abs() < 1e-15
        s2 = dm[(dm["scenario"] == "S2") & (dm["solver_method"] == method) & same_tb]
        s3 = dm[(dm["scenario"] == "S3") & (dm["solver_method"] == method) & same_tb]
        vals.append(float(s3.iloc[0]["final_relative_cost_yuan"] - s2.iloc[0]["final_relative_cost_yuan"]) if len(s2) and len(s3) else np.nan)
    ax.plot(x6, vals, marker="o", linewidth=1.6, label=mode.upper().replace("_", "-"), color=COLORS[mode])
ax.set_xticks(x6, labels6)
ax.set_ylabel("S3 minus S2 settled cost (yuan)")
ax.set_title("S2/S3 sensitivity to solver and tie-break coefficient")
ax.axhline(0, color="#444 444", linewidth=.8)
ax.grid(axis="y", color="#D9D9D9", linewidth=.6)
ax.legend(frameon=False)
save(fig, "fig6_stability")

```

```

(CHARTS / "figure_notes.md").write_text(
    "# 图表说明\n\n 所有图由 `scripts/make_charts.py` 直接读取 `frozen/` 冻结结果生成。"
    "\n图 1 为预注册消融，图 2 为 S0-S3 更新次数比较，图 3 为费用分项，图 4 为逐月配对差及 7 天移动块区间，"
    "\n图 5 为 {sample_date} 代表日，图 6 为 S2/S3 多求解器与并列破除系数复算。"
    "\n",
    encoding="utf-8",
)

```

```

if __name__ == "__main__":
    main()

```

scripts/make_evidence.py

```

#!/usr/bin/env python3
"""Create human-readable evidence tables from the frozen result registry."""
from __future__ import annotations

```

```

import csv
import hashlib
import json
import shutil
from pathlib import Path

```

```

ROOT = Path(__file__).resolve().parents[1]
FINAL = ROOT / "outputs" / "c_final_v1"
FROZEN = FINAL / "frozen"
EVIDENCE = FINAL / "evidence"

```

```

def digest(path: Path) -> str:
    h = hashlib.sha256(path.read_bytes()).hexdigest()
    return h

```

```

def rows(path: Path) -> list[dict[str, str]]:
    with path.open(encoding="utf-8-sig", newline="") as f:
        return list(csv.DictReader(f))

def money(value: str | float) -> str:
    return f"{float(value):,.2f}"

def main() -> None:
    EVIDENCE.mkdir(parents=True, exist_ok=True)
    for name in ("literature_evidence.md", "experiment_plan.md", "format_anonymity_checklist.md"):
        source = ROOT / "work" / "paper_agent" / name
        if source.exists():
            shutil.copy2(source, EVIDENCE / name)

    exp = rows(FROZEN / "experiment_summary.csv")
    official = rows(FROZEN / "official_summary.csv")
    summary_lines = [
        "# 冻结实验结果摘要", "",
        "正式口径为因果策略、2025-02-01 至 2025-12-31、SOC 6000→6000 kWh、最终调整量相对 0:00 计划结算。", "",
        "| 问题 | 计划费 | 调整净费用 | 紧急购电费 | 总费用 | 逐次结算 | 紧急购电量 | 未用能量 |",
        "|---|---|---|---|---|---|---|---|",
    ]
    for r in official:
        summary_lines.append(
            f"| {r['mode'].upper().replace('_', '-')} | {money(r['plan_cost_yuan'])} | {money(r['adjustment_cost_yuan'])} | "
            f"{money(r['emergency_cost_yuan'])} | {money(r['final_relative_cost_yuan'])} | {money(r['sequential_cost_yuan'])} | "
            f"{float(r['emergency_kwh']):.4f} | {float(r['unused_surplus_kwh']):.4f} | "
        )
    (EVIDENCE / "experiment_results.md").write_text("\n".join(summary_lines) + "\n", encoding="utf-8")

    cfg = json.loads((ROOT / "config" / "final.yaml").read_text(encoding="utf-8"))
    contract = f"""# C 题最终模型合同

## 时间与信息边界

- 内部十分钟时段按右端点解释附件时间戳；官方结果表标签保持原样，映射保存在 `frozen/time_index.csv`。
- 正式评估区间为 {cfg['study']['official_start']} 至 {cfg['study']['official_end']}；1 月只用于训练、偏差估计、场景抽取和终端参数选择。
- 负载与波动电价预测只使用决策日前最近 {cfg['forecast']['same_weekday_profiles']} 个同星期日期；实际量只在相应时段执行后用于结算。
- Q3/Q4-3 在 0、6、12、18 时求解未来 {cfg['rolling']['horizon_hours']} 小时并只执行下一个 {cfg['rolling']['execute_hours']} 小时。午夜前的下一日量为预览，不形成正式承诺。

## 电池与费用

- SOC：{cfg['storage']['soc_min_kwh']:.0f}–{cfg['storage']['soc_max_kwh']:.0f} kWh；额定容量 {cfg['storage']['capacity_kwh']:.0f} kWh；功率上限 {cfg['storage']['power_kw']:.0f} kW；充放电效率均为 {cfg['storage']['charge_efficiency']:.2f}。
- 正式端点统一为 {cfg['study']['initial_soc_kwh']:.0f}→{cfg['study']['terminal_soc_kwh']:.0f} kWh。
- 主结算为最终调整量相对 0:00 计划，下调、上调与紧急购电倍率分别为 {cfg['cost']['down_adjustment_multiplier']:.1f}、{cfg['cost']['up_adjustment_multiplier']:.1f}、{cfg['cost']['emergency_multiplier']:.1f}；逐次承诺变化另作敏感性。
- 紧急购电区间从全精度明细重建，仅省略绝对量小于 {cfg['audit']['emergency_display_omit_threshold_kwh']:.5f} kWh、即四位小数显示为 0 的十分钟记录。

## 三项增强

- 光伏偏差校正：按发布时间×提前小时取过去 {cfg['forecast']['bias_window_days']} 天残差均值，截为非负，并用过去 {cfg['forecast']['cap_window_days']} 天相应小时 {cfg['forecast']['cap_quantile']:.3%} 分位数约束异常值。
- 场景滚动：最近最多 {cfg['forecast']['scenario_count']} 条同发布时间完整残差轨迹；首个六小时块非预见，后续允许场景递补；最小化等概率期望费用。
- 终端软目标参数只由 1 月滚动验证选择；2–12 月不再调参。最后一个正式时段强制回到 {cfg['study']['terminal_soc_kwh']:.0f} kWh。

## 可复现与结论门

```

正式表、图和论文只读取 `frozen/frozen\_manifest.json` 登记的数据。S2/S3 使用三种 HiGHS 方法和三种并列破除系数复算；方向或分币不稳定时只能报告“数值不可区分”。完全信息模型仅在证据包中作事后对照。

```
"""
(EVIDENCE / "model_contract.md").write_text(contract, encoding="utf-8")

coverage = """# 要求覆盖表

| 题目或交付要求 | 正式证据 | 验收方式 |
|---|---|---|
| Q1 确定性日计划 | `workbooks/result1.xlsx`、`frozen/q1_dispatch.csv` | 144 时段、能量平衡、6000→6000 |
| Q2 因果日前策略 | `workbooks/result2.xlsx`、`frozen/q2_dispatch.csv` | 历史截止日早于决策、334×144 |
| Q3 日内调整 | `workbooks/result3.xlsx`、`frozen/q3_dispatch.csv`、`decision_versions.csv` | 承诺链、过去冻结、两种结算 |
| Q4 波动价格 | `workbooks/result4-2.xlsx`、`workbooks/result4-3.xlsx` | 因果价格预测、实际价格事后结算 |
| 24h 跨日滚动 | `decision_versions.csv`、SOC 列 | 下一日预览、执行块、跨日连续 |
| 三项增强与消融 | `experiment_summary.csv`、`monthly_effects.csv` | 同信息边界、同端点、同结算 |
| 数值稳定性 | `stability_summary.csv` | 3 方法×3 并列破除系数、分币判断 |
| 工作簿 | 五份 result 文件、`qa/workbook_build.json` | 重开、重算、错误扫描、逐表渲染 |
| 论文与匿名 | `paper/C 题论文.pdf`、`paper/C 题论文.docx` | 无目录、正文≤30 页、无身份信息 |
| AI 披露 | 论文参考文献前声明、`paper/AI 工具使用详情.pdf` | 团队人工核对并补录未知模型信息 |
| 支撑材料 | `支撑材料.zip`、`SHA256SUMS.json` | 两文件均小于 20MB、哈希冻结 |
"""
(EVIDENCE / "requirements_coverage.md").write_text(coverage, encoding="utf-8")

provenance_path = EVIDENCE / "result_provenance.csv"
with provenance_path.open("w", encoding="utf-8-sig", newline="") as f:
    w = csv.writer(f)
    w.writerow(["artifact", "metric_or_region", "frozen_source", "field", "generation_rule", "source_sha256"])
    mapping = [
        ("result1.xlsx", "全部", "q1_dispatch.csv"), ("result2.xlsx", "全部", "q2_dispatch.csv"),
        ("result3.xlsx", "全部", "q3_dispatch.csv"), ("result4-2.xlsx", "全部", "q4_2_dispatch.csv"),
        ("result4-3.xlsx", "全部", "q4_3_dispatch.csv"),
    ]
    for artifact, region, name in mapping:
        p = FROZEN / name
        w.writerow([artifact, region, f"frozen/{name}", "统一 dispatch schema", "scripts/build_workbooks.mjs ; 四位小数仅在展示层", digest(p)])
    w.writerow(["论文与图", "实验数字", "experiment_summary.csv", "各费用/能量/求解指标", "scripts/make_charts.py 与 scripts/build_documents.py 直接读取",
digest(FROZEN / "experiment_summary.csv")])

legacy = ROOT / "outputs" / "c_audit_v1" / "audit_summary.json"
if legacy.exists():
    shutil.copy2(legacy, EVIDENCE / "legacy_audit_summary.json")

if __name__ == "__main__":
    main()
```

scripts/package_final.py

```
#!/usr/bin/env python3
"""Build the under-20 MB C-problem support archive and SHA-256 manifest."""
from __future__ import annotations

import hashlib
import json
import lzma
import shutil
import tempfile
import zipfile
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
FINAL = ROOT / "outputs" / "c_final_v1"
MAX_BYTES = 20 * 1024 * 1024
```

```

def sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for block in iter(lambda: f.read(1024 * 1024), b''):
            h.update(block)
    return h.hexdigest()

def copy_if_exists(src: Path, dst: Path) -> None:
    if not src.exists():
        return
    dst.parent.mkdir(parents=True, exist_ok=True)
    if src.is_dir():
        shutil.copytree(
            src,
            dst,
            dirs_exist_ok=True,
            ignore=shutil.ignore_patterns("__pycache__", "*.pyc", ".DS_Store"),
        )
    else:
        shutil.copy2(src, dst)

def main() -> None:
    required = [
        FINAL / "paper" / "C 题论文.pdf",
        FINAL / "paper" / "C 题论文.docx",
        FINAL / "paper" / "AI 工具使用详情.pdf",
        *[FINAL / "workbooks" / n for n in ("result1.xlsx", "result2.xlsx", "result3.xlsx", "result4-2.xlsx", "result4-3.xlsx")],
        FINAL / "audit" / "audit_summary.json",
        FINAL / "frozen" / "frozen_manifest.json",
    ]
    missing = [str(p) for p in required if not p.exists()]
    if missing:
        raise FileNotFoundError("Cannot package; missing:\n" + "\n".join(missing))

    out_zip = FINAL / "支撑材料.zip"
    with tempfile.TemporaryDirectory(prefix="cumcm-support-") as td:
        stage = Path(td) / "支撑材料"
        stage.mkdir(parents=True)
        copy_if_exists(ROOT / "config", stage / "config")
        copy_if_exists(ROOT / "src", stage / "src")
        copy_if_exists(ROOT / "scripts", stage / "scripts")
        copy_if_exists(ROOT / "run_pipeline.sh", stage / "run_pipeline.sh")
        copy_if_exists(ROOT / "requirements.txt", stage / "requirements.txt")

        copy_if_exists(FINAL / "paper" / "AI 工具使用详情.pdf", stage / "AI 工具使用详情.pdf")
        copy_if_exists(FINAL / "audit", stage / "audit")
        copy_if_exists(FINAL / "evidence", stage / "evidence")
        copy_if_exists(FINAL / "charts", stage / "charts")
        copy_if_exists(FINAL / "workbooks", stage / "workbooks")

        frozen_dst = stage / "frozen"
        frozen_dst.mkdir()
        # The combined ledger is the canonical interface. The two per-question
        # ledgers are byte-for-byte subsets and are intentionally omitted from
        # the submission archive to avoid packaging large duplicate CSV data.
        duplicate_csv = {"q3_decision_versions.csv", "q4_3_decision_versions.csv"}
        for p in sorted((FINAL / "frozen").glob("**")):
            if p.name in duplicate_csv:
                continue
            if p.suffix == ".csv":
                with p.open("rb") as src, lzma.open(frozen_dst / f"{p.name}.xz", "wb", preset=9) as dst:
                    shutil.copyfileobj(src, dst)
            elif p.suffix in {".json", ".md"}:
                shutil.copy2(p, frozen_dst / p.name)
        (frozen_dst / "README.md").write_text(
            """# 冻结结果说明\n\n
            "CSV 以 XZ 保存。`decision_versions.csv.xz` 是完整合并账本；"
            "为避免重复占用提交空间，未另收录其两个子集"
            "`q3_decision_versions.csv` 与 `q4_3_decision_versions.csv`。"
            encoding="utf-8",
        )

    internal_manifest = []
    for p in sorted(x for x in stage.rglob("**") if x.is_file()):
        internal_manifest.append(
            "path": p.relative_to(stage).as_posix(),

```

```

        "bytes": p.stat().st_size,
        "sha256": sha256(p),
    })
    (stage / "SHA256SUMS.json").write_text(
        json.dumps({"generated_at": datetime.now(timezone.utc).isoformat(), "files": internal_manifest}, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )

    out_zip.parent.mkdir(parents=True, exist_ok=True)
    with zipfile.ZipFile(out_zip, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9) as zf:
        for p in sorted(x for x in stage.rglob(".*") if x.is_file()):
            zf.write(p, p.relative_to(stage).as_posix())

    if out_zip.stat().st_size > MAX_BYTES:
        raise RuntimeError(f"Support ZIP is {out_zip.stat().st_size / 1024 / 1024:.2f} MB; limit is 20 MB")

    manifest_entries = []
    for p in sorted(x for x in FINAL.rglob(".*") if x.is_file() and x.name != "SHA256SUMS.json"):
        if "qa" in p.relative_to(FINAL).parts:
            continue
        manifest_entries.append({"path": p.relative_to(FINAL).as_posix(), "bytes": p.stat().st_size, "sha256": sha256(p)})
    manifest = {
        "generated_at": datetime.now(timezone.utc).isoformat(),
        "support_zip_bytes": out_zip.stat().st_size,
        "support_zip_under_20mb": out_zip.stat().st_size <= MAX_BYTES,
        "paper_pdf_bytes": (FINAL / "paper" / "C 题论文.pdf").stat().st_size,
        "paper_pdf_under_20mb": (FINAL / "paper" / "C 题论文.pdf").stat().st_size <= MAX_BYTES,
        "files": manifest_entries,
    }
    (FINAL / "SHA256SUMS.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps({"support_zip": str(out_zip), "bytes": out_zip.stat().st_size, "files": len(internal_manifest)}, ensure_ascii=False))

if __name__ == "__main__":
    main()

```

scripts/qa_final.py

```

#!/usr/bin/env python3
"""Final cross-artifact checks for placeholders, anonymity, size and package integrity."""
from __future__ import annotations

```

```

import json
import re
import zipfile
from pathlib import Path

```

```

from docx import Document
from pypdf import PdfReader

```

```

ROOT = Path(__file__).resolve().parents[1]
FINAL = ROOT / "outputs" / "c_final_v1"
LIMIT = 20 * 1024 * 1024

```

```

def docx_text(path: Path) -> str:
    doc = Document(path)
    parts = [p.text for p in doc.paragraphs]
    for table in doc.tables:
        parts.extend(cell.text for row in table.rows for cell in row.cells)
    return "\n".join(parts)

```

```

def pdf_text(path: Path) -> str:
    return "\n".join((p.extract_text() or "") for p in PdfReader(path).pages)

```

```

def main() -> None:

```

```

paper_docx = FINAL / "paper" / "C 题论文.docx"
paper_pdf = FINAL / "paper" / "C 题论文.pdf"
ai_pdf = FINAL / "paper" / "AI 工具使用详情.pdf"
support = FINAL / "支撑材料.zip"
texts = {
    str(paper_docx): docx_text(paper_docx),
    str(paper_pdf): pdf_text(paper_pdf),
    str(ai_pdf): pdf_text(ai_pdf),
}
local_home = str(Path.home())
local_user = Path.home().name

prohibited = [re.escape(local_home), re.escape(local_user), r"\{(?:(?:RESULT|AI_[EVIDENCE]):", r"待补录"]
findings = []
for path, text in texts.items():
    for pattern in prohibited:
        if re.search(pattern, text, re.I): findings.append({"path": path, "pattern": pattern})
paper_text = texts[str(paper_pdf)]
paper_reader = PdfReader(paper_pdf)
page_texts = [(page.extract_text() or "") for page in paper_reader.pages]
appendix_pages = [i + 1 for i, text in enumerate(page_texts) if re.search(r"^(^|\n)附录(?:\s|$)", text)]
body_pages = (appendix_pages[0] - 1) if appendix_pages else len(page_texts)
ai_pos = paper_text.find("AI 工具使用声明"); ref_pos = paper_text.find("参考文献")
assertions = {
    "paper_pdf_under_20mb": paper_pdf.stat().st_size <= LIMIT,
    "support_zip_under_20mb": support.stat().st_size <= LIMIT,
    "paper_docx_exists": paper_docx.is_file(),
    "five_workbooks": len(list((FINAL / "workbooks").glob("result*.xlsx"))) == 5,
    "ai_statement_before_references": 0 <= ai_pos < ref_pos,
    "no_table_of_contents_heading": "目录" not in paper_text[:2000],
    "paper_body_at_most_30_pages": body_pages <= 30,
    "no_prohibited_text": not findings,
}
with zipfile.ZipFile(support) as zf:
    names = zf.namelist()
    assertions["ai_detail_in_support"] = "AI 工具使用详情.pdf" in names
    assertions["pipeline_in_support"] = "run_pipeline.sh" in names
    assertions["support_paths_relative"] = all(not n.startswith(("/", "../")) and ":" not in n for n in names)
    assertions["support_test_ok"] = zf.testzip() is None
    support_identity_hits = []
    for name in names:
        if Path(name).suffix.lower() not in {".py", ".md", ".json", ".csv", ".yaml", ".sh", ".txt"}:
            continue
        try:
            text = zf.read(name).decode("utf-8", errors="ignore")
        except KeyError:
            continue
        if local_home in text or re.search(rf"(?![\w-])\{re.escape(local_user)\}(?![\w-])", text, re.I):
            support_identity_hits.append(name)
    assertions["support_content_anonymous"] = not support_identity_hits
    findings.extend({"path": f"support:{name}", "pattern": "local identity path"} for name in support_identity_hits)
    audit = json.loads((FINAL / "audit" / "audit_summary.json").read_text(encoding="utf-8"))
    assertions["formal_audit_pass"] = audit.get("formal_delivery_gate") == "PASS"
    result = {"status": "PASS" if all(assertions.values()) else "FAIL", "assertions": assertions, "findings": findings,
             "paper_pages": len(paper_reader.pages), "paper_body_pages": body_pages,
             "appendix_start_page": appendix_pages[0] if appendix_pages else None,
             "paper_pdf_bytes": paper_pdf.stat().st_size,
             "support_zip_bytes": support.stat().st_size}
    out = FINAL / "qa" / "final_verification.json"; out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text(json.dumps(result, ensure_ascii=False, indent=2), encoding="utf-8")
    print(json.dumps(result, ensure_ascii=False))
    if result["status"] != "PASS": raise SystemExit(1)

```

```

if __name__ == "__main__":
    main()

```

scripts/run_model.py

```

#!/usr/bin/env python3
from src.model.cli import main

```

```

if __name__ == "__main__":
    raise SystemExit(main())

```

scripts/smoke_model.py

```

#!/usr/bin/env python3
from datetime import date

```

```

from src.model import load_config, load_data
from src.model.Experiments import solve_q1
from src.model.rolling import RunSpec, simulate
from src.model.forecast import corrected_pv_horizon, pv_cap_horizon, residual_scenarios

```

```

def main() -> int:
    cfg=load_config("config/final.yaml")
    data=load_data(cfg)
    assert len(data.dates)==365 and data.load_kw.shape==(365,144)
    q1=solve_q1(data,cfg)
    assert len(q1)==144 and abs(q1[-1]["soc_end_kwh"]-6000)<1e-7
    cfg["forecast"]["scenario_count"]=3
    cfg["forecast"]["scenario_history_days"]=3
    result=simulate(data,cfg,RunSpec("q3","S3",all_three,start_date=date(2025,2,1),end_date=date(2025,2,1)))
    assert len(result["dispatch"])==144
    assert result["summary"]["max_balance_residual_kwh"]<1e-7
    assert result["summary"]["max_soc_residual_kwh"]<1e-7
    assert all(v["new_commitment_kwh"]==" for v in result["decision_versions"] if v["is_preview_recourse"])
    assert all(date.fromisoformat(h)<date.fromisoformat(v["date"])
        for v in result["decision_versions"] for field in
            ("load_history_dates","price_history_dates","pv_bias_history_dates","residual_scenario_dates")
            for h in str(v[field]).split("|") if h)
    test_day=date(2025,2,1); base_=corrected_pv_horizon(data,test_day,0,144,cfg)
    scenarios_=residual_scenarios(data,test_day,0,base,cfg,centered=True); caps=pv_cap_horizon(data,test_day,0,144,cfg)
    assert (scenarios<=caps[None,:]+1e-10).all()
    chain={}
    versions=[v for v in result["decision_versions"] if not v["is_preview_next_day"]]
    previews=[v for v in result["decision_versions"] if v["is_preview_next_day"]]
    assert len(previews)==216 and all(v["preview_grid_kwh"]==" and v["new_commitment_kwh"]==" for v in previews)
    for hour,expected in ((0,144),(6,108),(12,72),(18,36)):
        block=[v for v in versions if v["decision_time"].endswith(f" {hour:02d}:00")]
        assert sum(v["sensitivity_new_commitment_kwh"]!=" for v in block)==expected
        for v in block:
            pos=int(v["position"])
            if hour==0:
                assert v["sensitivity_old_commitment_kwh"]=="
            else:
                assert abs(float(v["sensitivity_old_commitment_kwh"])-chain[pos])<1e-8
                chain[pos]=float(v["sensitivity_new_commitment_kwh"])
    print("smoke_model: PASS")
    return 0

```

```

if __name__=="__main__": raise SystemExit(main())

```

scripts/smoke_parallel_model.py

```

#!/usr/bin/env python3
from concurrent.futures import ProcessPoolExecutor
from datetime import date

```

```

from src.model import load_config, load_data
from src.model.rolling import RunSpec, simulate

```

```

def worker(data,cfg,mode):
    cfg["forecast"]["scenario_count"]=3; cfg["forecast"]["scenario_history_days"]=3
    return simulate(data,cfg,RunSpec(mode,"S2",all_three,start_date=date(2025,2,1),end_date=date(2025,2,2),collect_detail=False))["summary"]

```

```

def main()->int:
    cfg=load_config("config/final.yaml"); data=load_data(cfg)
    with ProcessPoolExecutor(max_workers=2) as pool:
        rows=[f.result() for f in (pool.submit(worker,data,cfg,"q3"),pool.submit(worker,data,cfg,"q4_3"))]
    assert all(r["days"]==2 and r["constraint_violation_count"]==0 for r in rows)
    print("smoke_parallel_model: PASS")
    return 0

```

```

if __name__=="__main__": raise SystemExit(main())

```

scripts/validate_model_outputs.py

```
#!/usr/bin/env python3
from __future__ import annotations

import csv
import argparse
import hashlib
import json
from pathlib import Path

from src.model import load_config

NUMERIC = (
    "plan_00_grid_kwh", "final_adjusted_grid_kwh", "charge_kwh", "discharge_kwh",
    "soc_start_kwh", "soc_end_kwh", "planned_unused_nonpv_kwh", "actual_load_kwh", "actual_pv_kwh",
    "settlement_price", "emergency_kwh", "unused_surplus_kwh", "plan_cost_yuan",
    "adjustment_cost_yuan", "emergency_cost_yuan", "final_relative_cost_yuan", "sequential_cost_yuan",
)

def sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for block in iter(lambda: f.read(1 < 20), b''): h.update(block)
    return h.hexdigest()

def main() -> int:
    ap = argparse.ArgumentParser(description="Validate frozen causal-model outputs and stability evidence")
    ap.add_argument("--config", default="config/final.yaml")
    args = ap.parse_args()
    cfg = load_config(args.config); frozen = Path(cfg["_output_root"])/"frozen"; evidence = Path(cfg["_output_root"])/"evidence"
    tol = 1e-7; checks = []
    def add(name: str, ok: bool, actual): checks.append({'check': name, 'status': "PASS" if ok else "FAIL", 'actual': actual})
    for name, expected in (('q1', 144), ('q2', 48096), ('q3', 48096), ('q4_2', 48096), ('q4_3', 48096)):
        path = frozen/f"{name}_dispatch.csv"; rows = []
        with path.open(encoding="utf-8-sig", newline='') as f:
            for row in csv.DictReader(f):
                for col in NUMERIC:
                    if row[col] == '': raise AssertionError(f'{name}: blank numeric {col}')
                    float(row[col])
                rows.append(row)
        add(f'{name}.row_count', len(rows) == expected, len(rows))
        by = {}
        for r in rows: by.setdefault(r["date"], []).append(r)
        add(f'{name}.daily_144_unique', all(len(x) == 144 and {int(r["position"]) for r in x} == set(range(1, 145)) for x in by.values()), len(by))
        max_soc = max(abs(float(r["soc_end_kwh"]) - float(r["soc_start_kwh"])), float(cfg["storage"]["charge_efficiency"])*float(r["charge_kwh"])+float(r["discharge_kwh"])/
float(cfg["storage"]["discharge_efficiency"]))) for r in rows)
        max_bal = max(abs(float(r["final_adjusted_grid_kwh"])+float(r["actual_pv_kwh"])+float(r["discharge_kwh"])+float(r["emergency_kwh"])-float(r["actual_load_kwh"])-
float(r["charge_kwh"])-float(r["unused_surplus_kwh"]))) for r in rows)
        max_cost = max(abs(float(r["plan_cost_yuan"])+float(r["adjustment_cost_yuan"])+float(r["emergency_cost_yuan"])-float(r["final_relative_cost_yuan"]))) for r in rows)
        continuity = max((abs(float(rows[i]["soc_start_kwh"])-float(rows[i-1]["soc_end_kwh"]))) for i in range(1, len(rows))), default=0.0)
        simultaneous = sum(float(r["charge_kwh"])>tol and float(r["discharge_kwh"])>tol for r in rows)
        soc_bounds = sum(float(r["soc_end_kwh"])-float(cfg["storage"]["soc_min_kwh"])-tol or float(r["soc_end_kwh"])>float(cfg["storage"]["soc_max_kwh"])+tol for r in
rows)
        curtail_bounds = sum(float(r["curtail_pv_kwh"])>float(r["forecast_pv_kwh"])+tol for r in rows)
        add(f'{name}.soc_recursion', max_soc < tol, max_soc); add(f'{name}.actual_balance', max_bal < tol, max_bal)
        add(f'{name}.cost_reconcile', max_cost < tol, max_cost); add(f'{name}.soc_continuity', continuity < tol, continuity)
        add(f'{name}.no_simultaneous_charge_discharge', simultaneous == 0, simultaneous); add(f'{name}.soc_bounds', soc_bounds == 0, soc_bounds)
        add(f'{name}.curtail_within_forecast_pv', curtail_bounds == 0, curtail_bounds)
        add(f'{name}.endpoints', abs(float(rows[0]["soc_start_kwh"])-6000) < tol and abs(float(rows[-1]["soc_end_kwh"])-6000) < tol, [rows[0]["soc_start_kwh"], rows[-1]
["soc_end_kwh"]])
        if name in {"q3", "q4_3"}:
            delta = sum(float(r["sequential_cost_yuan"])-float(r["final_relative_cost_yuan"]) for r in rows)
            add(f'{name}.settlement_sensitivity_distinct', abs(delta) > .01, delta)
    for name in ("q3", "q4_3"):
        plans = {}
        with (frozen/f"{name}_dispatch.csv").open(encoding="utf-8-sig", newline='') as f:
            for r in csv.DictReader(f): plans[(r["date"], int(r["position"]))] = float(r["plan_00_grid_kwh"])
        path = frozen/f"{name}_decision_versions.csv"; bad_history = 0; bad_preview = 0; bad_chain = 0; count = 0; chain = {}; per_day = {}; previews = {}
        with path.open(encoding="utf-8-sig", newline='') as f:
            for r in csv.DictReader(f):
                count += 1
                for col in ("load_history_dates", "price_history_dates", "pv_bias_history_dates", "residual_scenario_dates"):
                    bad_history += sum(h == r["date"] for h in r[col].split(",") if h)
                if r["is_preview_recourse"] == "1" and (r["old_commitment_kwh"] != "" or r["new_commitment_kwh"] != ""): bad_preview += 1
                day = r["date"]; hour = int(r["decision_time"])[11:13]; pos = int(r["position"])
                per_day.setdefault(day, {0: 0, 6: 0, 12: 0, 18: 0})
                if r["is_preview_next_day"] == "1":
                    previews.setdefault(day, {0: 0, 6: 0, 12: 0, 18: 0})[hour] += 1
                if (r["sensitivity_old_commitment_kwh"] != "" or r["sensitivity_new_commitment_kwh"] != "")
```

```

        or r.get("preview_grid_kwh","")==" or r.get("target_date","")<=day): bad_chain+=1
    continue
    if r.get("target_date",day)!=day: bad_chain+=1
    per_day[day][hour]+=int(r["sensitivity_new_commitment_kwh"]!="")
    key=(day,pos); new=float(r["sensitivity_new_commitment_kwh"])
    if hour==0:
        if r["sensitivity_old_commitment_kwh"]!=" or abs(new-plans[key])>tol: bad_chain+=1
    else:
        if key not in chain or abs(float(r["sensitivity_old_commitment_kwh"])-chain[key])>tol: bad_chain+=1
        chain[key]=new
    add(f"{name}.causal_history",bad_history==0,bad_history); add(f"{name}.recourse_not_committed",bad_preview==0,bad_preview);
add(f"{name}.version_rows",count>0,count)
bad_counts=sum(v!={0:144,6:108,12:72,18:36} for v in per_day.values())
bad_preview_counts=sum(v!={0:0,6:36,12:72,18:108} for d,v in previews.items() if d!="2025-12-31")
bad_preview_counts+=sum(d!="2025-12-31" and d not in previews for d in per_day)
add(f"{name}.sensitivity_chain",bad_chain==0,bad_chain); add(f"{name}.sensitivity_counts",bad_counts==0,bad_counts)
add(f"{name}.full_next_day_preview",bad_preview_counts==0,bad_preview_counts)
stability_path=frozen/"stability_summary.csv"; stability=[]
with stability_path.open(encoding="utf-8-sig",newline="") as f: stability=list(csv.DictReader(f))
expected={(m,s.method,float(tb)) for m in ("q3","q4_3") for s in ("S2","S3")}
    for method in cfg["experiments"][["solver_methods"]] for tb in cfg["experiments"][["tiebreak_coefficients"]]
actual={(r["mode"],r["scenario"],r["solver_method"],float(r["tiebreak"])) for r in stability}
add("stability.full_factorial",len(stability)==36 and actual==expected,{"rows":len(stability),"unique":len(actual)})
stability_bad=sum(int(r["days"])!=334 or abs(float(r["initial_soc_kwh"])-6000)>tol
    or abs(float(r["terminal_soc_kwh"])-6000)>tol or int(r["constraint_violation_count"])!=0
    for r in stability)
add("stability.annual_feasible",stability_bad==0,stability_bad)
with (frozen/"stability_assessment.csv").open(encoding="utf-8-sig",newline="") as f: assessment=list(csv.DictReader(f))
assessment_keys={(r["mode"],r["settlement"],r["comparison"]) for r in assessment}
expected_assessment={(m,s,"S3_minus_S2") for m in ("q3","q4_3")}
    for s in ("final_relative_to_00_plan","sequential_sensitivity")
add("stability.two_settlements",len(assessment)==4 and assessment_keys==expected_assessment,
    {"rows":len(assessment),"keys":sorted(assessment_keys)})
manifest=json.loads((frozen/"frozen_manifest.json").read_text(encoding="utf-8")); mismatch=[]
for item in manifest["files"]:
    p=frozen/item["path"]
    if not p.is_file() or sha256(p)!=item["sha256"]: mismatch.append(item["path"])
add("manifest.hashes",not mismatch,mismatch)
leaks=[]
home_marker=str(Path.home())
for p in frozen.glob("*.json"):
    if home_marker in p.read_text(encoding="utf-8"): leaks.append(p.name)
add("portable_metadata",not leaks,leaks)
result={"status":"PASS" if all(x["status"]=="PASS" for x in checks) else "FAIL","checks":checks}
evidence.mkdir(parents=True,exist_ok=True); (evidence/"model_integrity_check.json").write_text(json.dumps(result,ensure_ascii=False,indent=2),encoding="utf-8")
print(json.dumps({"status":result["status"],"checks":len(checks)},ensure_ascii=False))
return 0 if result["status"]=="PASS" else 1

```

```

if __name__=="__main__": raise SystemExit(main())

```

scripts/build_workbooks.mjs

```

import fs from "node:fs/promises";
import path from "node:path";
import { FileBlob, SpreadsheetFile, Workbook } from "@oai/artifact-tool";

```

```

const projectRoot = path.resolve(path.dirname(new URL(import.meta.url).pathname), "..");
const finalRoot = path.join(projectRoot, "outputs", "c_final_v1");
const frozenRoot = path.join(finalRoot, "frozen");
const workbookRoot = path.join(finalRoot, "workbooks");
const qaRoot = path.join(finalRoot, "qa", "workbook_renderers");

```

```

function parseCsv(text) {
    const rows = [];
    let row = [], value = "", quoted = false;
    for (let i = 0; i < text.length; i++) {
        const ch = text[i];
        if (quoted) {
            if (ch === '"' && text[i + 1] === '"') { value += '"'; i++; }
            else if (ch === '"') quoted = false;
            else value += ch;
        } else if (ch === '"') quoted = true;
        else if (ch === ',') { row.push(value); value = ""; }
        else if (ch === '\n') { row.push(value.replace(/r$/, "")); rows.push(row); row = []; value = ""; }
        else value += ch;
    }
    if (value.length || row.length) { row.push(value.replace(/r$/, "")); rows.push(row); }
    if (rows.length && rows[0][0]?.charCodeAt(0) === 0xfeff) rows[0][0] = rows[0][0].slice(1);
    const headers = rows.shift();
    return rows.filter(r => r.some(v => v !== "")).map(r => Object.fromEntries(headers.map((h, i) => [h, r[i] ?? ""])));
}

```

```

async function readCsv(name) {

```

```

    return parseCsv(await fs.readFile(path.join(frozenRoot, name), "utf8"));
  }

function n(v) {
  const x = Number(v);
  if (!Number.isFinite(x)) throw new Error(`Expected numeric value, got ${JSON.stringify(v)}`);
  return x;
}

function roundHalfUp(v, places = 4) {
  // Excel-facing values must follow the audit contract's decimal ROUND_HALF_UP
  // rule. Number.toFixed() rounds the already-binary IEEE-754 value and gives
  // the wrong result at exact decimal half units such as 2238.56265.
  const source = typeof v === "string" ? v.trim() : String(n(v));
  const match = source.match(/^([+-]?)(\d+)(?:(\.\d*))?(?:[eE]([+-]?d+))?$$/);
  if (!match) throw new Error(`Expected decimal value, got ${JSON.stringify(v)}`);
  const negative = match[1] === "-";
  const fraction = match[3] ?? "";
  const exponent = Number(match[4] ?? 0);
  let magnitude = BigInt(` ${match[2]}${fraction}` || "0");
  const scale = fraction.length - exponent;
  if (scale <= places) {
    magnitude *= 10n ** BigInt(places - scale);
  } else {
    const divisor = 10n ** BigInt(scale - places);
    const remainder = magnitude % divisor;
    magnitude /= divisor;
    if (remainder * 2n >= divisor) magnitude += 1n;
  }
  const signed = negative ? -magnitude : magnitude;
  return Number(signed) / 10 ** places;
}

function rounded4(v) { return roundHalfUp(v, 4); }

function decimalParts(v) {
  const source = typeof v === "string" ? v.trim() : String(n(v));
  const match = source.match(/^([+-]?)(\d+)(?:(\.\d*))?(?:[eE]([+-]?d+))?$$/);
  if (!match) throw new Error(`Expected decimal value, got ${JSON.stringify(v)}`);
  const fraction = match[3] ?? "";
  const exponent = Number(match[4] ?? 0);
  let magnitude = BigInt(` ${match[2]}${fraction}` || "0");
  let scale = fraction.length - exponent;
  if (scale < 0) {
    magnitude *= 10n ** BigInt(-scale);
    scale = 0;
  }
  if (match[1] === "-") magnitude = -magnitude;
  return { magnitude, scale };
}

function decimalSum(values) {
  const parts = values.map(decimalParts);
  const scale = Math.max(0, ...parts.map(p => p.scale));
  const total = parts.reduce(
    (acc, p) => acc + p.magnitude * 10n ** BigInt(scale - p.scale),
    0n,
  );
  const negative = total < 0n;
  const digits = (negative ? -total : total).toString().padStart(scale + 1, "0");
  if (scale === 0) return `${negative ? "-" : ""}${digits}`;
  return `${negative ? "-" : ""}${digits.slice(0, -scale)}${digits.slice(-scale)}`;
}

function excelDate(s) { return new Date(` ${s}T00:00:00Z`); }

function officialLabel(position) {
  const start = position * 10;
  const end = (position + 1) * 10;
  const fmt = (mins) => {
    if (mins >= 1440) return `${Math.floor((mins - 1440) / 60)}:${String((mins - 1440) % 60).padStart(2, "0")+1}`;
    return `${Math.floor(mins / 60)}:${String(mins % 60).padStart(2, "0")}`;
  };
  return `${fmt(start)}-${fmt(end)}`;
}

function physicalLabel(startPosition, endPositionExclusive) {
  const fmt = (mins) => `${Math.floor(mins / 60)}:${String(mins % 60).padStart(2, "0")}`;
  return `${fmt((startPosition - 1) * 10)}-${fmt(endPositionExclusive * 10)}`;
}

function groupByDate(rows) {
  const out = new Map();
  for (const row of rows) {

```

```

    if (!out.has(row.date)) out.set(row.date, []);
    out.get(row.date).push(row);
  }
  for (const rs of out.values()) rs.sort((a, b) => n(a.position) - n(b.position));
  return out;
}

function validateDispatch(rows, name, expectedDays) {
  if (rows.length !== expectedDays * 144) throw new Error(`${name}: expected ${expectedDays * 144} rows, got ${rows.length}`);
  const byDate = groupByDate(rows);
  if (byDate.size !== expectedDays) throw new Error(`${name}: expected ${expectedDays} dates, got ${byDate.size}`);
  const referenceLabels = [...byDate.values()][0].map(r => r.official_label);
  if (referenceLabels.some(v => !v) || new Set(referenceLabels).size !== 144) {
    throw new Error(`${name}: official labels must contain 144 nonblank unique values`);
  }
  for (const [date, rs] of byDate) {
    if (rs.length !== 144) throw new Error(`${name} ${date}: expected 144 rows, got ${rs.length}`);
    for (let i = 0; i < 144; i++) {
      if (n(rs[i].position) !== i + 1) throw new Error(`${name} ${date}: bad position at ${i + 1}`);
      if (rs[i].official_label !== referenceLabels[i]) throw new Error(`${name} ${date}: official label mismatch at ${i + 1}`);
    }
  }
  return byDate;
}

function styleSheet(sheet, usedRange, numericRanges = []) {
  // Keep the template's cell boundaries visible. Several adjacent fields
  // (date/time and charge/discharge/time) otherwise read as one value in the
  // rendered workbook even though the underlying cells are correct.
  sheet.showGridLines = true;
  usedRange.format.font = { name: "Arial", size: 10, color: "#111827" };
  usedRange.format.verticalAlignment = "center";
  const header = usedRange.getRow(0);
  header.format.fill = "#1F4E78";
  header.format.font = { name: "Arial", size: 10, bold: true, color: "#FFFFFF" };
  header.format.horizontalAlignment = "center";
  header.format.wrapText = true;
  header.format.rowHeight = 30;
  for (const range of numericRanges) range.format.numberFormat = "0.0000";
}

function addPlanSheet(wb, name, byDate, valueField, costField) {
  const sheet = wb.worksheets.add(name);
  const labels = [...byDate.values()][0].map(r => r.official_label);
  const matrix = [["日期\\时间", ...labels, "全天购电量", "全天购电费"]];
  for (const [date, rows] of byDate) {
    matrix.push([
      excelDate(date),
      ...rows.map(r => rounded4(r[valueField])),
      rounded4(decimalSum(rows.map(r => r[valueField]))),
      rounded4(rows.reduce((s, r) => s + n(r[costField]), 0)),
    ]);
  }
  const used = sheet.getRangeByIndexes(0, 0, matrix.length, matrix[0].length);
  used.values = matrix;
  styleSheet(sheet, used, [sheet.getRangeByIndexes(1, 1, matrix.length - 1, matrix[0].length - 1)]);
  sheet.getRangeByIndexes(1, 0, matrix.length - 1, 1).format.numberFormat = "yyyy-mm-dd";
  sheet.freezePanes.freezeRows(1);
  sheet.freezePanes.freezeColumns(1);
  sheet.getRange(`A1:A${matrix.length}`).format.columnWidth = 12;
  sheet.getRange(`B1:EO${matrix.length}`).format.columnWidth = 10;
  sheet.getRange(`EP1:EQ${matrix.length}`).format.columnWidth = 15;
  return sheet;
}

function addQ1Sheets(wb, rows) {
  const plan = wb.worksheets.add("计划购电量");
  // result1's official template uses a different final label spelling from
  // the annual templates; preserve that literal template label.
  const planMatrix = [["时间段", "购电量", ...rows.map((r, i) => [
    officialLabel(i + 1),
    rounded4(r.plan_00_grid_kwh),
  ])]];
  const pUsed = plan.getRangeByIndexes(0, 0, planMatrix.length, 2);
  pUsed.values = planMatrix;
  styleSheet(plan, pUsed, [plan.getRange(`B2:B${planMatrix.length}`)]);
  plan.freezePanes.freezeRows(1);
  plan.getRange(`A1:A${planMatrix.length}`).format.columnWidth = 19;
  plan.getRange(`B1:B${planMatrix.length}`).format.columnWidth = 14;

  const charge = wb.worksheets.add("充放电电量");
  const cm = [["时间段", "充电量", "放电量", "时刻", "储电量"]];
  for (let block = 0; block < 6; block++) {
    const rs = rows.slice(block * 24, (block + 1) * 24);
    cm.push([

```

```

        `${block * 4}:00-${(block + 1) * 4}:00`,
        rounded4(decimalSum(rs.map(r => r.charge_kwh))),
        rounded4(decimalSum(rs.map(r => r.discharge_kwh))),
        block === 0 ? "0:00" : block === 1 ? "24:00" : null,
        block === 0 ? rounded4(rows[0].soc_start_kwh) : block === 1 ? rounded4(rows[143].soc_end_kwh) : null,
    ]);
}
const cUsed = charge.getRangeByIndexes(0, 0, cm.length, 5);
cUsed.values = cm;
styleSheet(charge, cUsed, [charge.getRange("B2:C7"), charge.getRange("E2:E3")]);
charge.getRange(`A1:E${cm.length}`).format.columnWidth = 18;
}

function addChargeSheet(wb, byDate) {
    const sheet = wb.worksheets.add("充放电电量");
    const matrix = [["日期", "时间段", "充电量", "放电量", "时刻", "储电量"]];
    for (const [date, rows] of byDate) {
        for (let block = 0; block < 6; block++) {
            const rs = rows.slice(block * 24, (block + 1) * 24);
            matrix.push([
                block === 0 ? excelDate(date) : null,
                `${block * 4}:00-${(block + 1) * 4}:00`,
                rounded4(decimalSum(rs.map(r => r.charge_kwh))),
                rounded4(decimalSum(rs.map(r => r.discharge_kwh))),
                block === 0 ? "0:00" : block === 1 ? "24:00" : null,
                block === 0 ? rounded4(rows[0].soc_start_kwh) : block === 1 ? rounded4(rows[143].soc_end_kwh) : null,
            ]);
        }
    }
    const used = sheet.getRangeByIndexes(0, 0, matrix.length, 6);
    used.values = matrix;
    styleSheet(sheet, used, [sheet.getRangeByIndexes(1, 2, matrix.length - 1, 2), sheet.getRangeByIndexes(1, 5, matrix.length - 1, 1)]);
    sheet.getRangeByIndexes(1, 0, matrix.length - 1, 1).format.numberFormat = "yyyy-mm-dd";
    sheet.freezePanes.freezeRows(1);
    sheet.getRange(`A1:A${matrix.length}`).format.columnWidth = 13;
    sheet.getRange(`B1:B${matrix.length}`).format.columnWidth = 16;
    sheet.getRange(`C1:F${matrix.length}`).format.columnWidth = 14;
}

function addEmergencySheet(wb, byDate, threshold) {
    const sheet = wb.worksheets.add("紧急购电量");
    const matrix = [["日期", "购电时间段", "购电量"]];
    for (const [date, rows] of byDate) {
        const groups = [];
        let current = null;
        for (const r of rows) {
            const pos = n(r.position), q = n(r.emergency_kwh);
            // The configured half-unit threshold implements ROUND_HALF_UP at four decimals.
            const shown = Math.abs(q) >= threshold;
            if (!shown) { if (current) { groups.push(current); current = null; } continue; }
            if (current) current = { start: pos, end: pos, amounts: [r.emergency_kwh] };
            else if (pos === current.end + 1) { current.end = pos; current.amounts.push(r.emergency_kwh); }
            else { groups.push(current); current = { start: pos, end: pos, amounts: [r.emergency_kwh] }; }
        }
        if (current) groups.push(current);
        groups.forEach((g, i) => matrix.push([i === 0 ? excelDate(date) : null, physicalLabel(g.start, g.end), rounded4(decimalSum(g.amounts))]));
    }
    const used = sheet.getRangeByIndexes(0, 0, matrix.length, 3);
    used.values = matrix;
    styleSheet(sheet, used, matrix.length > 1 ? [sheet.getRangeByIndexes(1, 2, matrix.length - 1, 1)] : []);
    if (matrix.length > 1) sheet.getRangeByIndexes(1, 0, matrix.length - 1, 1).format.numberFormat = "yyyy-mm-dd";
    sheet.freezePanes.freezeRows(1);
    sheet.getRange(`A1:A${matrix.length}`).format.columnWidth = 13;
    sheet.getRange(`B1:B${matrix.length}`).format.columnWidth = 18;
    sheet.getRange(`C1:C${matrix.length}`).format.columnWidth = 14;
}

async function exportAndVerify(wb, filename, sheetNames) {
    wb.calculate();
    const file = path.join(workbookRoot, filename);
    const blob = await SpreadsheetFile.exportXlsx(wb);
    await blob.save(file);
    const reopened = await SpreadsheetFile.importXlsx(await FileBlob.load(file));
    const errors = await reopened.inspect({
        kind: "match",
        searchTerm: "#REF!#DIV/0!#VALUE!#NAME?#N/A#NUM!#NULL!#SPILL!#CALC!",
        options: { useRegex: true, maxResults: 100 },
        maxChars: 6000,
        summary: `${filename} formula error scan`,
    });
    const errorLines = (errors.ndjson || "").split("\n").filter(line => /"address"s":/ .test(line));
    if (errorLines.length) throw new Error(`${filename}: formula error detected: ${errorLines.join("\n")}`);
    // Some artifact-tool versions spill inspect output beside large workbooks.
    // It is a transient diagnostic, not part of the official submission.
    await fs.rm(`${file}.inspect.ndjson`, { force: true });
    const renderDir = path.join(qaRoot, filename.replace(/\.\xlsx$/, ""));
    await fs.mkdir(renderDir, { recursive: true });
    for (const sheetName of sheetNames) {

```

```

const sheet = reopened.worksheets.getItem(sheetName);
const used = sheet.getUsedRange(true);
const rowCount = used?.rowCount ?? 1;
const colCount = used?.columnCount ?? 1;
const ranges = [];
if (colCount > 20) {
  ranges.push(`A1:P${Math.min(rowCount, 12)}`);
  ranges.push(`EI:EQ${Math.min(rowCount, 12)}`);
  if (rowCount > 20) ranges.push(`A${Math.max(1, rowCount - 5)}:P${rowCount}`);
} else {
  ranges.push(`A1:${String.fromCharCode(64 + Math.min(colCount, 26))}${Math.min(rowCount, 40)}`);
  if (rowCount > 45) ranges.push(`A${rowCount - 5}:${String.fromCharCode(64 + Math.min(colCount, 26))}${rowCount}`);
}
for (let i = 0; i < ranges.length; i++) {
  const png = await reopened.render({ sheetName, range: ranges[i], scale: 1.5, format: "png" });
  const safe = sheetName.replace(/[\^pL]p[N_]_/gu, "_");
  await fs.writeFile(path.join(renderDir, `${safe}_${i + 1}.png`), new Uint8Array(await png.arrayBuffer()));
}
}
return file;
}

async function main() {
  await fs.mkdir(workbookRoot, { recursive: true });
  await fs.mkdir(qaRoot, { recursive: true });
  const cfg = JSON.parse(await fs.readFile(path.join(projectRoot, "config", "final.yaml"), "utf8"));
  const threshold = Number(cfg.audit.emergency_display_omit_threshold_kwh);

  const specs = [
    ["q1_dispatch.csv", "result1.xlsx", "q1"],
    ["q2_dispatch.csv", "result2.xlsx", "q2"],
    ["q3_dispatch.csv", "result3.xlsx", "q3"],
    ["q4_2_dispatch.csv", "result4-2.xlsx", "q4_2"],
    ["q4_3_dispatch.csv", "result4-3.xlsx", "q4_3"],
  ];
  const created = [];
  for (const [csvName, xlsName, mode] of specs) {
    const rows = await readCsv(csvName);
    const expectedDays = mode === "q1" ? 1 : 334;
    const byDate = validateDispatch(rows, mode, expectedDays);
    const wb = Workbook.create();
    let sheetNames;
    if (mode === "q1") {
      addQ1Sheets(wb, rows);
      sheetNames = ["计划购电量", "充放电量"];
    } else {
      const initialCostField = (mode === "q2" || mode === "q4_2") ? "final_relative_cost_yuan" : "plan_cost_yuan";
      addPlanSheet(wb, "计划购电量", byDate, "plan_00_grid_kwh", initialCostField);
      if (mode === "q3" || mode === "q4_3") addPlanSheet(wb, "调整购电量", byDate, "final_adjusted_grid_kwh", "final_relative_cost_yuan");
      addChargeSheet(wb, byDate);
      addEmergencySheet(wb, byDate, threshold);
      sheetNames = (mode === "q3" || mode === "q4_3")
        ? ["计划购电量", "调整购电量", "充放电量", "紧急购电量"]
        : ["计划购电量", "充放电量", "紧急购电量"];
    }
    created.push(await exportAndVerify(wb, xlsName, sheetNames));
  }
  await fs.writeFile(
    path.join(finalRoot, "qa", "workbook_build.json"),
    JSON.stringify({ created: created.map(file => path.relative(finalRoot, file)), threshold, verified: true }, null, 2),
  );
}

await main();

```

tests/test_audit_final.py

```

from datetime import date
import math
import csv
import tempfile
import unittest

```

```

from src.audit.audit_final import AuditConfig, display_round, emergency_segments, load_config, load_versions, required_number

```

```

def row(position: int, quantity: float):
    return {"date": date(2025, 2, 1), "position": position, "emergency": quantity}

```

```

class AuditFinalTests(unittest.TestCase):
    def test_required_number_never_silently_coerces_missing_or_invalid(self):
        for value in [None, "", " ", "abc", "#REF!", float("nan"), float("inf"), True]:
            with self.subTest(value=value), self.assertRaises(ValueError):
                required_number(value, "test")

    def test_required_number_preserves_real_zero(self):
        self.assertEqual(required_number(0, "test"), 0.0)
        self.assertEqual(required_number("0", "test"), 0.0)

    def test_half_up_display_threshold(self):
        self.assertEqual(display_round(0.000049, 4), 0)
        self.assertNotEqual(display_round(0.00005, 4), 0)
        self.assertNotEqual(display_round(0.0009, 4), 0)

    def test_emergency_segments_filter_before_grouping_and_keep_00009(self):
        cfg = AuditConfig()
        segments = emergency_segments([
            row(1, 1.0), row(2, 0.00004), row(3, 0.0009), row(4, 2.0),
        ], cfg)
        self.assertEqual([(x["start_position"], x["end_position"]) for x in segments], [(1, 1), (3, 4)])
        self.assertTrue(math.isclose(segments[1]["display_kwh"], 2.0009))

    def test_json_as_yaml_config_is_not_silently_defaulted(self):
        from pathlib import Path
        cfg, raw, parser = load_config(Path("config/final.yaml"))
        self.assertEqual(parser, "json")
        self.assertEqual(cfg.initial_soc_kwh, 6000.0)
        self.assertEqual(cfg.storage_power_kw, 5000.0)
        self.assertEqual(cfg.emergency_display_threshold_kwh, 0.00005)
        self.assertIn("emergency_display_omit_threshold_kwh", raw)

    def test_unified_decision_versions_filters_mode_and_preview(self):
        fields = ["date", "mode", "scenario", "model_arm", "decision_time", "forecast_issue_time", "position",
            "old_commitment_kwh", "new_commitment_kwh", "is_preview_next_day", "is_preview_recourse",
            "sensitivity_old_commitment_kwh", "sensitivity_new_commitment_kwh", "decision_basis",
            "load_history_dates", "price_history_dates", "pv_bias_history_dates", "residual_scenario_dates"]
        with tempfile.TemporaryDirectory() as directory:
            path = _import_((pathlib).Path(directory) / "decision_versions.csv")
            with path.open("w", newline="", encoding="utf-8") as handle:
                writer = csv.DictWriter(handle, fieldnames=fields)
                writer.writeheader()
                writer.writerow({"date": "2025-02-01", "mode": "q3", "scenario": "S3", "model_arm": "all_three",
                    "decision_time": "2025-02-01 06:00", "position": 37,
                    "forecast_issue_time": "2025-02-01 06:00",
                    "old_commitment_kwh": 1, "new_commitment_kwh": 2,
                    "sensitivity_old_commitment_kwh": 1, "sensitivity_new_commitment_kwh": 2,
                    "decision_basis": "next_block_stochastic",
                    "is_preview_next_day": 0, "is_preview_recourse": 0,
                    "load_history_dates": "2025-01-25"})
                writer.writerow({"date": "2025-02-01", "mode": "q3", "scenario": "S3", "model_arm": "all_three",
                    "decision_time": "2025-02-01 06:00", "position": 73,
                    "forecast_issue_time": "2025-02-01 06:00",
                    "old_commitment_kwh": "", "new_commitment_kwh": "",
                    "sensitivity_old_commitment_kwh": 5, "sensitivity_new_commitment_kwh": 6,
                    "decision_basis": "deterministic_point_forecast_sensitivity",
                    "is_preview_next_day": 0, "is_preview_recourse": 1})
                writer.writerow({"date": "2025-02-01", "mode": "q4_3", "scenario": "S3", "model_arm": "all_three",
                    "decision_time": "2025-02-01 06:00", "position": 37,
                    "forecast_issue_time": "2025-02-01 06:00",
                    "old_commitment_kwh": 3, "new_commitment_kwh": 4,
                    "sensitivity_old_commitment_kwh": 3, "sensitivity_new_commitment_kwh": 4,
                    "is_preview_next_day": 0, "is_preview_recourse": 0})
            ledgers, selected, scan = load_versions(_import_((pathlib).Path(directory), "result3.xlsx"))
            self.assertEqual(selected, path)
            self.assertEqual(len(ledgers["formal"]), 1)
            self.assertEqual(len(ledgers["sensitivity"]), 2)
            self.assertEqual(ledgers["formal"][0]["decision_hour"], 6)
            self.assertEqual(scan["preview_recourse_rows"], 1)
            self.assertEqual(scan["future_history_references"], 0)
            self.assertEqual(scan["forecast_release"]["decision_time_mismatches"], 0)

    def test_decision_version_forecast_release_must_match_decision_and_attachment(self):
        fields = ["date", "mode", "scenario", "model_arm", "decision_time", "forecast_issue_time", "position",
            "old_commitment_kwh", "new_commitment_kwh", "sensitivity_old_commitment_kwh",
            "sensitivity_new_commitment_kwh", "decision_basis"]
        with tempfile.TemporaryDirectory() as directory:
            path = _import_((pathlib).Path(directory) / "decision_versions.csv")
            with path.open("w", newline="", encoding="utf-8") as handle:
                writer = csv.DictWriter(handle, fieldnames=fields)
                writer.writeheader()

```

```

        writer.writerow({"date": "2025-02-01", "mode": "q3", "scenario": "S3", "model_arm": "all_three",
                        "decision_time": "2025-02-01 06:00", "forecast_issue_time": "2025-02-01 12:00",
                        "position": 37, "old_commitment_kwh": 1, "new_commitment_kwh": 2,
                        "sensitivity_old_commitment_kwh": 1, "sensitivity_new_commitment_kwh": 2,
                        "decision_basis": "next_block_stochastic"})
    _, scan = load_versions(__import__("pathlib").Path(directory), "result3.xlsx",
                          {(date(2025, 2, 1), "06:00")})
    self.assertEqual(scan["forecast_release"]["checked_rows"], 1)
    self.assertEqual(scan["forecast_release"]["decision_time_mismatches"], 1)
    self.assertEqual(scan["forecast_release"]["attachment_release_missing"], 1)

def test_next_day_preview_requires_future_target_blank_commitments_and_nonnegative_grid(self):
    fields = ["date", "mode", "scenario", "model_arm", "decision_time", "forecast_issue_time", "position",
              "target_date", "preview_grid_kwh", "old_commitment_kwh", "new_commitment_kwh",
              "sensitivity_old_commitment_kwh", "sensitivity_new_commitment_kwh", "decision_basis",
              "is_preview_next_day", "is_preview_recourse"]
    with tempfile.TemporaryDirectory() as directory:
        path = __import__("pathlib").Path(directory) / "decision_versions.csv"
        with path.open("w", newline="", encoding="utf-8") as handle:
            writer = csv.DictWriter(handle, fieldnames=fields)
            writer.writeheader()
            writer.writerow({"date": "2025-02-01", "mode": "q3", "scenario": "S3", "model_arm": "all_three",
                            "decision_time": "2025-02-01 06:00", "forecast_issue_time": "2025-02-01 06:00",
                            "position": 1, "target_date": "2025-02-01", "preview_grid_kwh": -0.1,
                            "old_commitment_kwh": "", "new_commitment_kwh": "",
                            "sensitivity_old_commitment_kwh": "", "sensitivity_new_commitment_kwh": "",
                            "decision_basis": "next_day_preview", "is_preview_next_day": 1,
                            "is_preview_recourse": 1})
    _, scan = load_versions(__import__("pathlib").Path(directory), "result3.xlsx")
    preview = scan["preview_next_day_integrity"]
    self.assertEqual(preview["rows"], 1)
    self.assertEqual(preview["commitment_violations"], 0)
    self.assertEqual(preview["target_date_violations"], 1)
    self.assertEqual(preview["preview_grid_violations"], 1)
    self.assertFalse(preview["sequence_exact"])

if __name__ == "__main__":
    unittest.main()

```